Computer Vision

Bogdan Alexe

bogdan.alexe@fmi.unibuc.ro

University of Bucharest, 2nd semester, 2025-2026

Course structure

1. **Low-level vision: Features and filters**

   Linear filters, color, texture, edge detection, template matching

2. **Mid-level vision: Grouping and fitting**

   Fitting curves and lines, robust fitting, RANSAC, Hough transform, segmentation

3. **Mid-level vision: Multiple views**

   Local invariant feature and description, epipolar geometry and stereo, object instance recognition

4. **High-level vision: Object recognition**

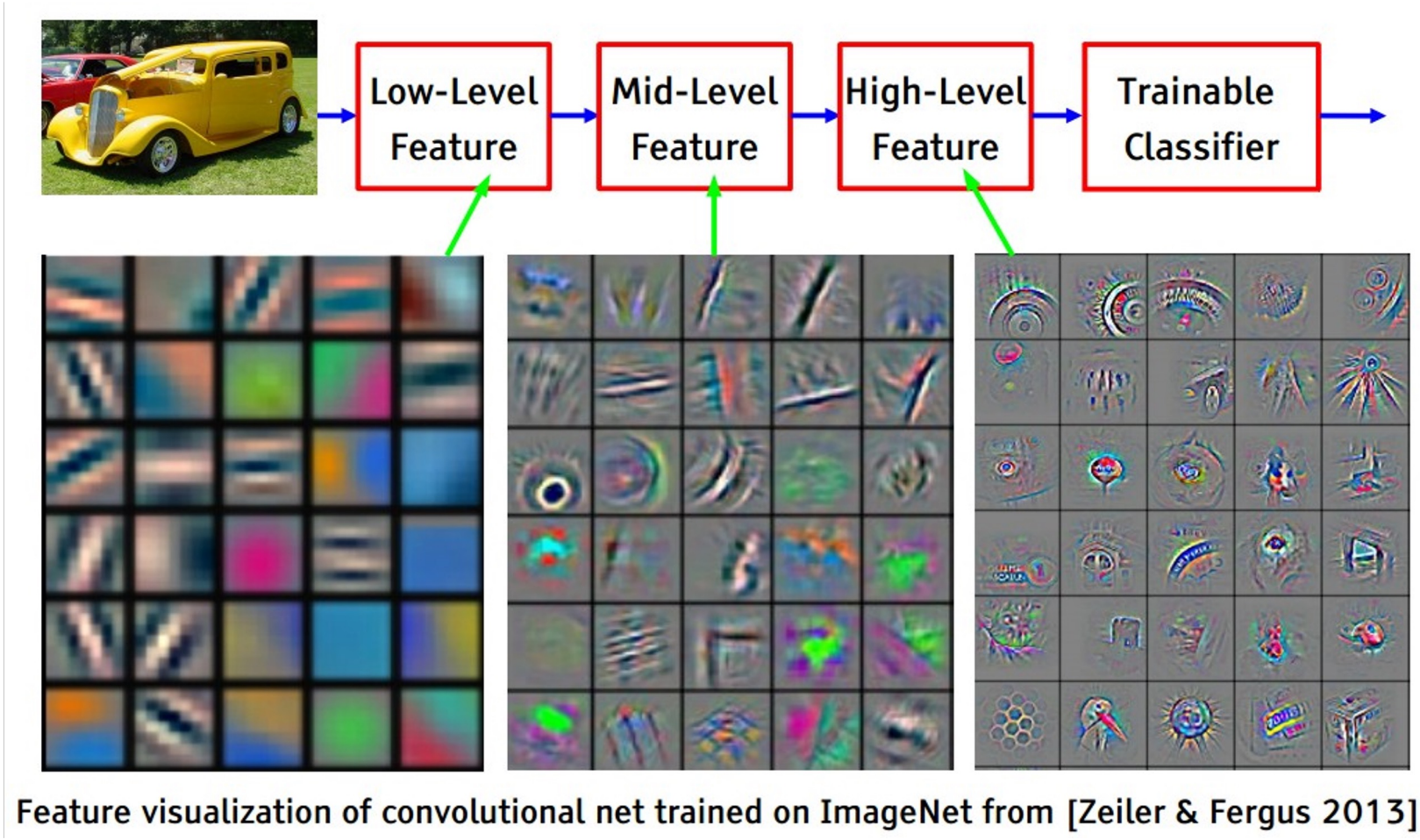
   Object classification, object detection, part based models, bovw models

5. **High-level vision: Video understanding**

   Object tracking, background subtraction, motion descriptors, optical flow

Gradients & edges

Visualization of CNN Feature Maps



Feature visualization of convolutional net trained on ImageNet from [Zeiler & Fergus 2013]

First layers → Detect edges and simple textures.
Middle layers → Capture textures and object parts.
Final layers → Recognize entire objects.

[Slide credit: Yann LeCun]

Images can naturally be decomposed in:
pixel → edge → texton → superpixel → part → object

1.Pixel

-the smallest unit of an image, a single point with a specific color (or intensity in grayscale).
-each pixel doesn't carry much information, it's only when grouping pixels together meaningful structures emerge.

2.Edge

-an edge is a boundary where there is a sharp change in color or intensity.
- by detecting edges, we start to see outlines and boundaries of shapes, the first step in for higher-level features.

3.Texton

-a "texton" is a small, fundamental visual pattern (like a local texture element) which often repeats in a region. For example, the basic brick shape in a brick wall, or a small patch of repeated grain in wood texture.
-textons capture local texture statistic, recognizing them help moving beyond just raw edges to textures/patterns.

4.Superpixel

-a cluster of pixels (often hundreds or thousands) that share similar color or texture. Instead of looking at each individual pixel, you group them into a larger, more meaningful "patch."
-superpixels simplify the image representation. Each superpixel represents a region that's uniform in some way (color, texture, brightness), which makes further processing more efficient and structured.

5.Part

-a significant portion of an object. For instance, if you have a face, "parts" would be the eyes, nose, or mouth; if you have a car, "parts" might be the wheels, doors, or windshield.
-once you group superpixels into larger, functionally relevant segments, these start to represent meaningful "parts" of an object. Parts are closer to what we humans intuitively recognize.

6.Object

-the complete, recognizable entity in the scene (e.g., a person, a car, a tree).
-combining all the parts (which are themselves made up of superpixels, which in turn come from pixels) yields the full, identifiable object.

Images can naturally be decomposed in:
pixel → edge → texton → superpixel → part → object

1.Pixel

-the smallest unit of an image, a single point with a specific color (or intensity in grayscale).
-each pixel doesn't carry much information, it's only when grouping pixels together meaningful structures emerge.

2.Edge

-an edge is a boundary where there is a sharp change in color or intensity.
- by detecting edges, we start to see outlines and boundaries of shapes, the first step in for higher-level features.

3.Texton

-a "texton" is a small, fundamental visual pattern (like a local texture element) which often repeats in a region. For example, the basic brick shape in a brick wall, or a small patch of repeated grain in wood texture.
-textons capture local texture statistic, recognizing them help moving beyond just raw edges to textures/patterns.

4.Superpixel

-a cluster of pixels (often hundreds or thousands) that share similar color or texture. Instead of looking at each individual pixel, you group them into a larger, more meaningful "patch."
-superpixels simplify the image representation. Each superpixel represents a region that's uniform in some way (color, texture, brightness), which makes further processing more efficient and structured.

5.Part

-a significant portion of an object. For instance, if you have a face, "parts" would be the eyes, nose, or mouth; if you have a car, "parts" might be the wheels, doors, or windshield.
-once you group superpixels into larger, functionally relevant segments, these start to represent meaningful "parts" of an object. Parts are closer to what we humans intuitively recognize.

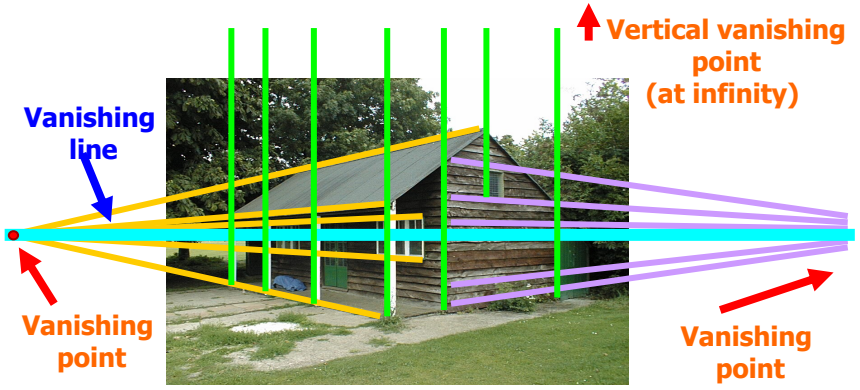
6.Object

-the complete, recognizable entity in the scene (e.g., a person, a car, a tree).
-combining all the parts (which are themselves made up of superpixels, which in turn come from pixels) yields the full, identifiable object.

Why do we care about edges?

- Extract information, recognize objects

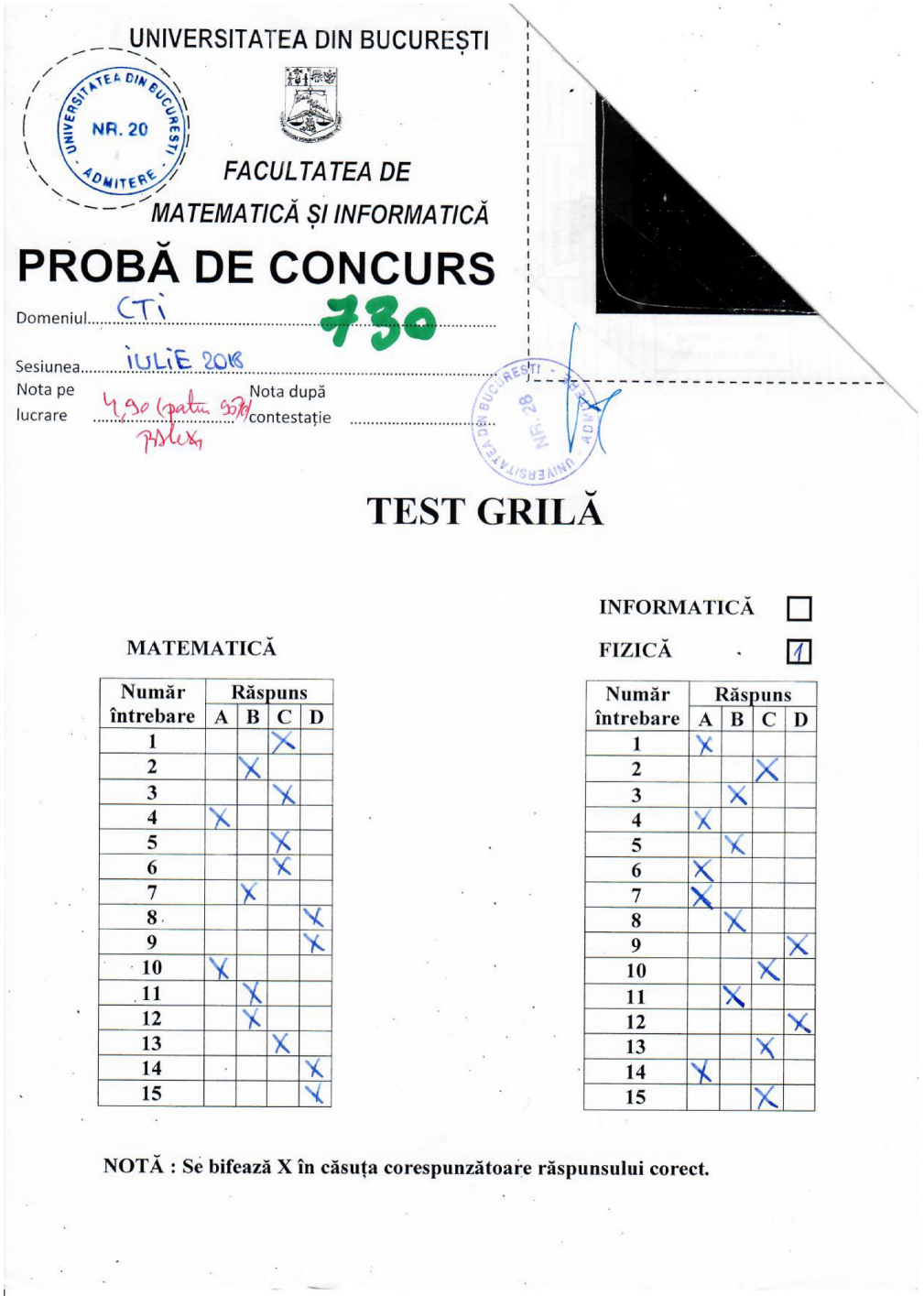
- Recover geometry and viewpoint



Lab class 2
Automatic grading of multiple choice tests

- a simpler version of a past project

https://tinyurl.com/CV-2020-Project1



UNIVERSITATEA DIN BUCUREȘTI

FACULTATEA DE MATEMATICĂ ȘI INFORMATICĂ

PROBĂ DE CONCURS

Domeniul CTI 730

Sesiunea IULIE 2018

Nota pe lucrare 4,90 (peste 3?) Nota după contestație

TEST GRILĂ

INFORMATICĂ ☐

FIZICĂ ☑

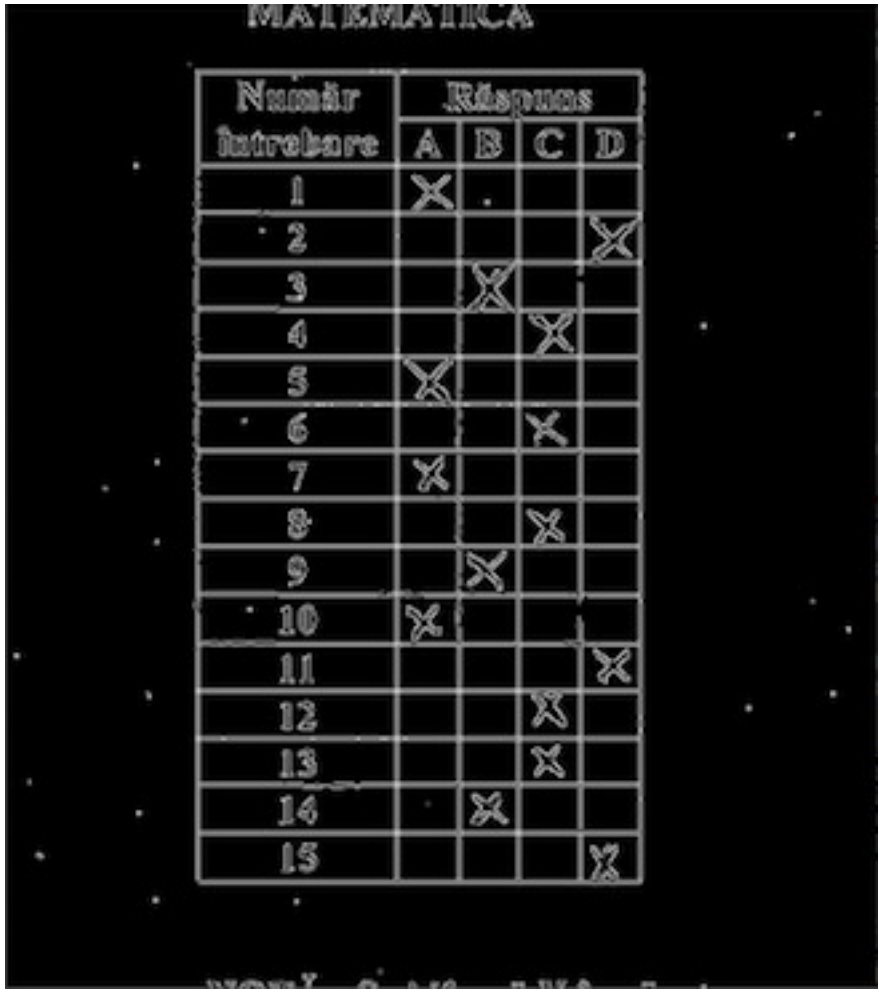
MATEMATICĂ

Număr întrebare	Răspuns A	B	C	D
1			X	
2		X		
3			X	
4	X			
5			X	
6			X	
7		X		
8				X
9				X
10	X			
11		X		
12		X		
13			X	
14				X
15				X

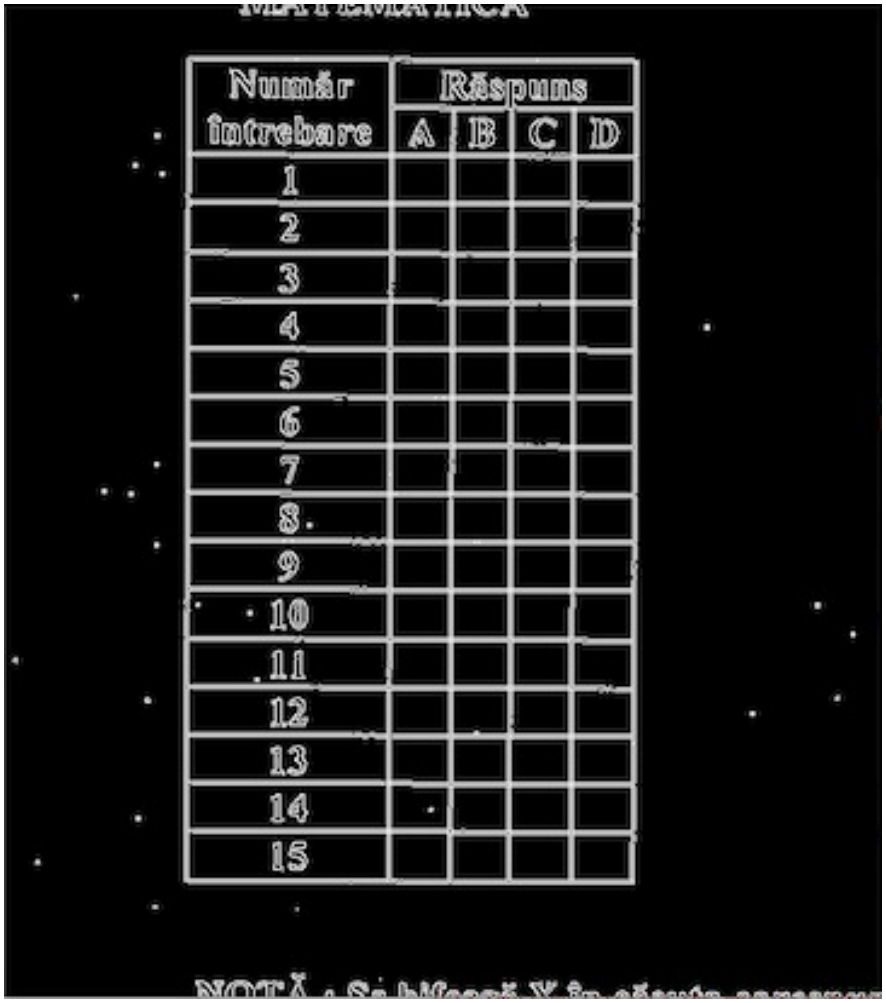
Număr întrebare	Răspuns A	B	C	D
1	X			
2			X	
3		X		
4	X			
5		X		
6	X			
7	X			
8		X		
9				X
10			X	
11		X		
12				X
13			X	
14	X			
15			X	

NOTĂ : Se bifează X în căsuța corespunzătoare răspunsului corect.

Lab class 2 – Automatic grading of multiple choice tests



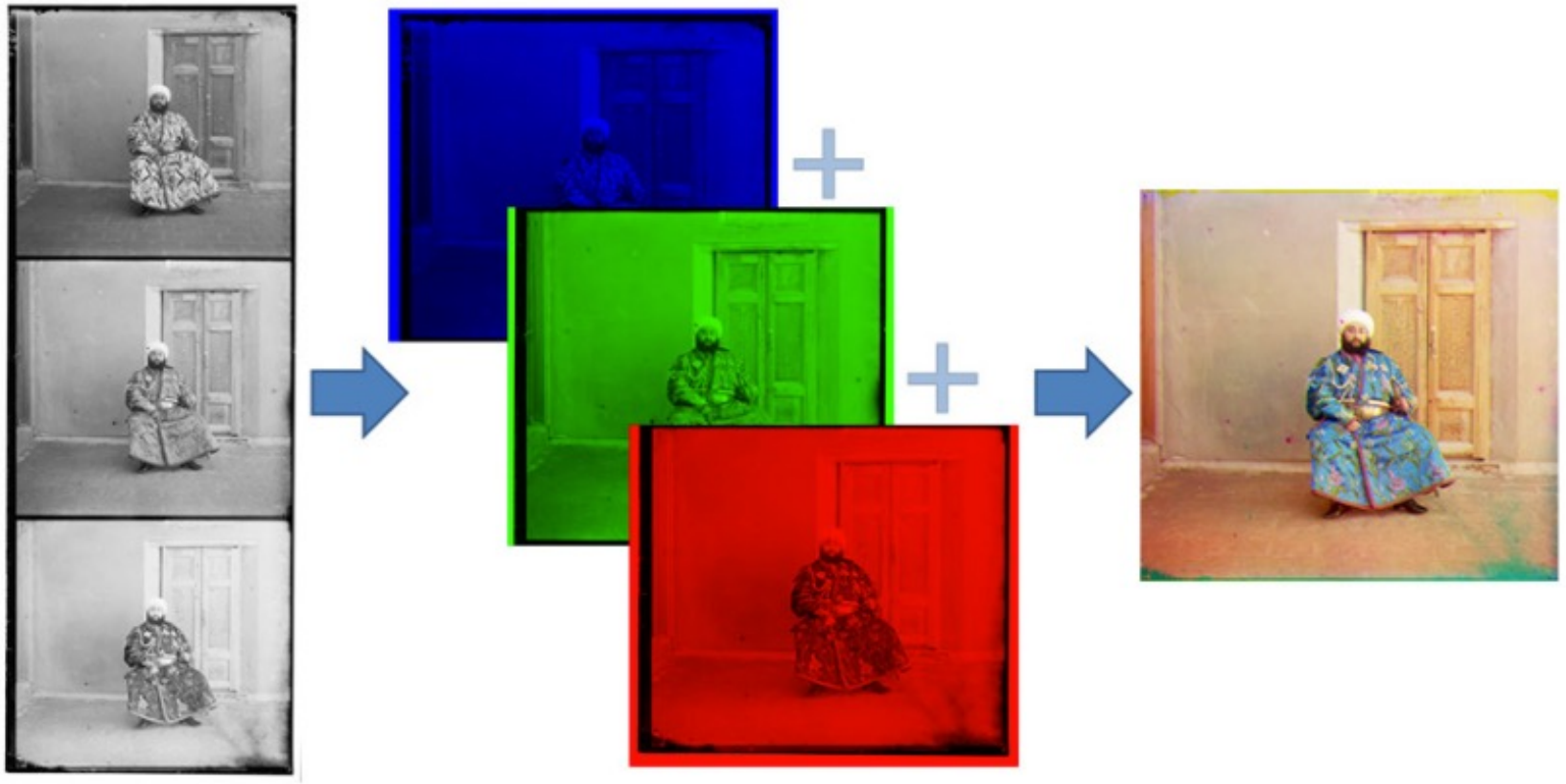
Edge image for query



Edge image for template

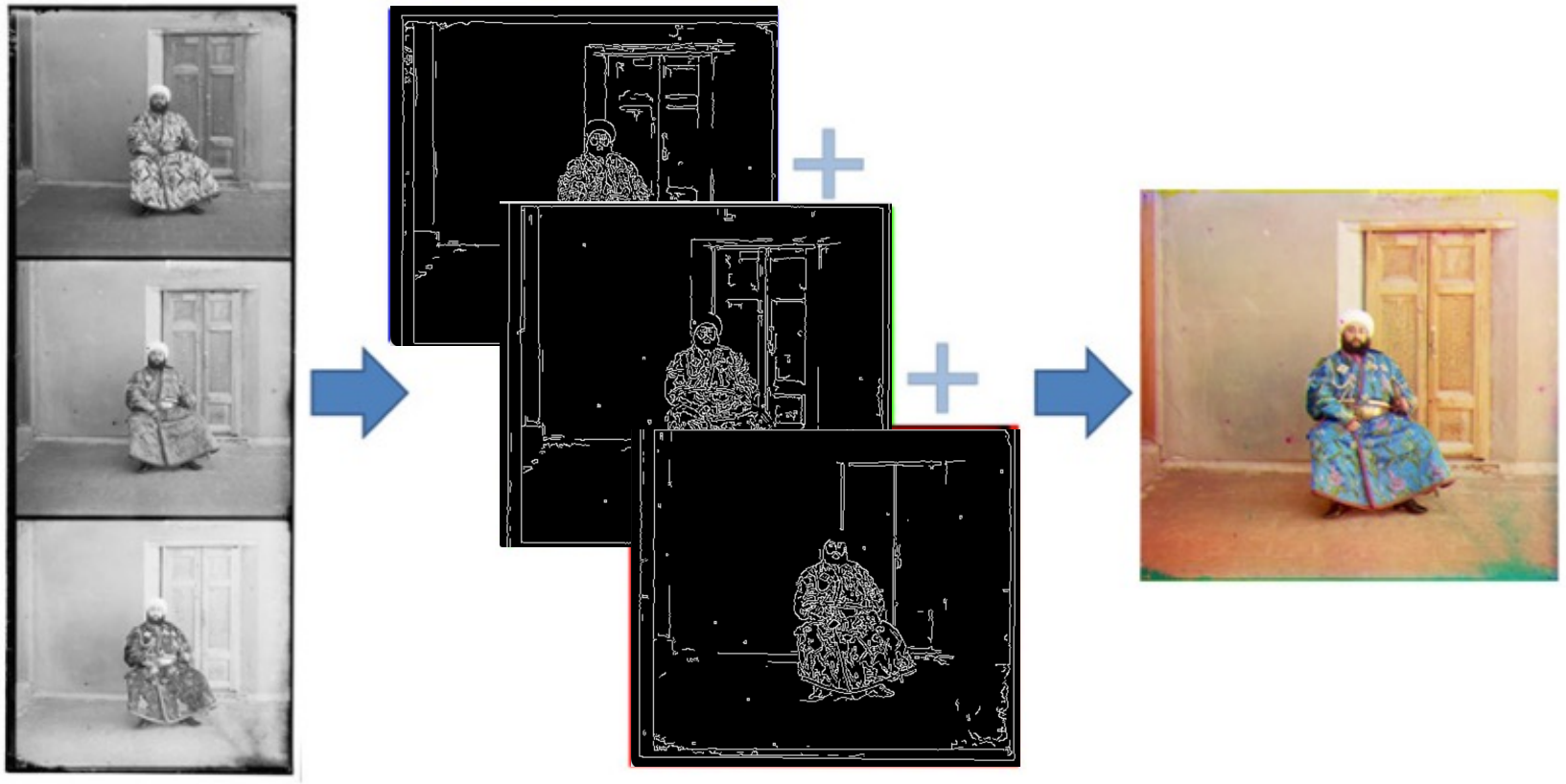
Lab class 1 – Aligning channels in glass plate images

Your first application in Computer Vision

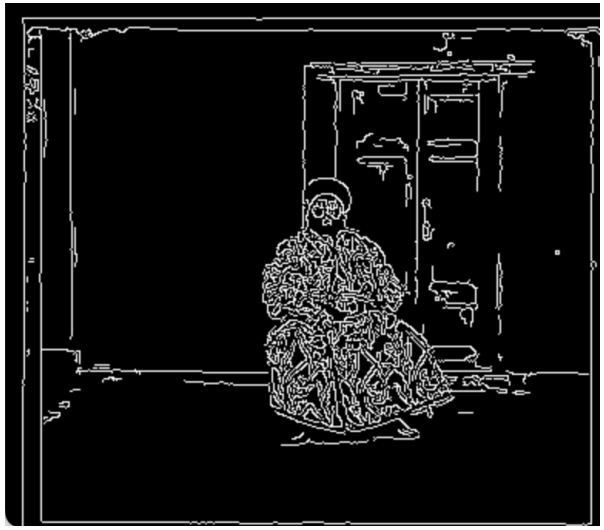


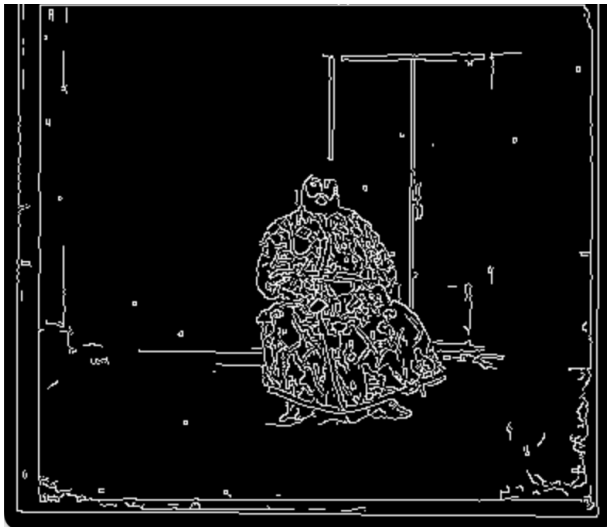
Lab class 1 – Aligning channels in glass plate images

Your first application in Computer Vision

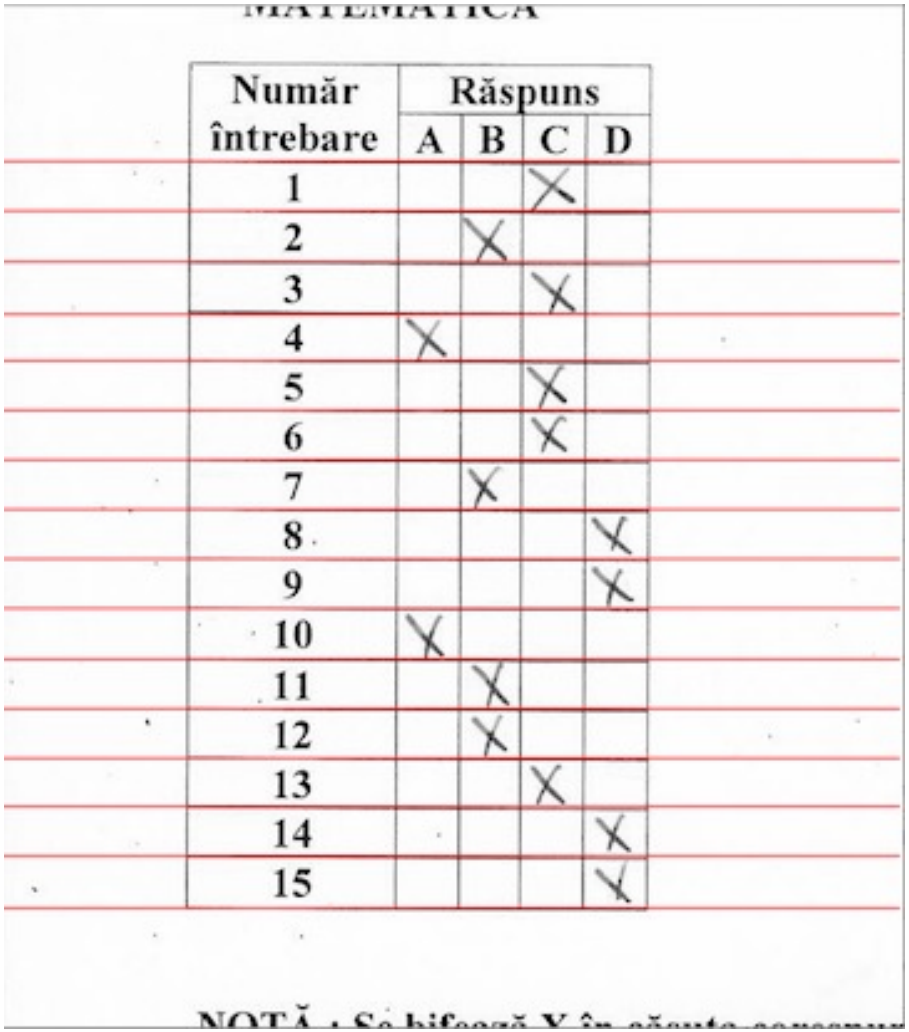


Lab class 1 – Aligning channels in glass plate images



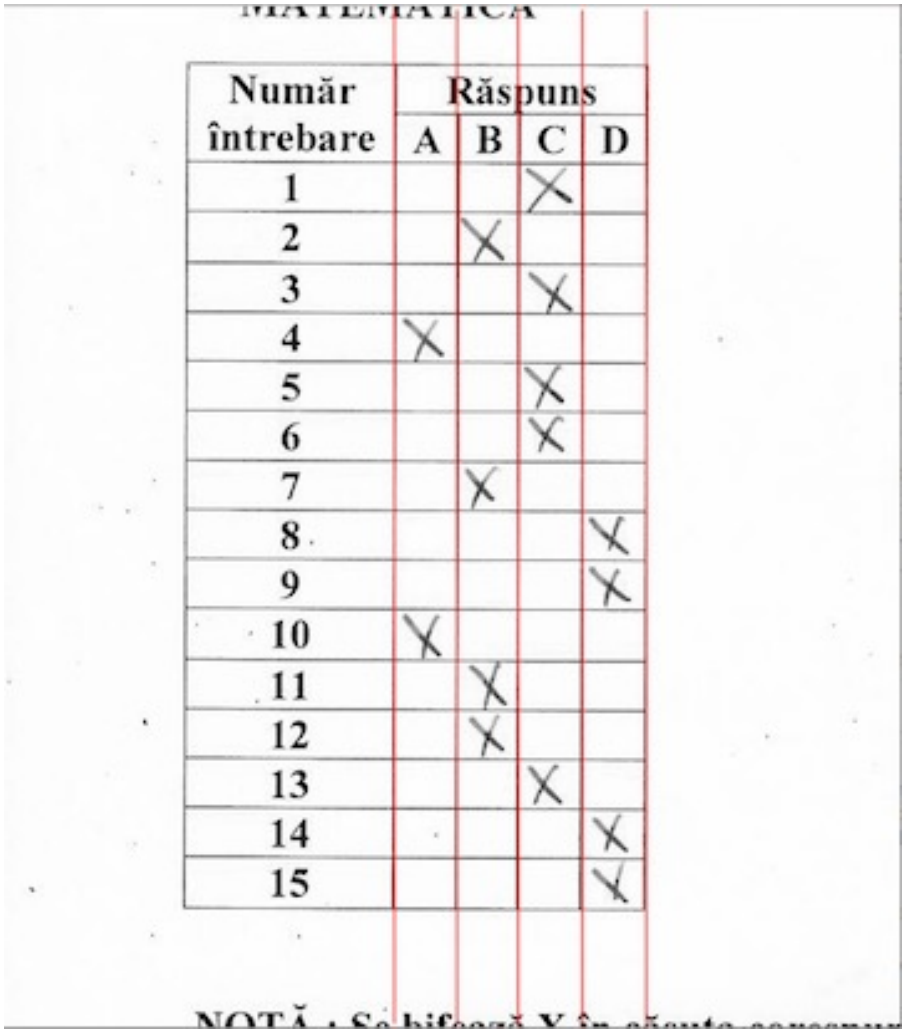


Lab class 2 – Automatic grading of multiple choice tests



Număr întrebare	Răspuns			
	A	B	C	D
1			X	
2		X		
3			X	
4	X			
5			X	
6			X	
7		X		
8				X
9				X
10	X			
11		X		
12		X		
13			X	
14				X
15				X

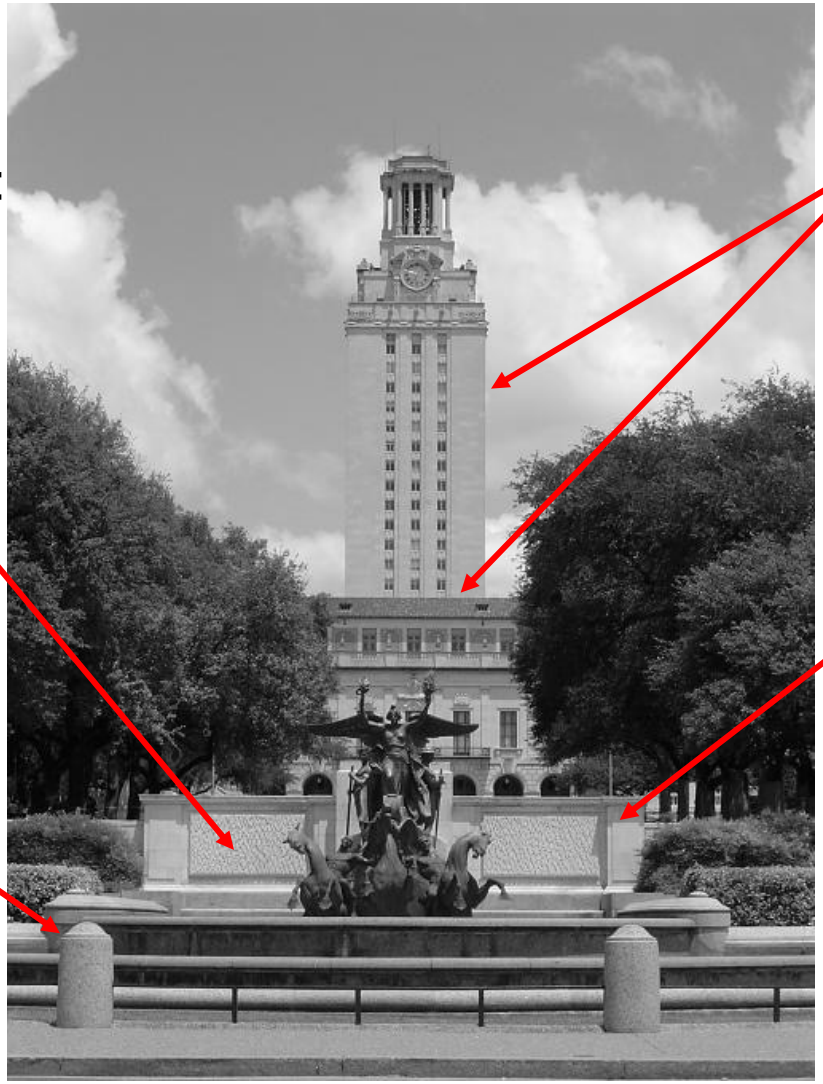
Horizontal lines found with Hough transform



Număr întrebare	Răspuns			
	A	B	C	D
1			X	
2		X		
3			X	
4	X			
5			X	
6			X	
7		X		
8				X
9				X
10	X			
11		X		
12		X		
13			X	
14				X
15				X

Vertical lines found with Hough transform

What causes an edge?



Reflectance change: appearance information, texture

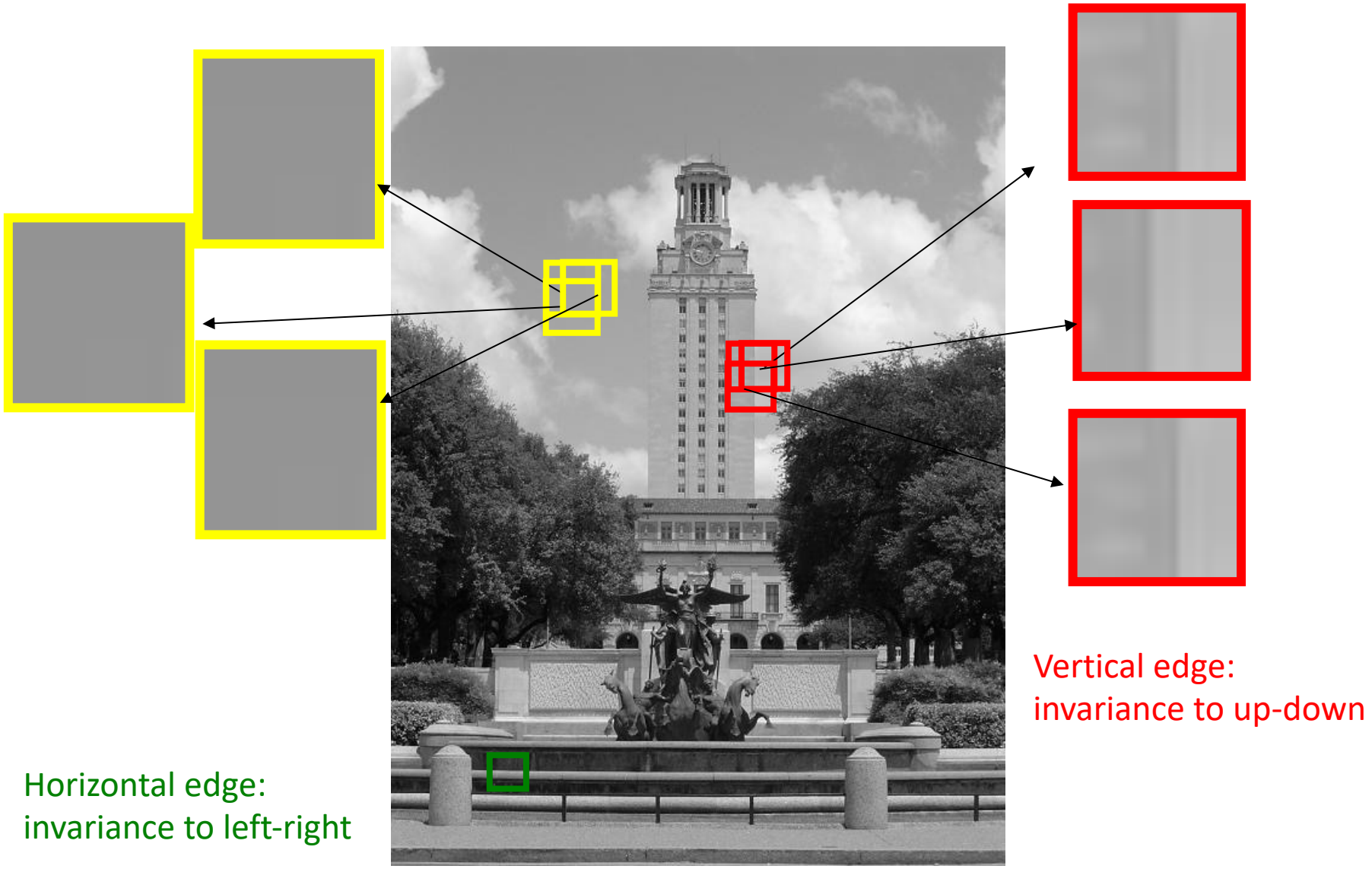
Depth discontinuity: object boundary

Cast shadows

Change in surface orientation: shape

Slide credit: Kristen Grauman

Edges/gradients and invariance



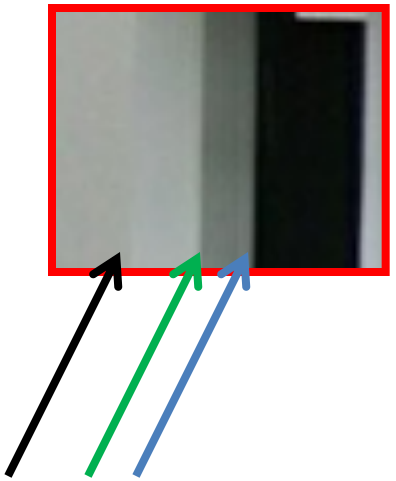
Horizontal edge:
invariance to left-right

Vertical edge:
invariance to up-down

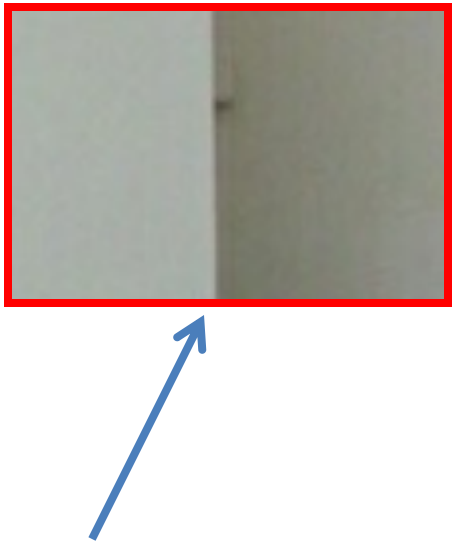
Slide credit: Kristen Grauman

Closeup of edges

Closeup of edges



Closeup of edges

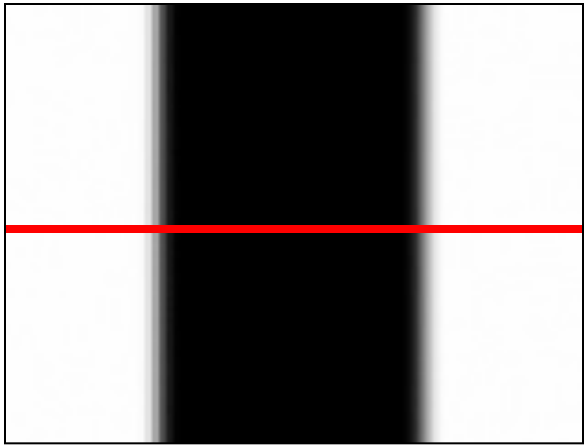


Closeup of edges

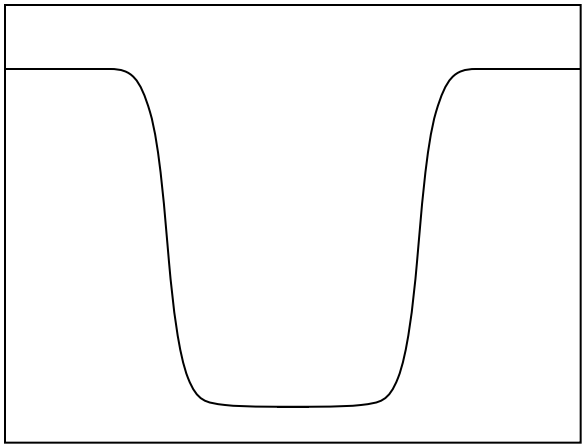
Characterizing edges

- An edge is a place of rapid change in the image intensity function

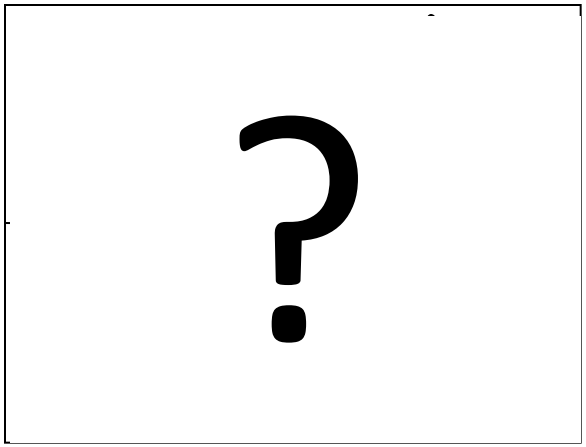
image



intensity function
(along horizontal scanline)



first derivative



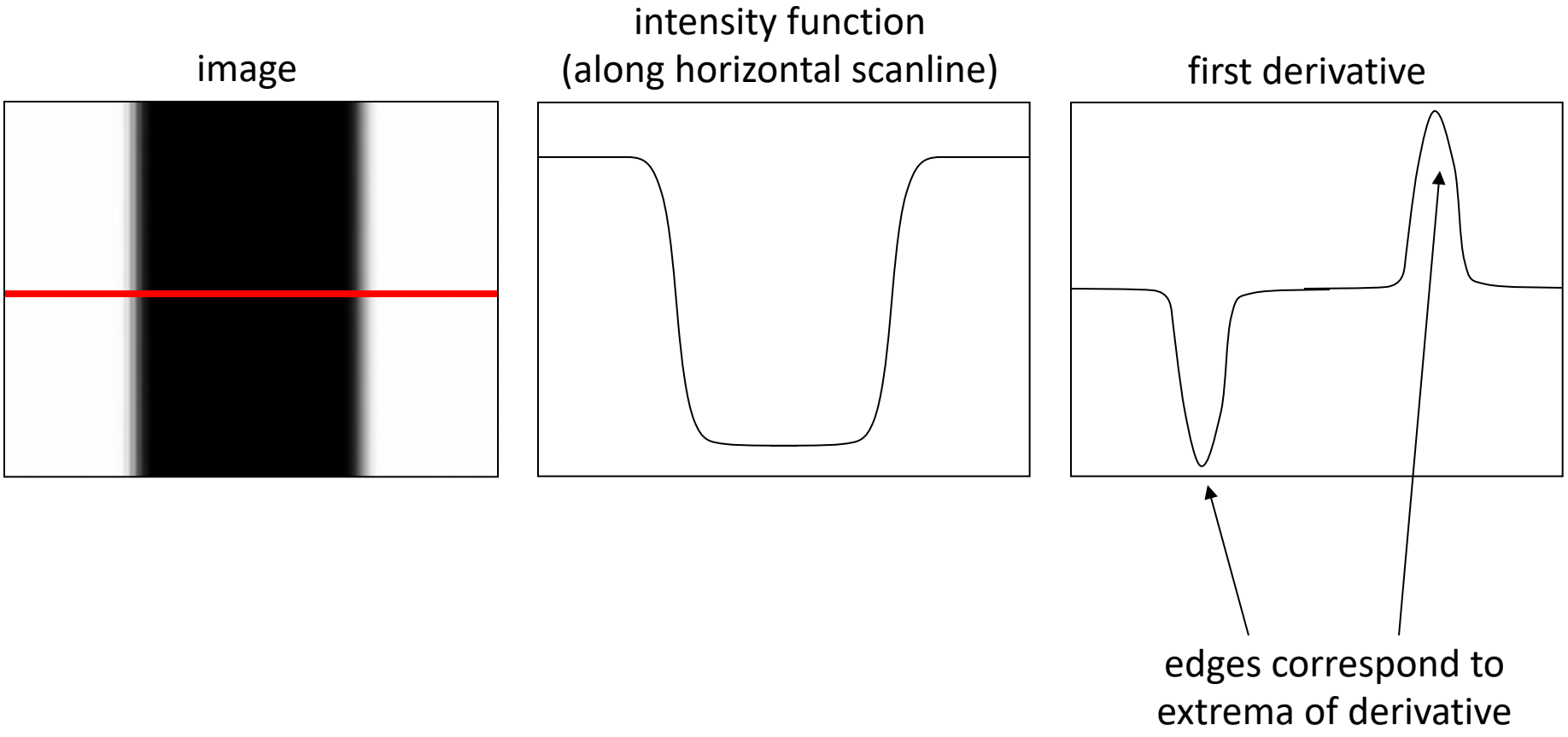
Derivative of a function

- slope of the tangent line to the graph of the function at that point
- **green > 0**
- **black = 0**
- **red < 0**

https://en.wikipedia.org/wiki/Derivative

Characterizing edges

- An edge is a place of rapid change in the image intensity function



Derivatives with convolution

For 2D function, f(x,y), the partial derivative is:

$$\frac{\partial f(x,y)}{\partial x} = \lim_{\varepsilon \to 0} \frac{f(x+\varepsilon, y) - f(x,y)}{\varepsilon}$$

For discrete data, we can approximate using finite differences:

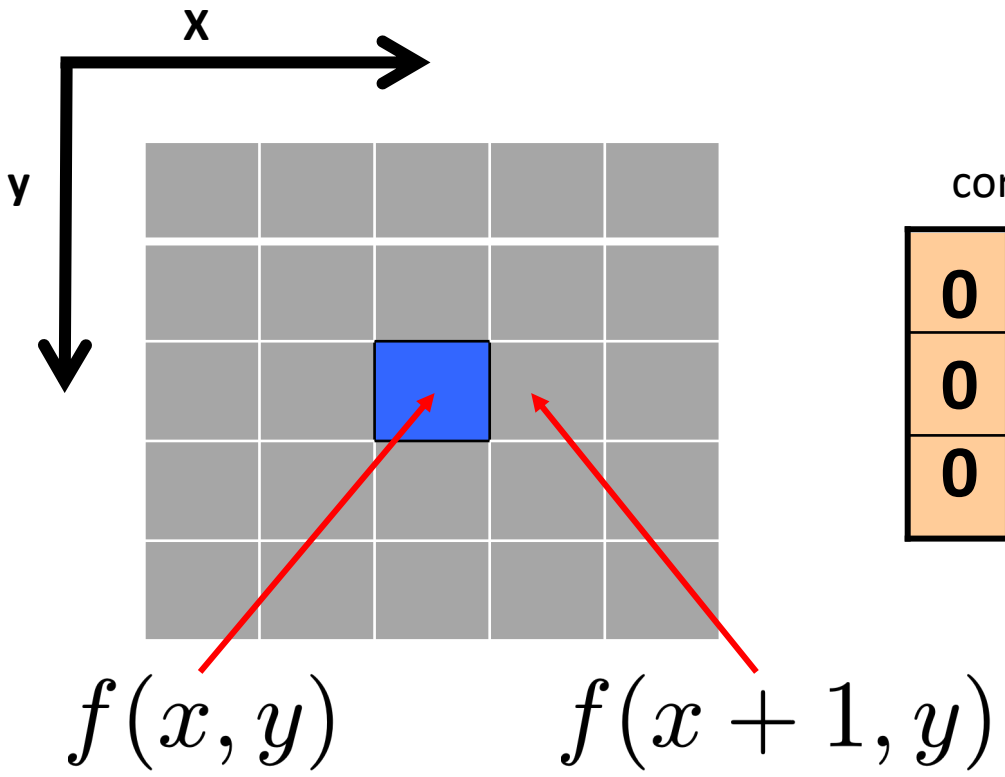
$$\frac{\partial f(x,y)}{\partial x} \approx \frac{f(x+1, y) - f(x,y)}{1}$$

To implement above as convolution, what would be the associated filter?

Slide credit: Kristen Grauman

Derivatives with convolution

$$\frac{\partial f(x,y)}{\partial x} \approx \frac{f(x+1,y) - f(x,y)}{1}$$



correlation

0	0	0
0	-1	1
0	0	0

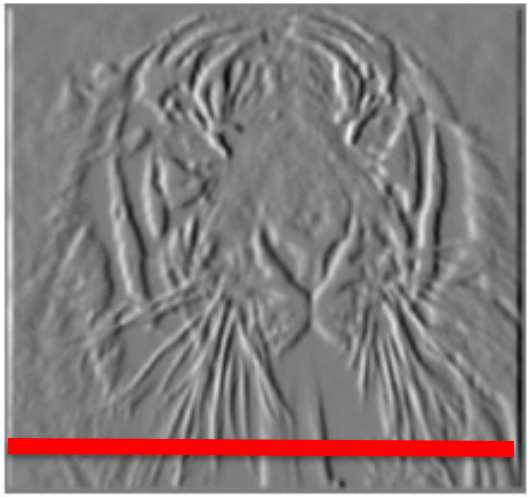
convolution

1	-1	

Slide credit: Kristen Grauman

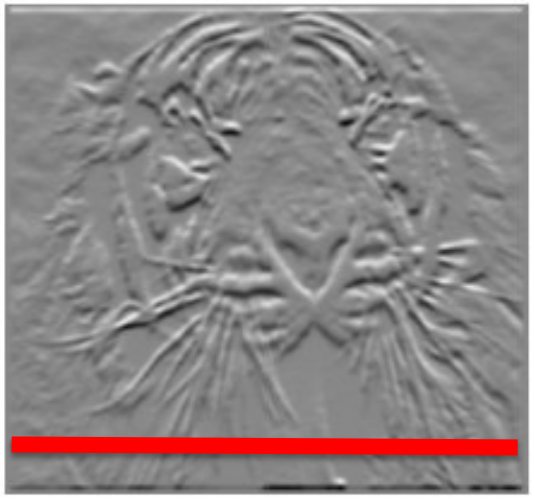
Partial derivatives of an image

$$\frac{\partial f(x,y)}{\partial x}$$



-1	1

$$\frac{\partial f(x,y)}{\partial y}$$



-1
1

Which shows changes with respect to x?

(showing filters for correlation)

Slide credit: Kristen Grauman

Sobel filters

There exists other approximations for computing partial derivatives:

- Sobel filters: compute derivatives considering larger nieghborhoods (3×3)

-1	0	1
-2	0	2
-1	0	1

Vertical Sobel filter for computing the partial derivative

$$\frac{\partial f(x,y)}{\partial x}$$

1	2	1
0	0	0
-1	-2	-1

Horizontal Sobel filter for computing the partial derivative

$$\frac{\partial f(x,y)}{\partial y}$$

Slide credit: Kristen Grauman

Other filters for finite diferences

Prewitt: $M_x = \begin{bmatrix} -1 & 0 & 1 \\ -1 & 0 & 1 \\ -1 & 0 & 1 \end{bmatrix}$; $M_y = \begin{bmatrix} 1 & 1 & 1 \\ 0 & 0 & 0 \\ -1 & -1 & -1 \end{bmatrix}$

Sobel: $M_x = \begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix}$; $M_y = \begin{bmatrix} 1 & 2 & 1 \\ 0 & 0 & 0 \\ -1 & -2 & -1 \end{bmatrix}$

Roberts: $M_x = \begin{bmatrix} 0 & 1 \\ -1 & 0 \end{bmatrix}$; $M_y = \begin{bmatrix} 1 & 0 \\ 0 & -1 \end{bmatrix}$

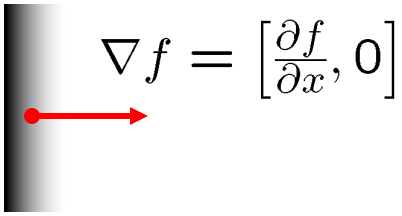
```
img = cv2.imread('simona.jpg',0)
sobelx = cv2.Sobel(img,cv2.CV_64F,1,0,ksize=3)
plt.imshow(sobelx,cmap = 'gray')
plt.show()
```

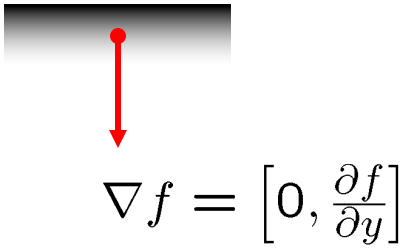
Image gradient

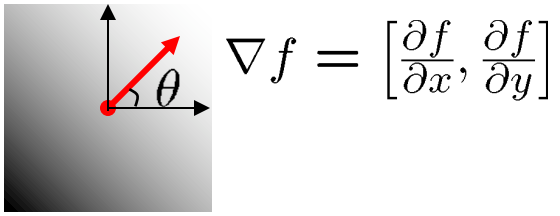
The gradient of an image:

$$\nabla f = \left[\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y}\right]$$

The gradient points in the direction of most rapid change in intensity





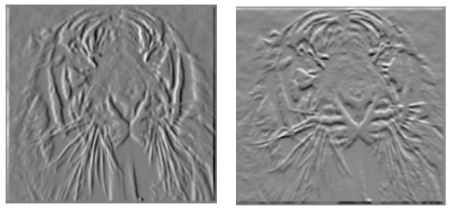


The **gradient direction** (orientation of edge normal) is given by:

$$\theta = \tan^{-1}\left(\frac{\partial f}{\partial y} / \frac{\partial f}{\partial x}\right)$$

The **edge strength** is given by the gradient magnitude

$$\|\nabla f\| = \sqrt{\left(\frac{\partial f}{\partial x}\right)^2 + \left(\frac{\partial f}{\partial y}\right)^2}$$



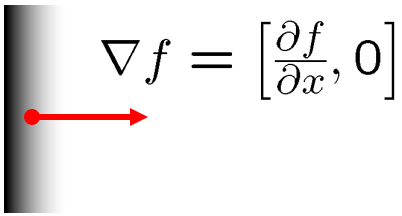
Slide credit Steve Seitz

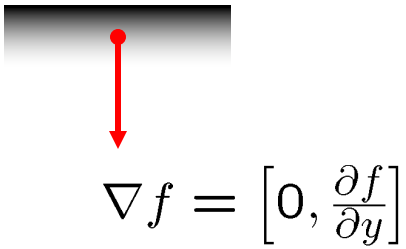
Image gradient

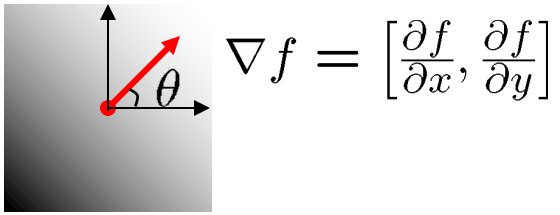
The gradient of an image:

$$\nabla f = \left[\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y}\right]$$

The gradient points in the direction of most rapid change in intensity







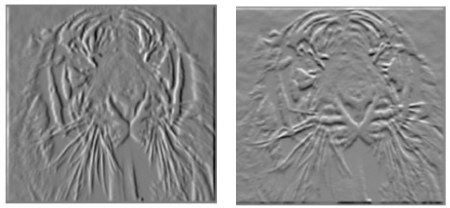
The **gradient direction** (orientation of edge normal) is given by:

$$\theta = \tan^{-1}\left(\frac{\partial f}{\partial y} / \frac{\partial f}{\partial x}\right)$$

The **edge strength** is given by the gradient magnitude

$$\|\nabla f\| = \sqrt{\left(\frac{\partial f}{\partial x}\right)^2 + \left(\frac{\partial f}{\partial y}\right)^2}$$

or $$\|\nabla f\| = \left|\frac{\partial f}{\partial x}\right| + \left|\frac{\partial f}{\partial y}\right|$$



Slide credit Steve Seitz

Computing the image gradient

1. compute the partial derivatives:

$$\frac{\partial f(x,y)}{\partial x} \qquad \frac{\partial f(x,y)}{\partial y}$$

2. compute the gradient magnitude:

$$\|\nabla f\| = \sqrt{\left(\frac{\partial f}{\partial x}\right)^2 + \left(\frac{\partial f}{\partial y}\right)^2} \quad \text{or} \quad \|\nabla f\| = \left|\frac{\partial f}{\partial x}\right| + \left|\frac{\partial f}{\partial y}\right|$$

Computing the image gradient

1. compute the partial derivative $\frac{\partial f(x,y)}{\partial x}$

Use the vertical Sobel filter

$M_x =$

-1	0	1
-2	0	2
-1	0	1

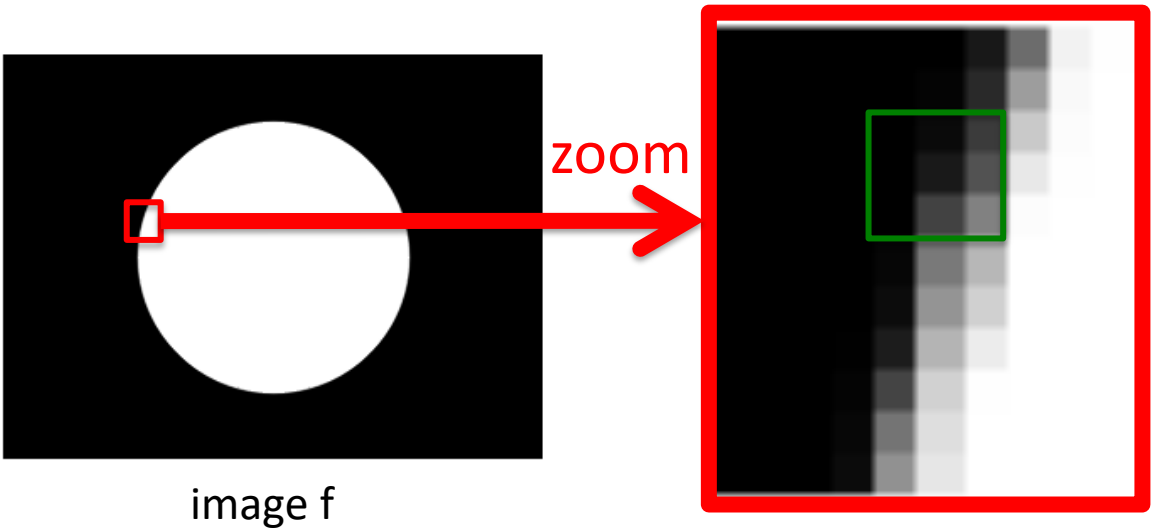
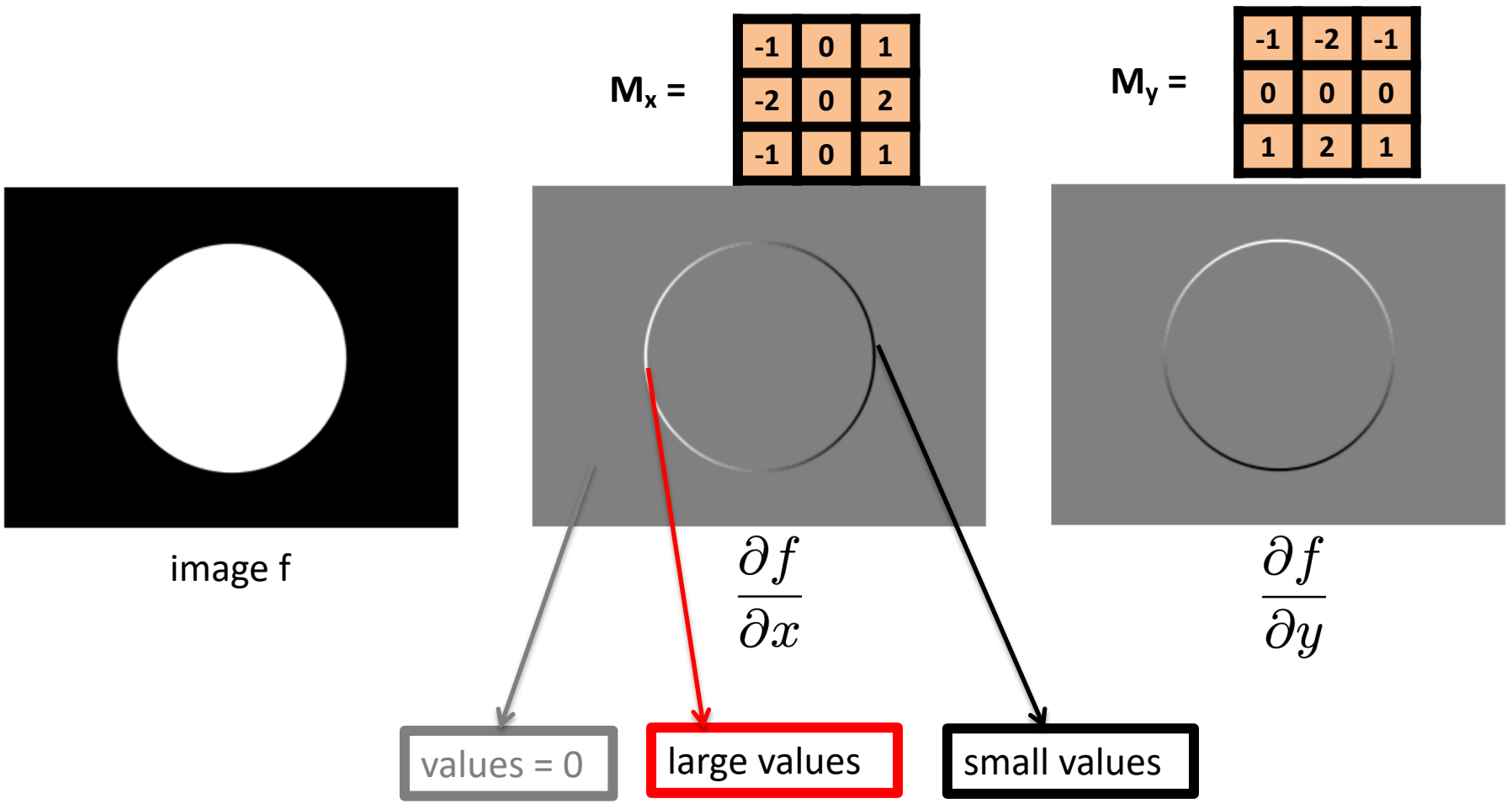


image f

0	0	0	0	4	41	157	249	255
0	0	0	0	10	59	201	252	255
0	0	0	0	25	82	232	254	255
0	0	0	0	66	129	252	255	255
0	0	0	6	121	183	255	255	255
0	0	0	1	148	208	255	255	255
0	0	1	2	180	236	255	255	255
0	0	4	6	209	254	255	255	255
0	0	7	11	222	255	255	255	255

= (-1) * 0 + 0 * 0 + 1 * 25
+
+ (-2) * 0 + 0 * 0 + 2 * 66
+
+ (-1) * 0 + 0 * 6 + 1 * 121
= 278

Computing the image gradient



Computing the image gradient

M_x =

-1	0	1
-2	0	2
-1	0	1

M_y =

-1	-2	-1
0	0	0
1	2	1

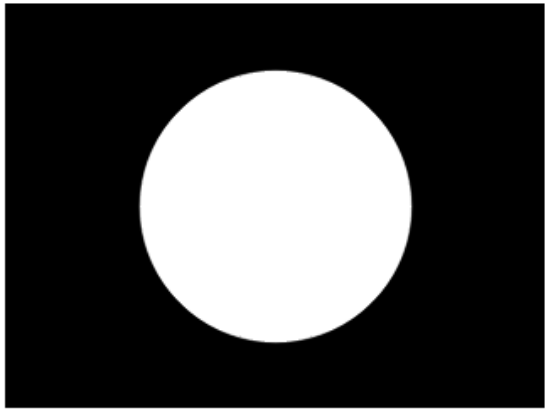
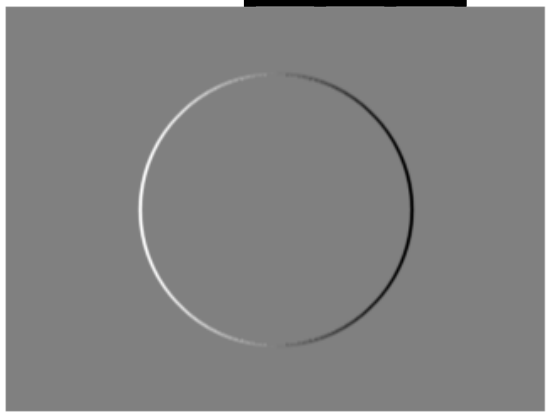
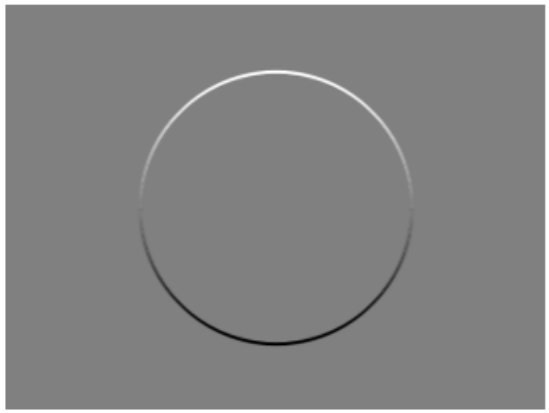


image f



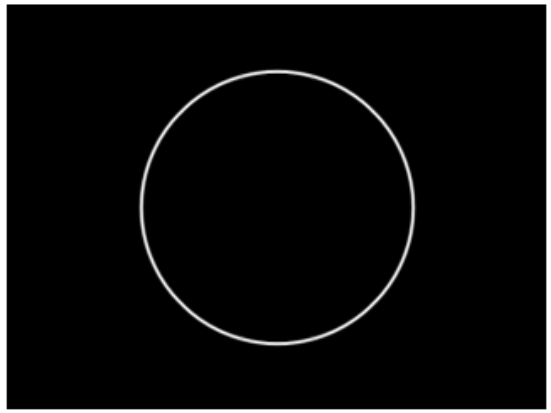
$$\frac{\partial f}{\partial x}$$



$$\frac{\partial f}{\partial y}$$

$$\|\nabla f\| = \sqrt{\left(\frac{\partial f}{\partial x}\right)^2 + \left(\frac{\partial f}{\partial y}\right)^2}$$

Gradient magnitude



Computing the image gradient

M_x =

-1	0	1
-2	0	2
-1	0	1

M_y =

-1	-2	-1
0	0	0
1	2	1

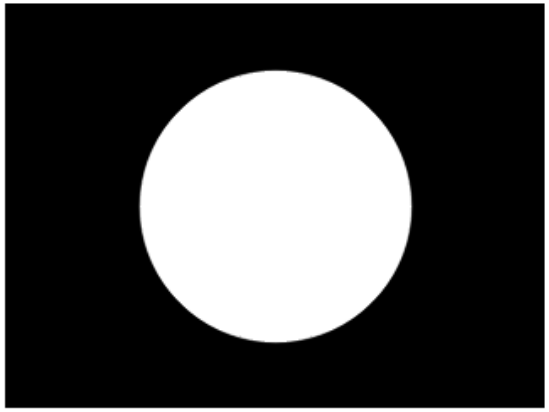
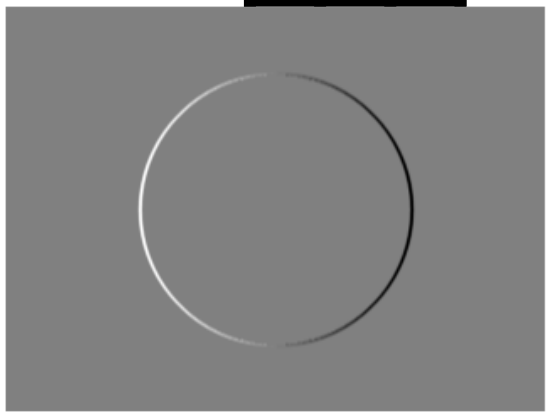
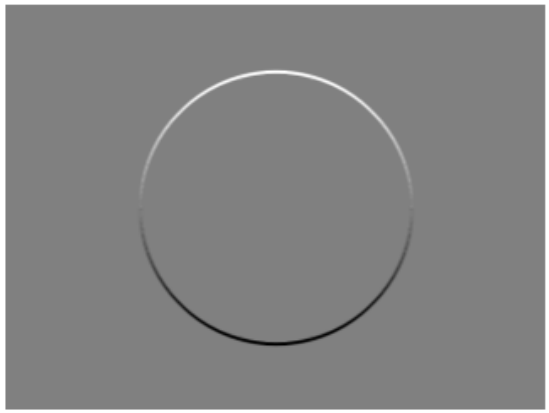


image f



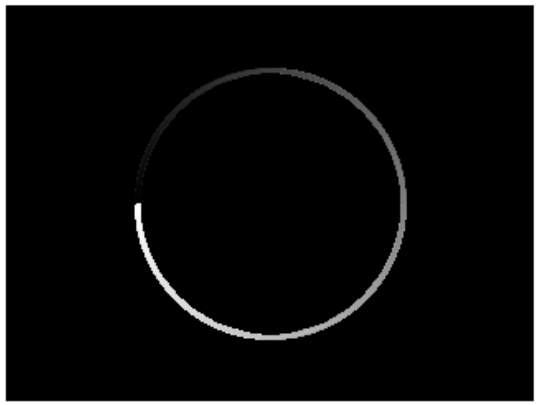
$$\frac{\partial f}{\partial x}$$

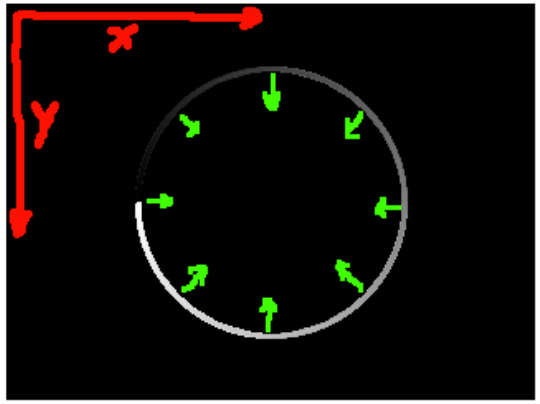


$$\frac{\partial f}{\partial y}$$

$$\theta = \tan^{-1}\left(\frac{\partial f}{\partial y} / \frac{\partial f}{\partial x}\right)$$

Gradient orientation





Computing the image gradient

M_x =

-1	0	1
-2	0	2
-1	0	1

M_y =

-1	-2	-1
0	0	0
1	2	1

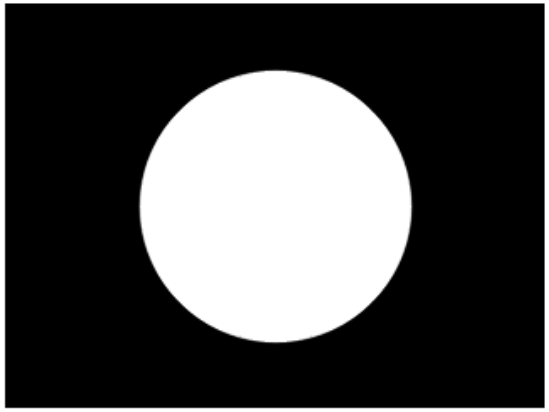
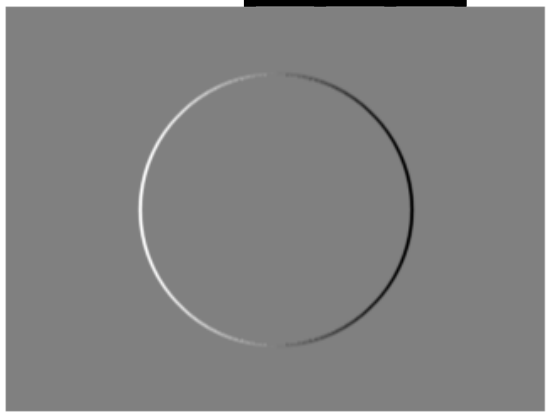
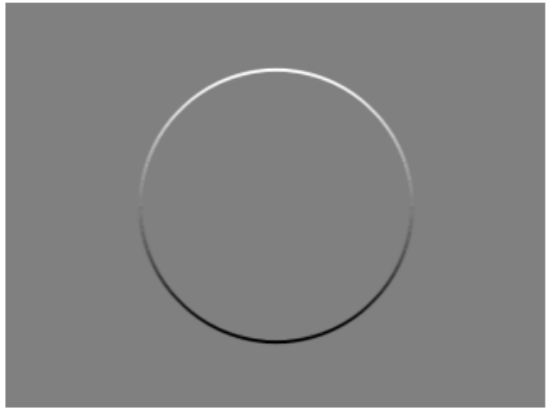


image f



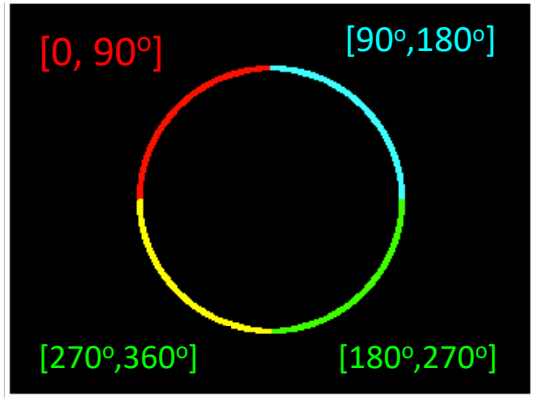
$\frac{\partial f}{\partial x}$

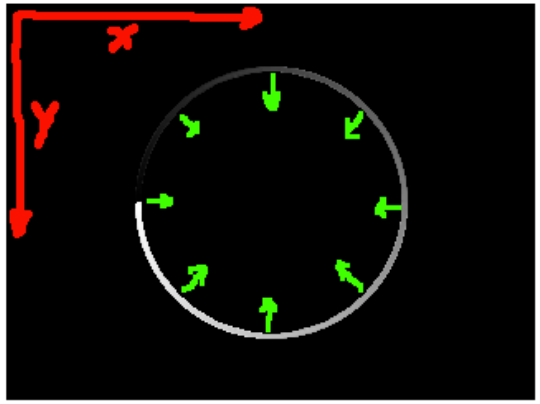


$\frac{\partial f}{\partial y}$

$$\theta = \tan^{-1}\left(\frac{\partial f}{\partial y} / \frac{\partial f}{\partial x}\right)$$

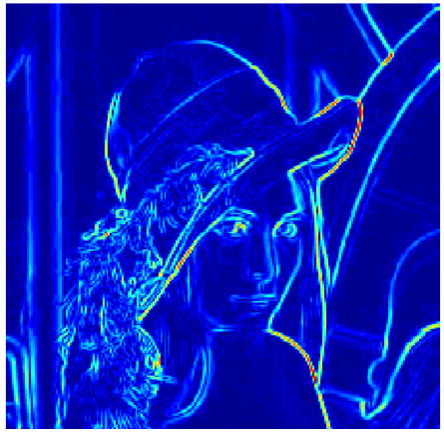
[0, 90°]
[90°, 180°]
[180°, 270°]
[270°, 360°]



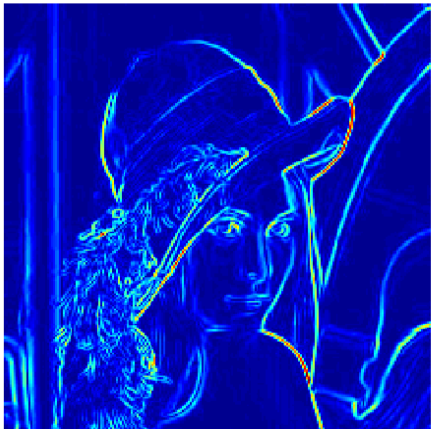


From gradients to edges

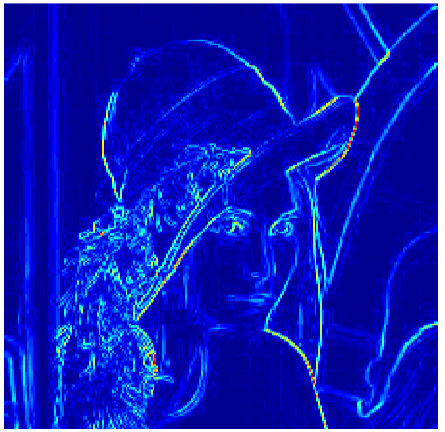
Gradients



Gradients



Gradients



Sobel filter

Roberts filter

Prewitt filter

Edges

Edges

Edges

Edge detection using 1st derivative

Using first-derivative (gradient-based) methods

- these methods compute the gradient of the image = a vector pointing in the direction of the greatest rate of intensity change.
- common filters: Sobel, Prewitt, and Roberts operators.
- because first derivatives respond strongly to noise, it is common to include smoothing, such as applying a Gaussian blur before taking the gradient (e.g., in Canny edge detection).
- steps typically involve:
  - **compute partial derivatives** in the x and y directions (for example, using a Sobel operator).
  - **calculate the gradient magnitude** from those partial derivatives. A large magnitude indicates a strong edge.
  - **threshold the gradient magnitude** to decide whether an edge is present.

Edge detection using 2nd derivative

Using second-derivative methods

- these methods rely on the second derivative, which measures how quickly the first derivative (the gradient) changes.
- a classic second-derivative operator is the Laplacian:

$$\nabla^2 I = \frac{\partial^2 I}{\partial x^2} + \frac{\partial^2 I}{\partial y^2}$$

- since the second derivative amplifies noise, a smoothing step is almost always applied before or as part of taking the Laplacian. This leads to the **Laplacian of Gaussian (LoG)** filter or the closely related **Difference of Gaussians (DoG)**.
- in a second-derivative approach, an edge often corresponds to **zero-crossings** in the filtered image: where the Laplacian output changes sign (changes in sign +- or -+ in a 4 or 8 connected neighborhood).

Edge detection using 1st and 2nd derivative

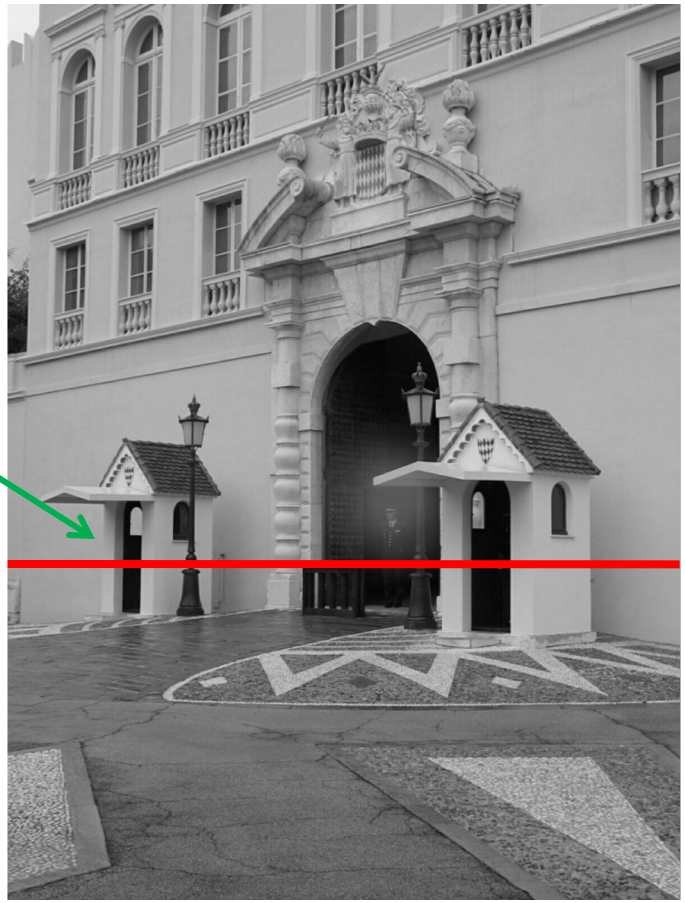
Sobel

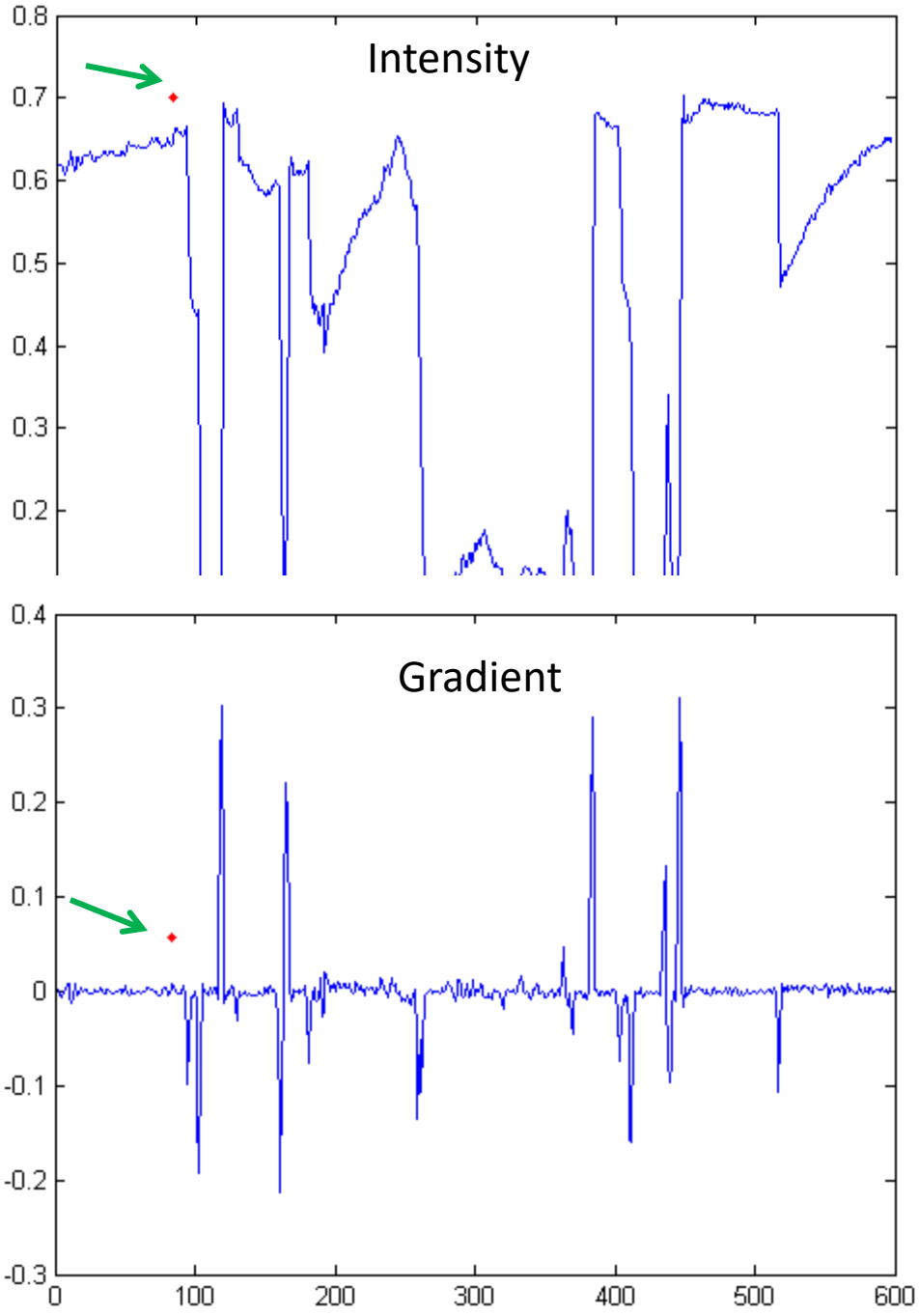
Roberts

Prewitt

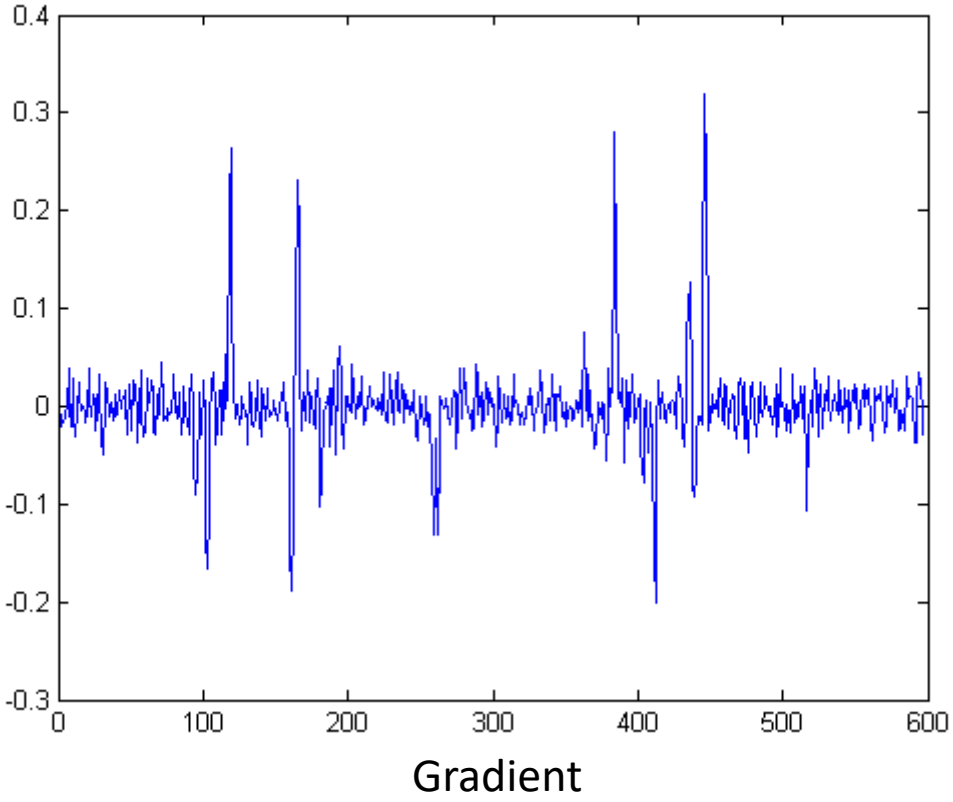
Laplacian

Intensity profile



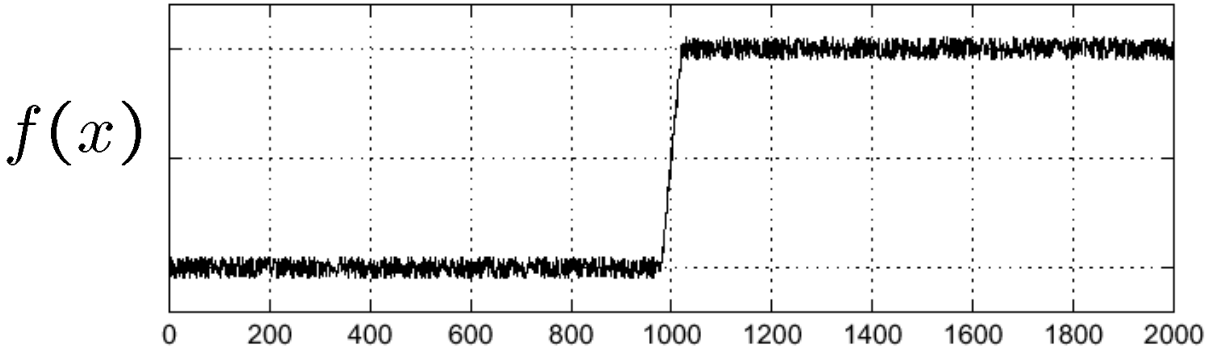


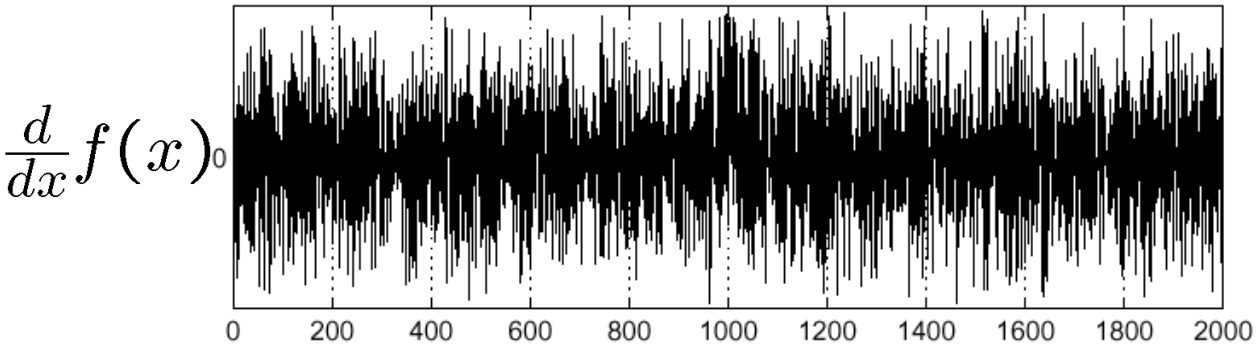
With a little Gaussian noise



Effects of noise

- Consider a single row or column of the image
  - Plotting intensity as a function of position gives a signal





Where is the edge?

Effects of noise

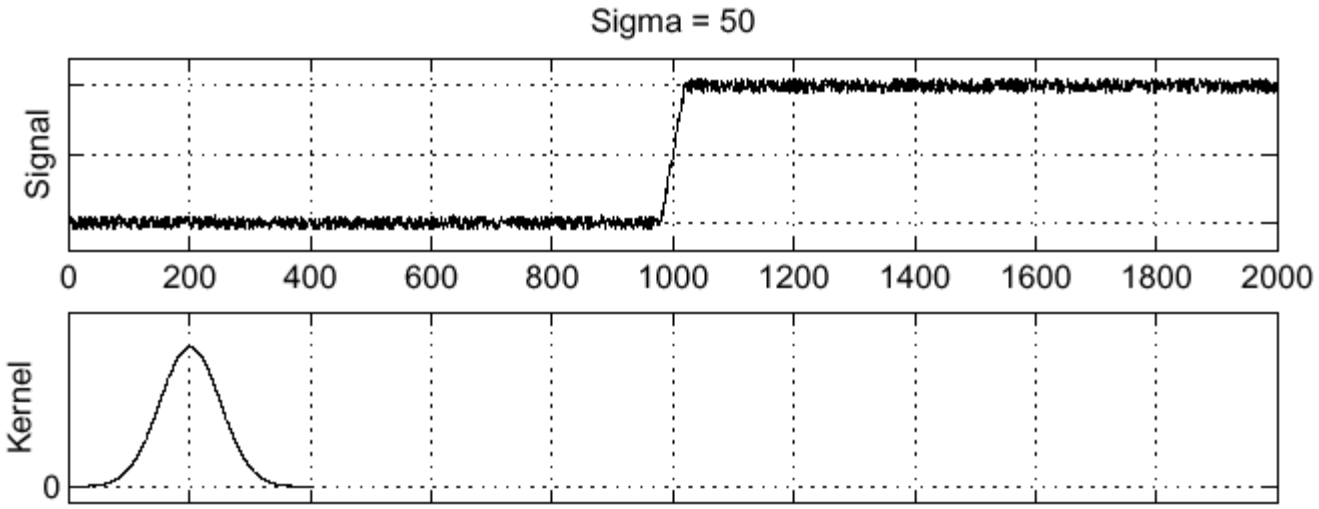
- Difference filters respond strongly to noise
  - Image noise results in pixels that look very different from their neighbors
  - Generally, the larger the noise the stronger the response
- What can we do about it?

Source: D. Forsyth

Solution: smooth first

f

h



Where is the edge?

Look for peaks in $\frac{\partial}{\partial x}(h \star f)$

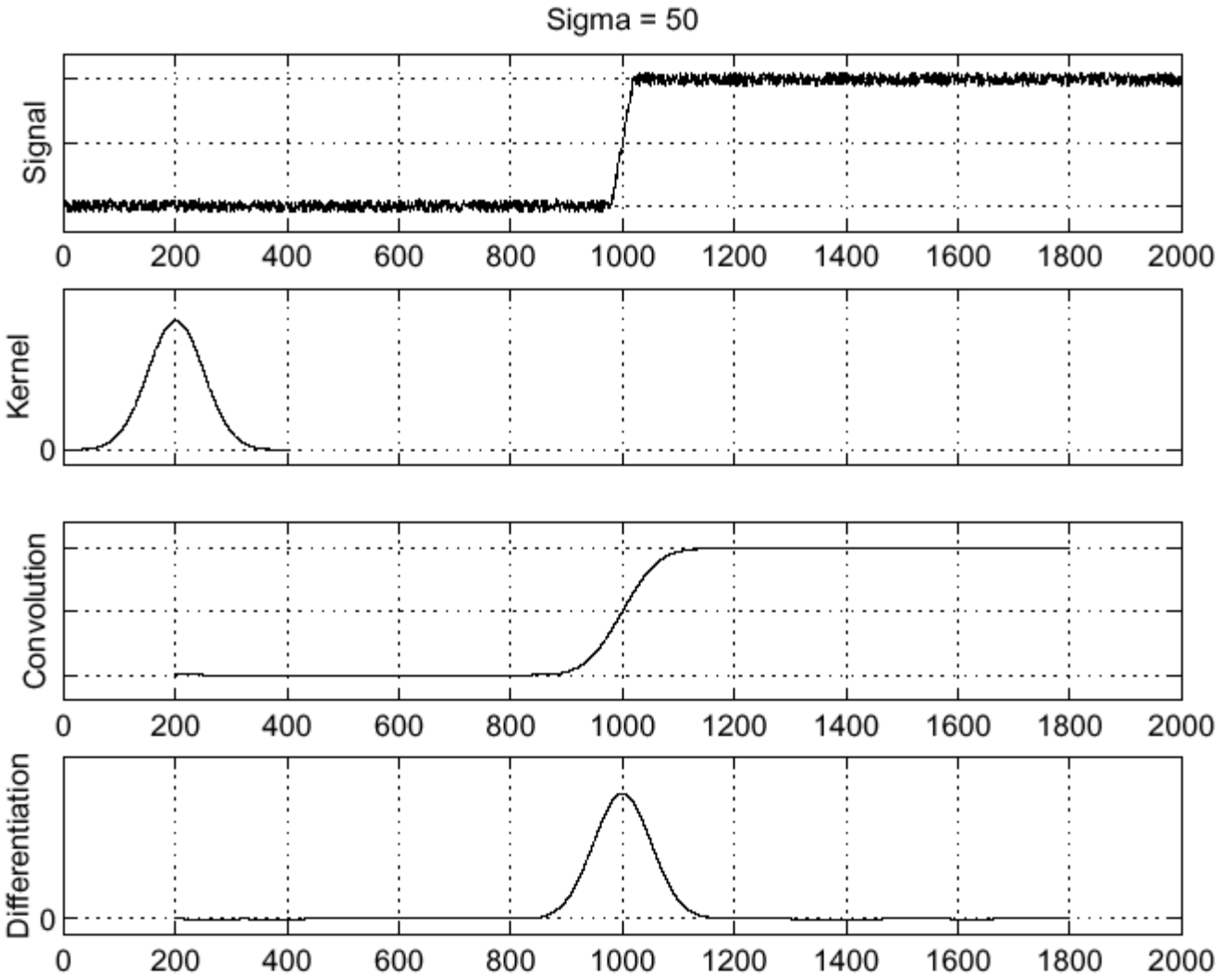
Solution: smooth first

f

h

$h \star f$

$\frac{\partial}{\partial x}(h \star f)$



Where is the edge?

Look for peaks in $\frac{\partial}{\partial x}(h \star f)$

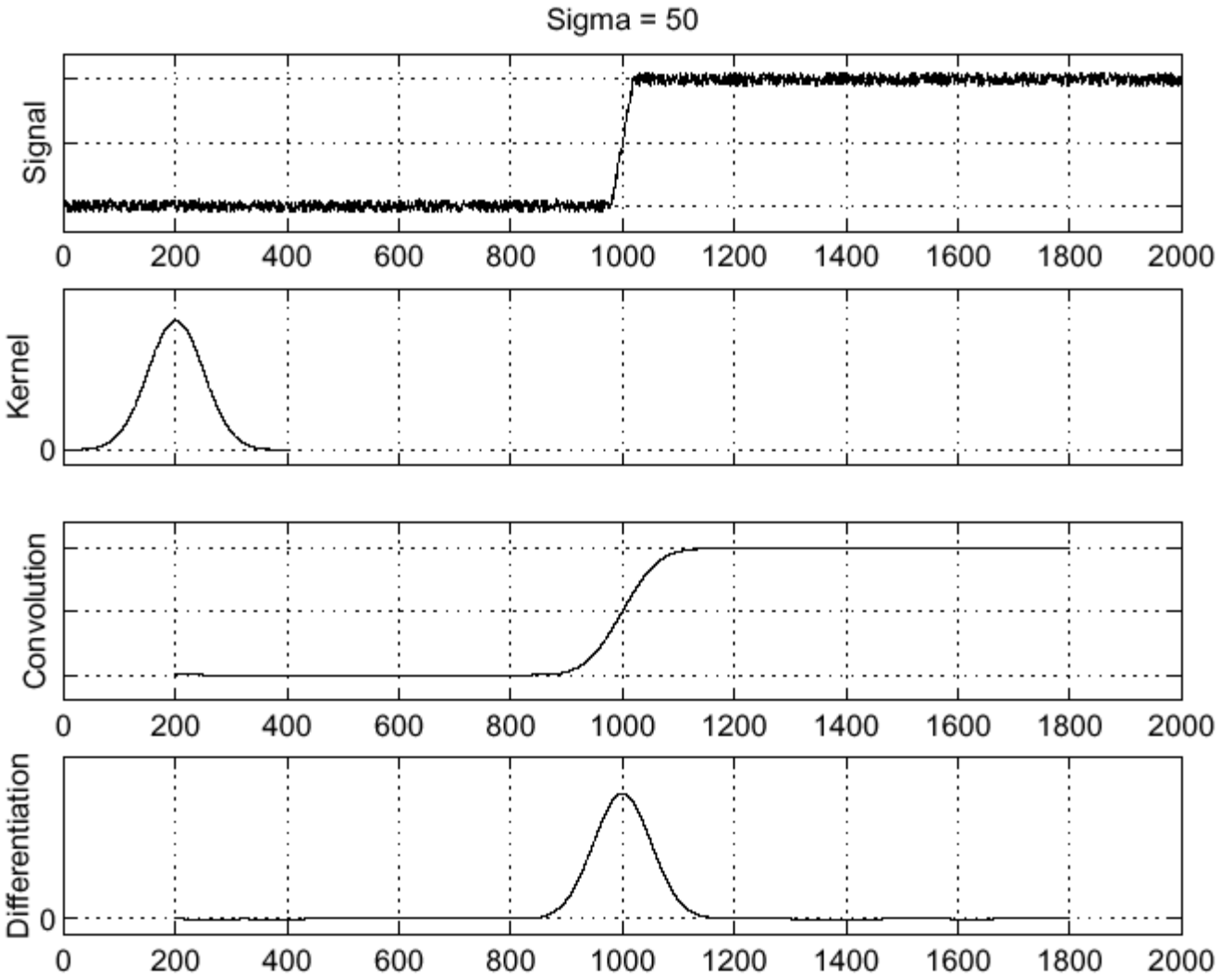
Solution: smooth first

f

h

$h \star f$

$\frac{\partial}{\partial x}(h \star f)$



Where is the edge?

Look for peaks in $\frac{\partial}{\partial x}(h \star f)$ = zero's of the function $\frac{\partial^2}{\partial x^2}(h \star f)$

Derivative theorem of convolution

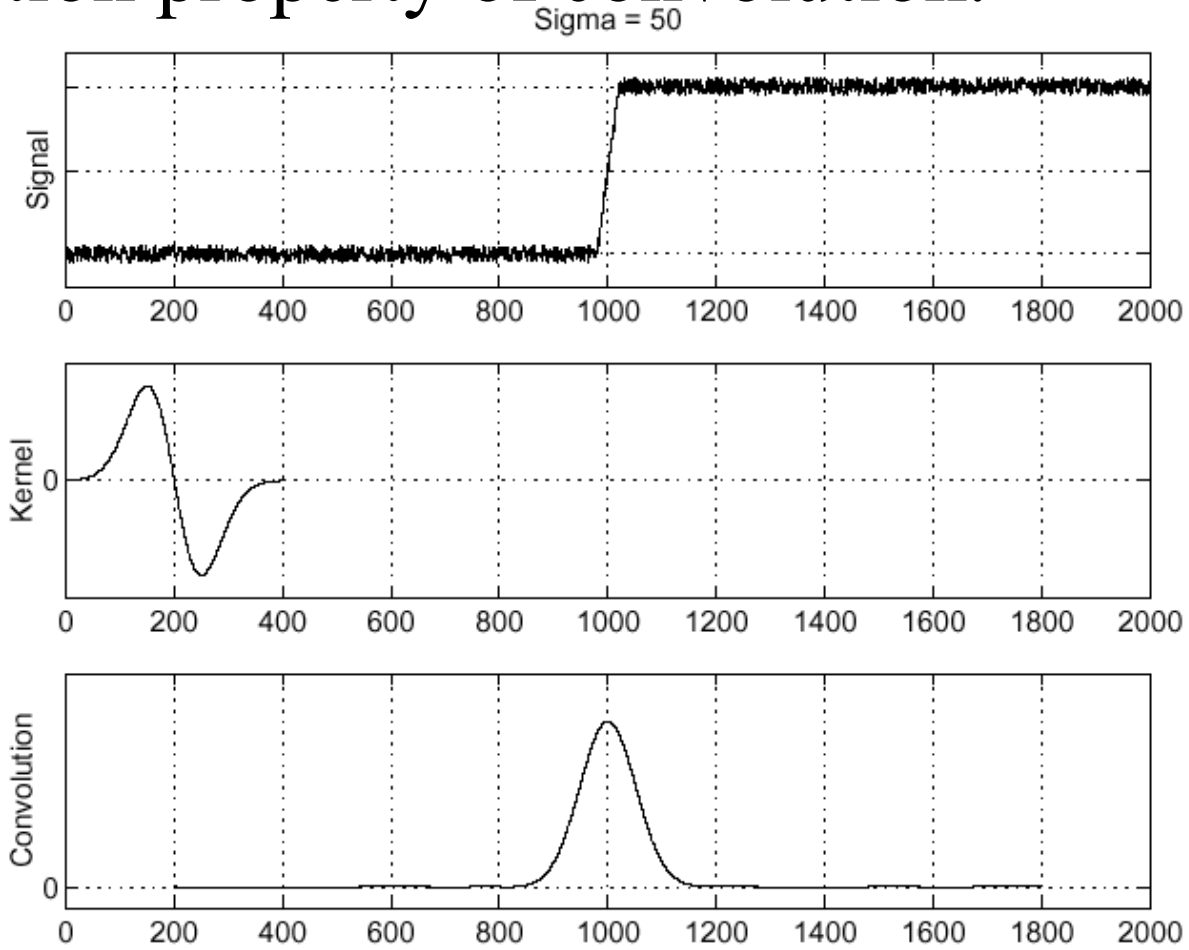
$$\frac{\partial}{\partial x}(h \star f) = (\frac{\partial}{\partial x}h) \star f$$

Differentiation property of convolution.

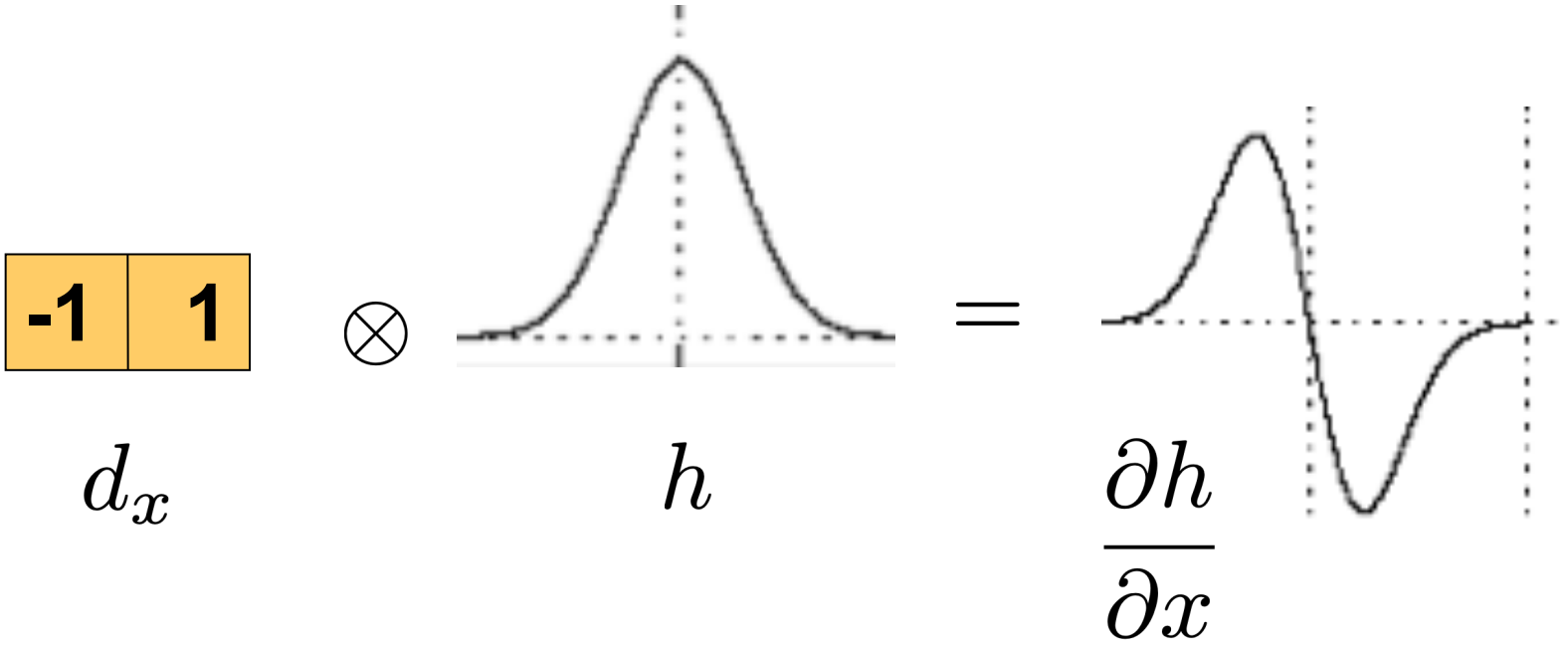
f

$\frac{\partial}{\partial x}h$

$(\frac{\partial}{\partial x}h) \star f$



1D



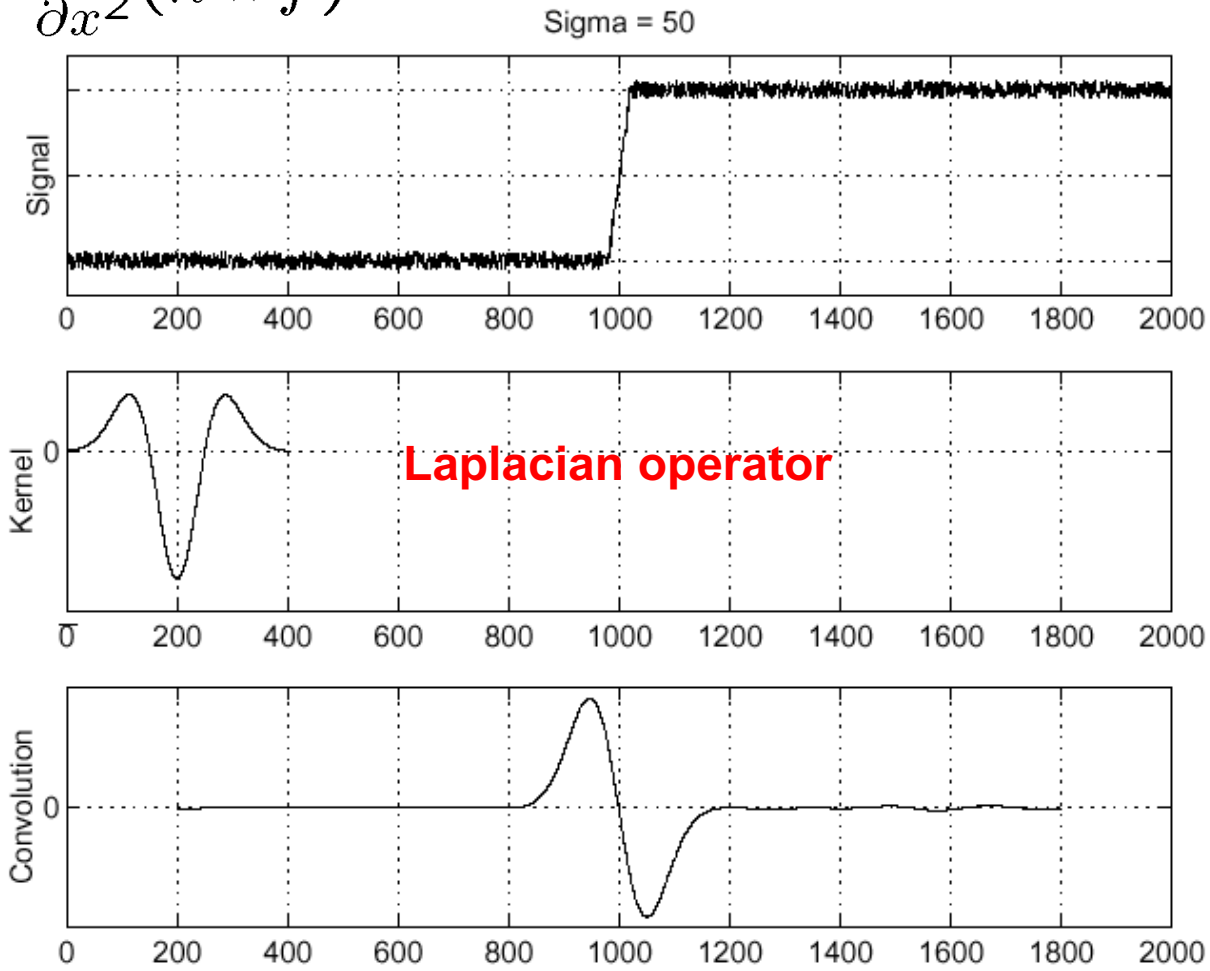
Laplacian of a Gaussian – 1D

Consider $\frac{\partial^2}{\partial x^2}(h \star f)$

f

$\frac{\partial^2}{\partial x^2}h$

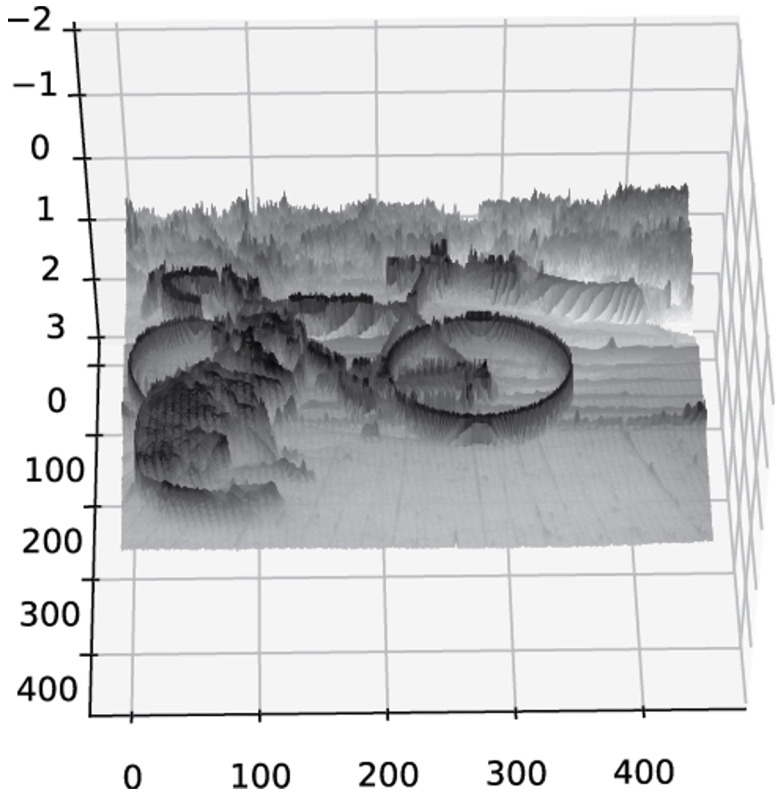
$(\frac{\partial^2}{\partial x^2}h) \star f$



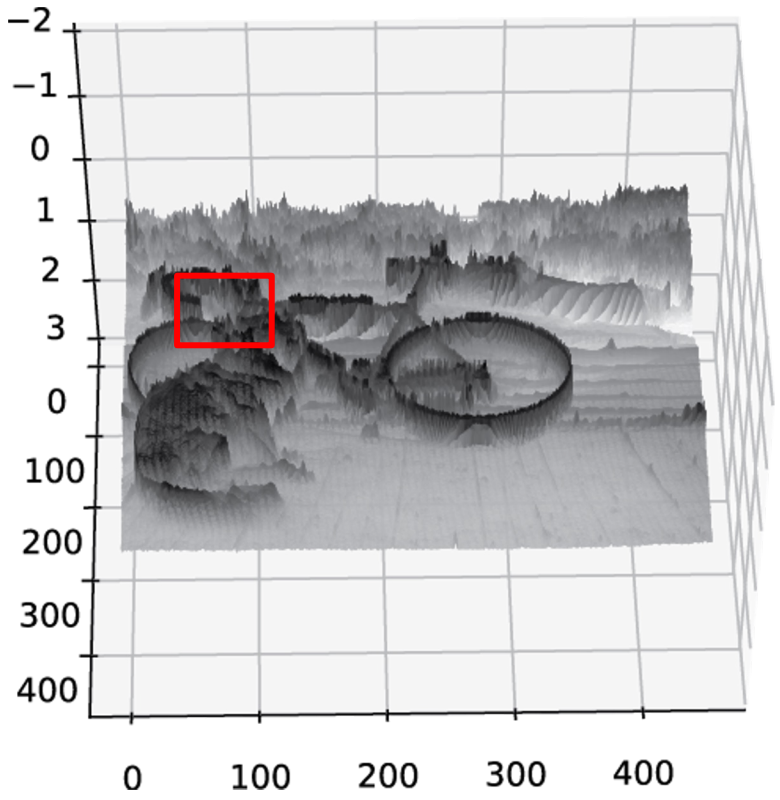
Where is the edge?

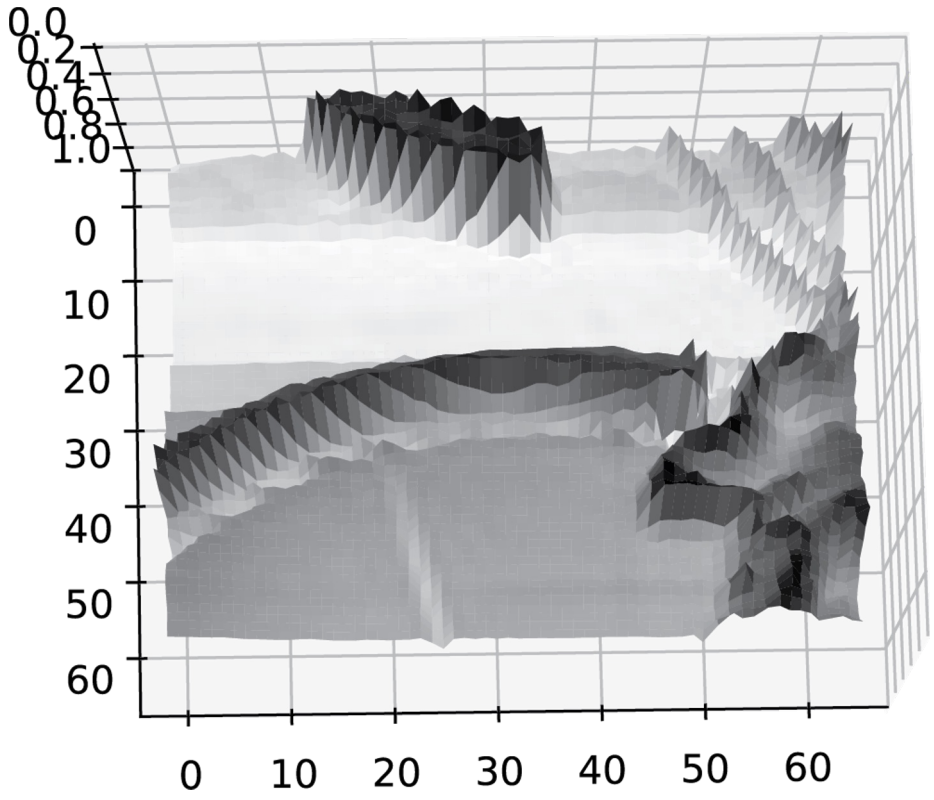
Zero-crossing of the function $(\frac{\partial^2}{\partial x^2}h) \star f$

2D: Images are noisy

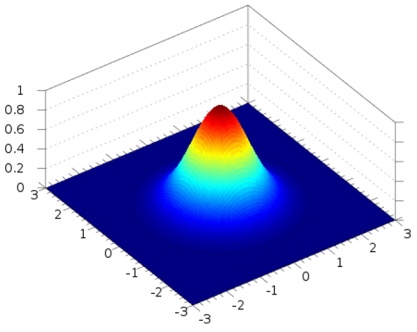


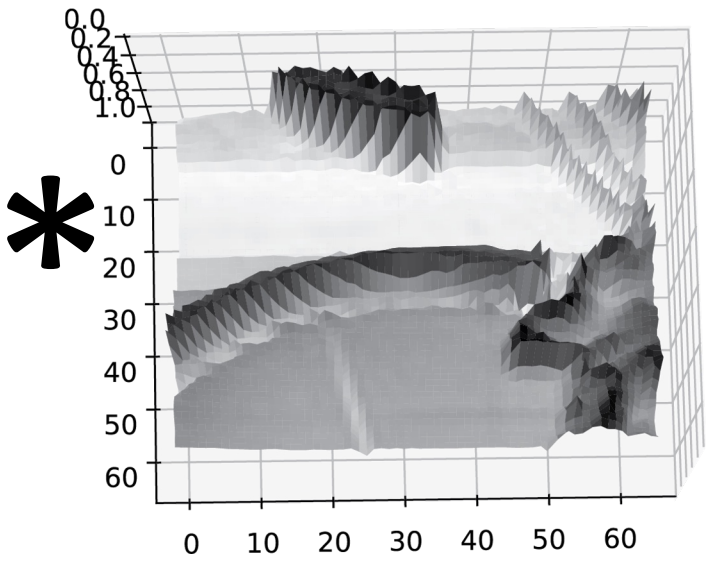
2D: Images are noisy

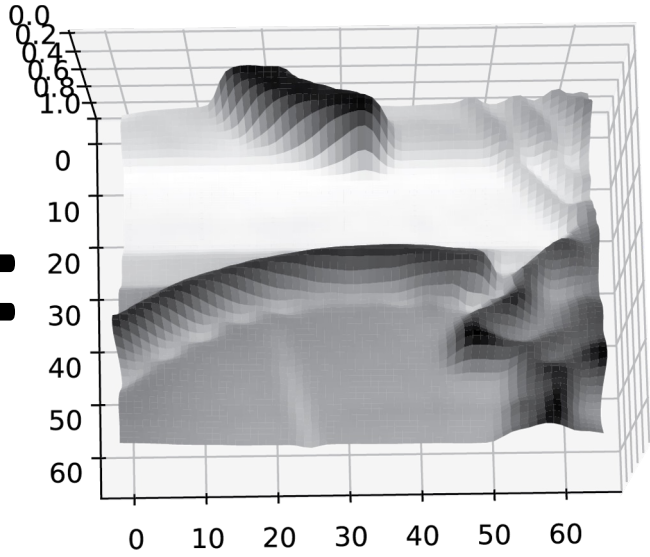




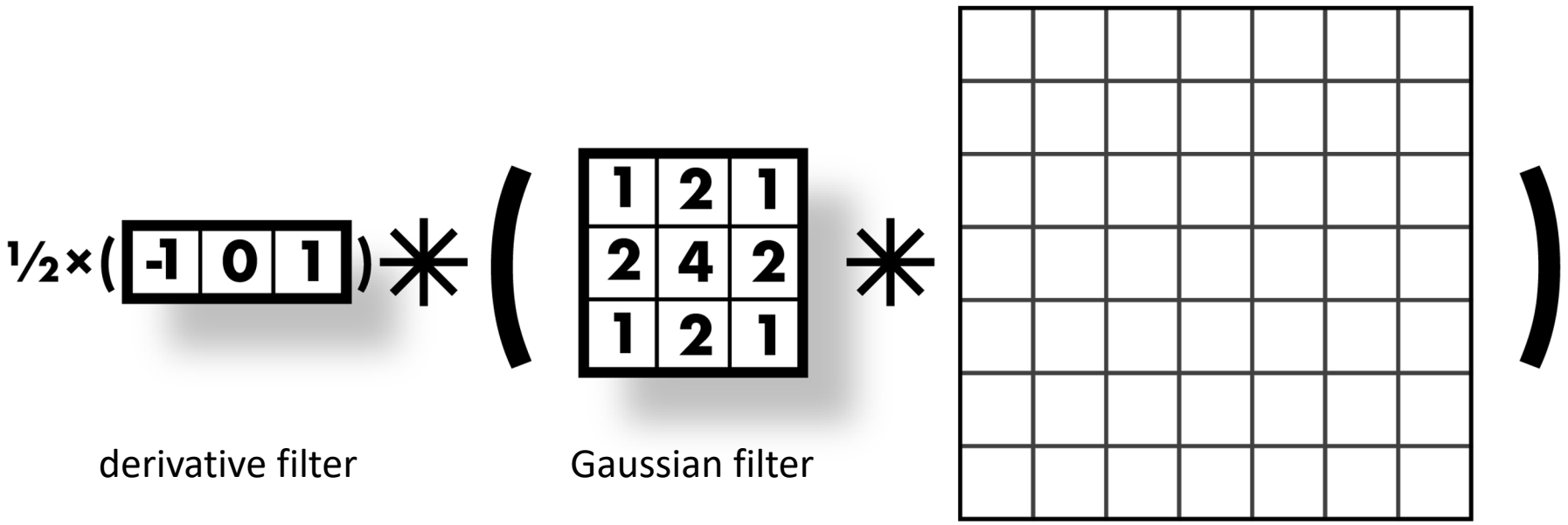
Solution: smooth first







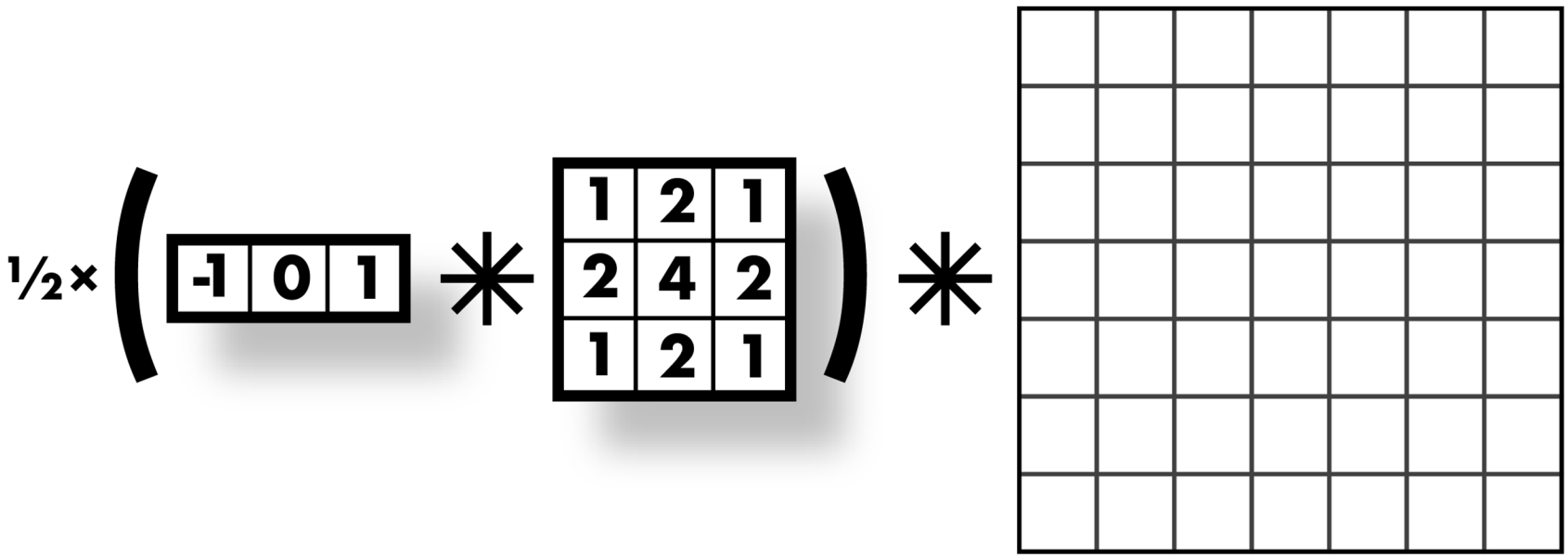
Smooth first, then derivative



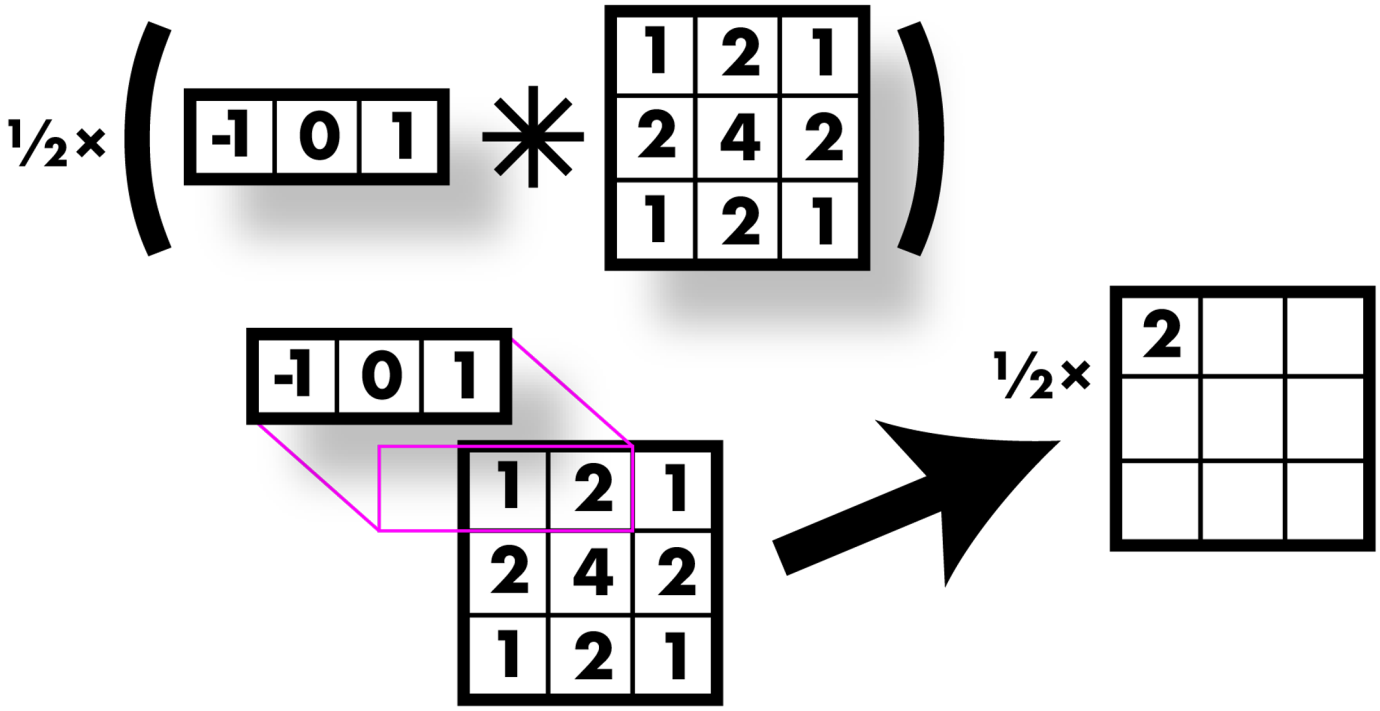
derivative filter

Gaussian filter

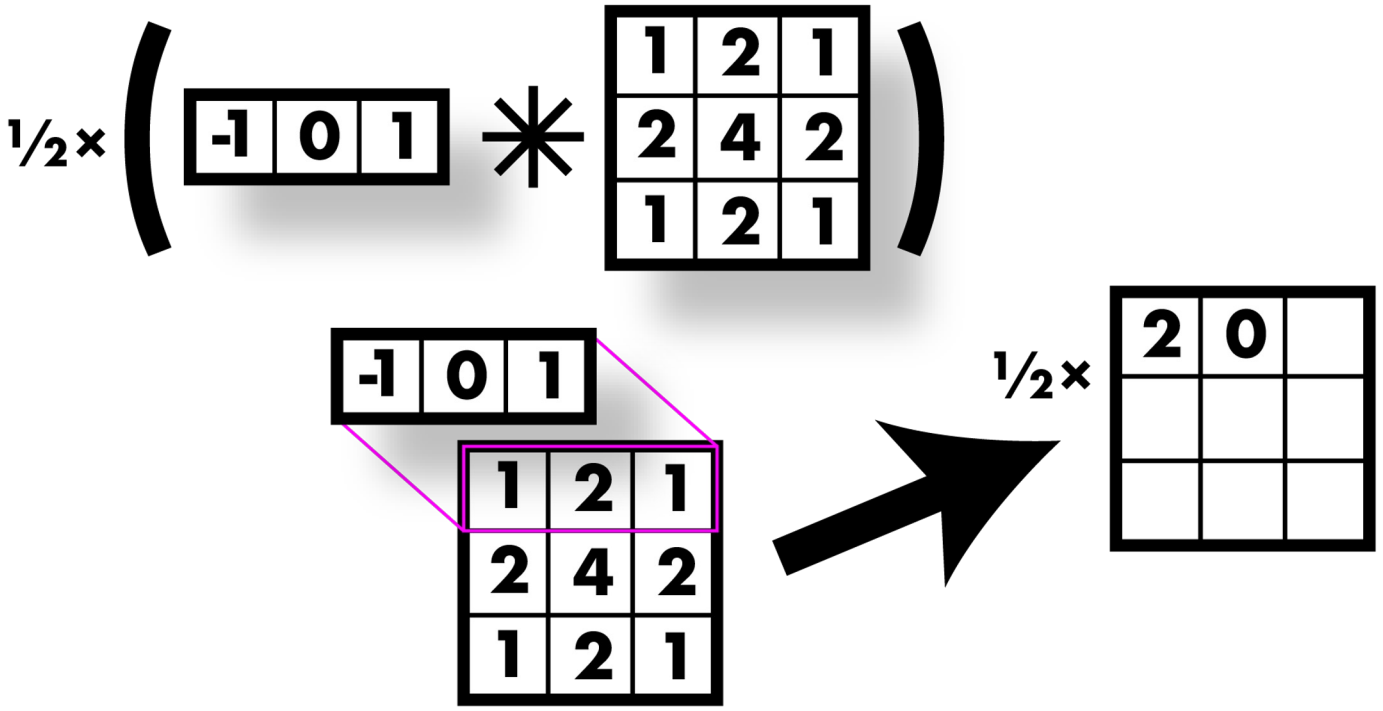
Smooth first, then derivative



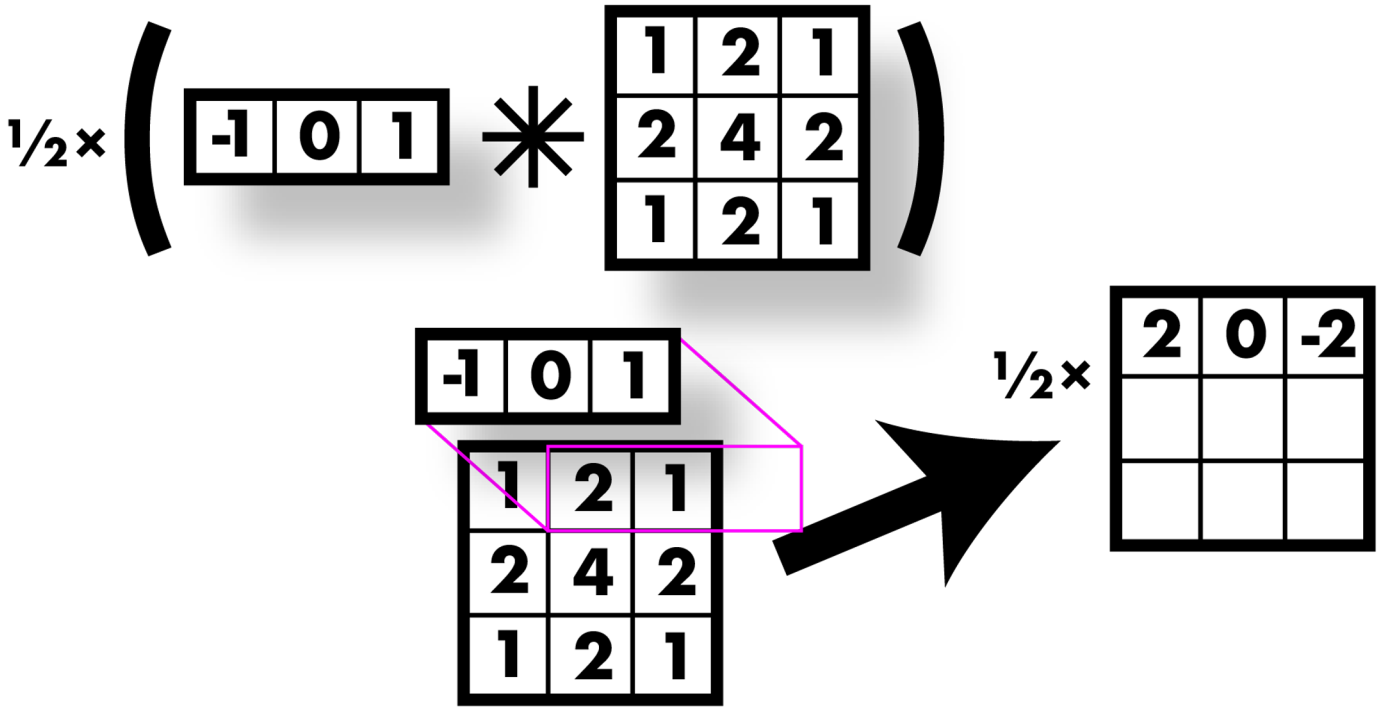
Smooth first, then derivative



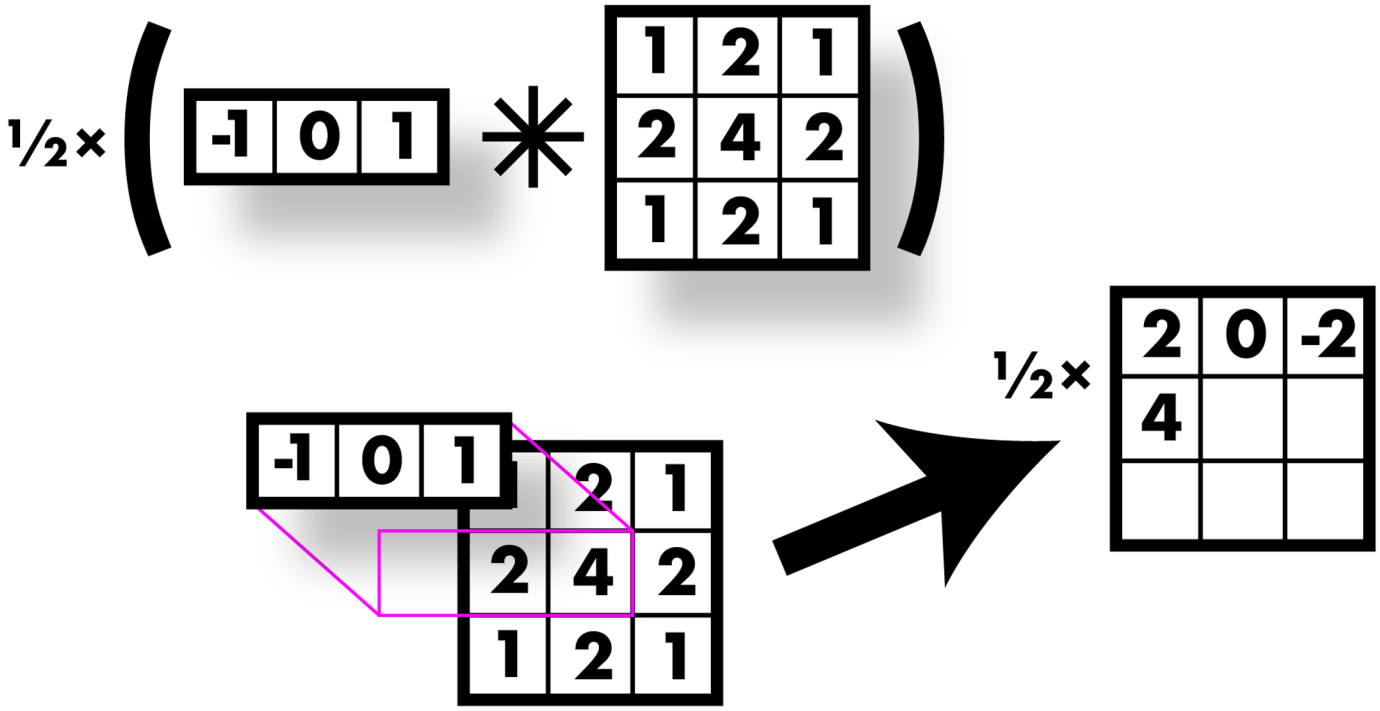
Smooth first, then derivative



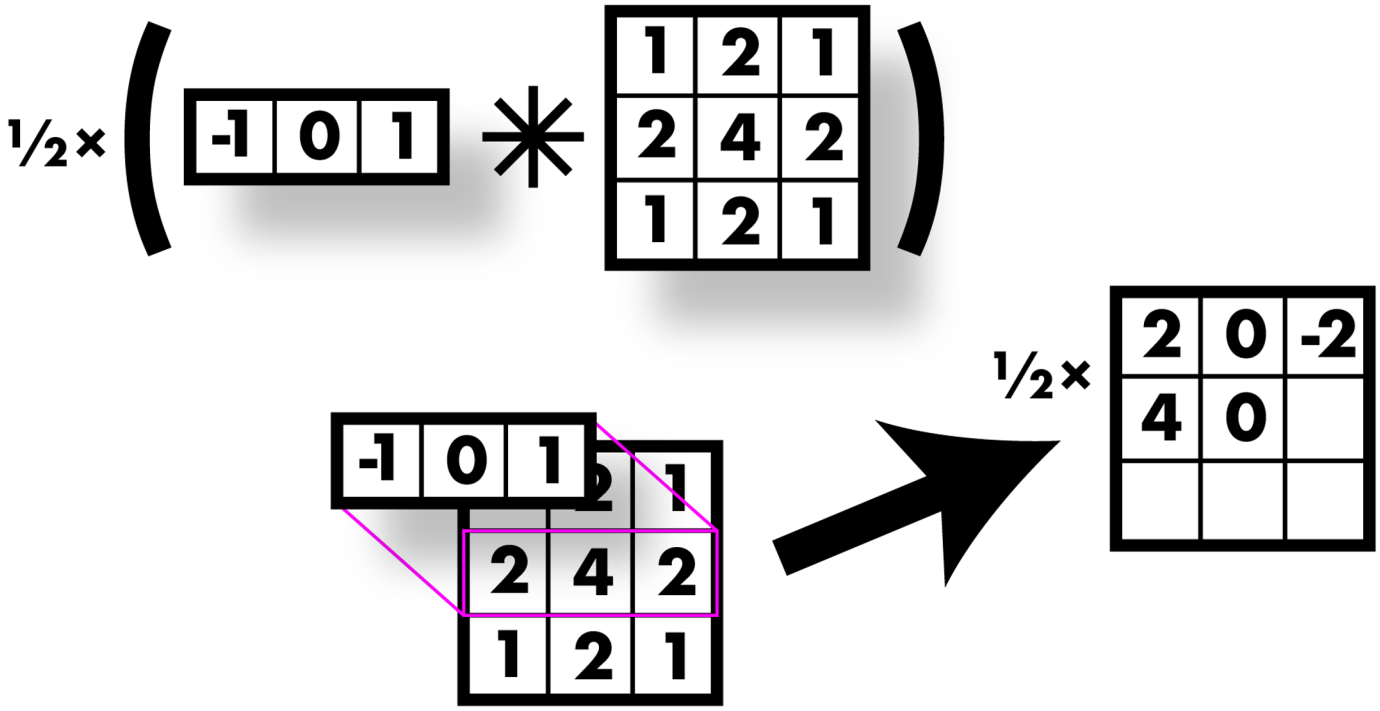
Smooth first, then derivative



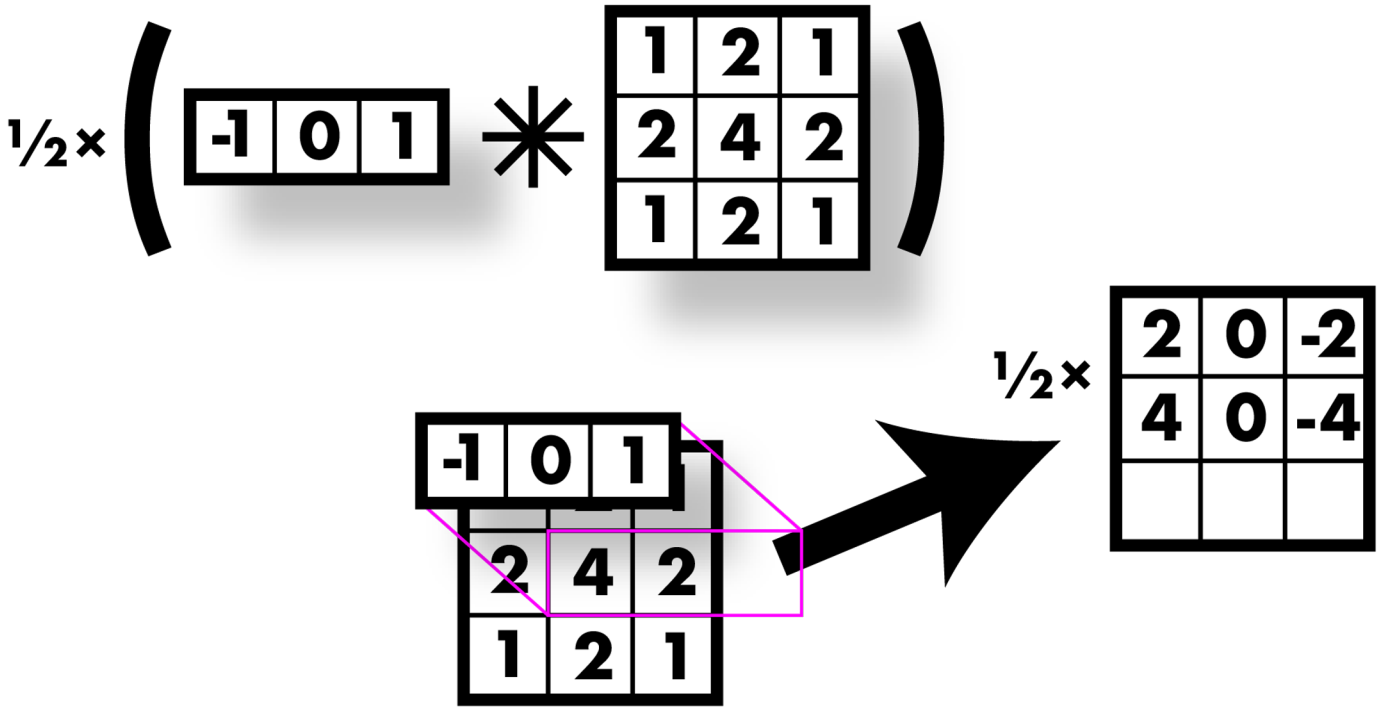
Smooth first, then derivative



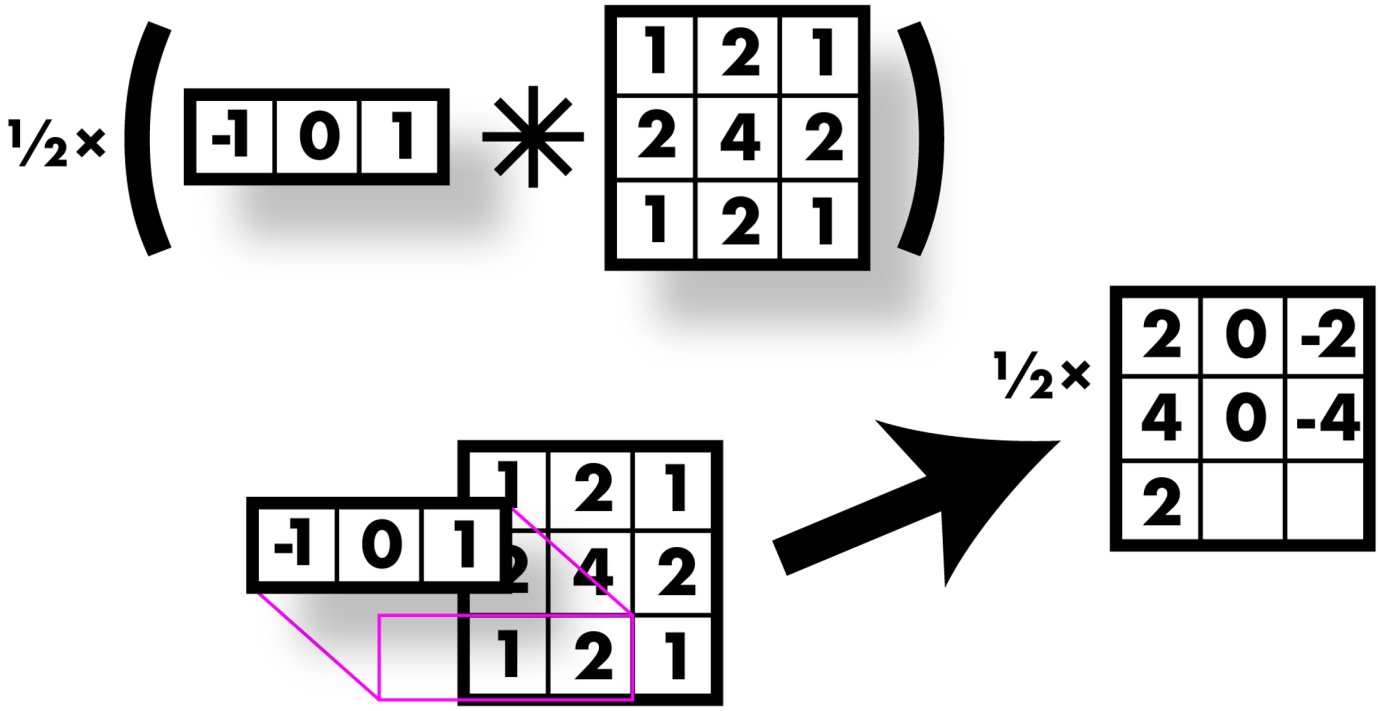
Smooth first, then derivative



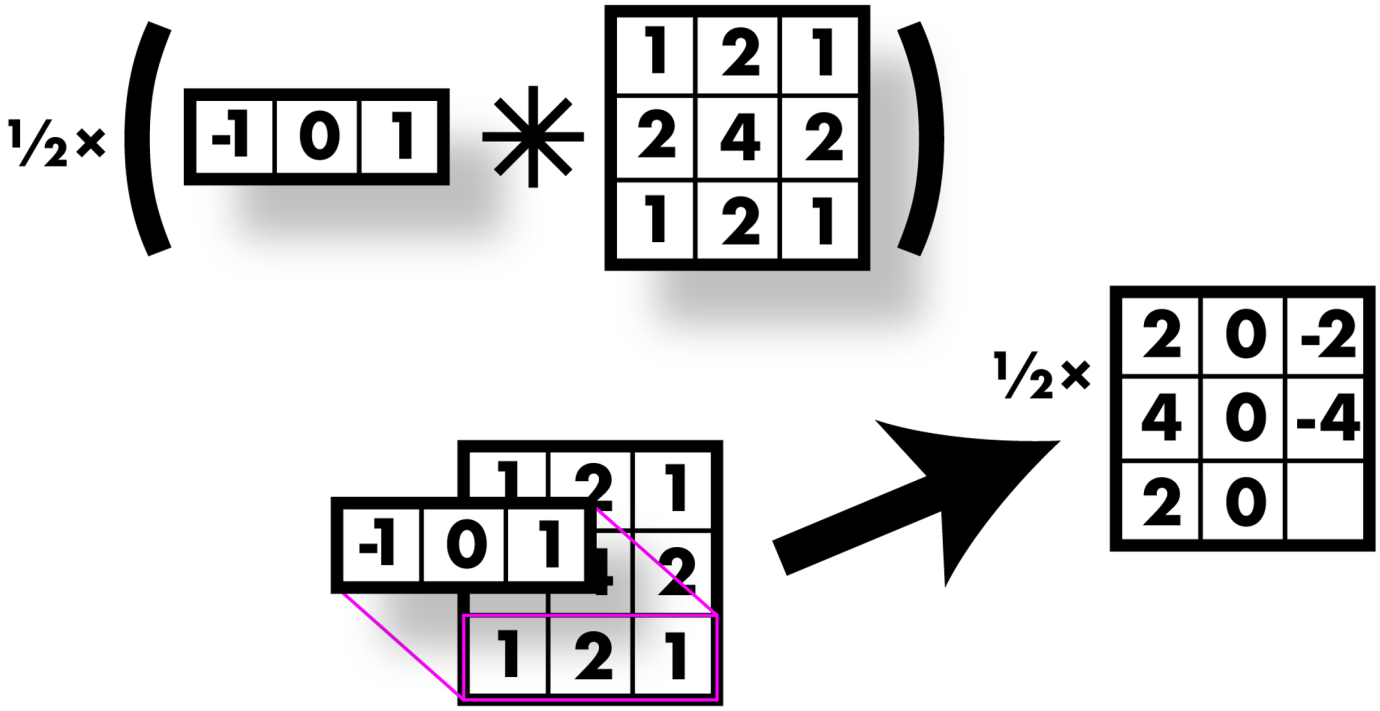
Smooth first, then derivative



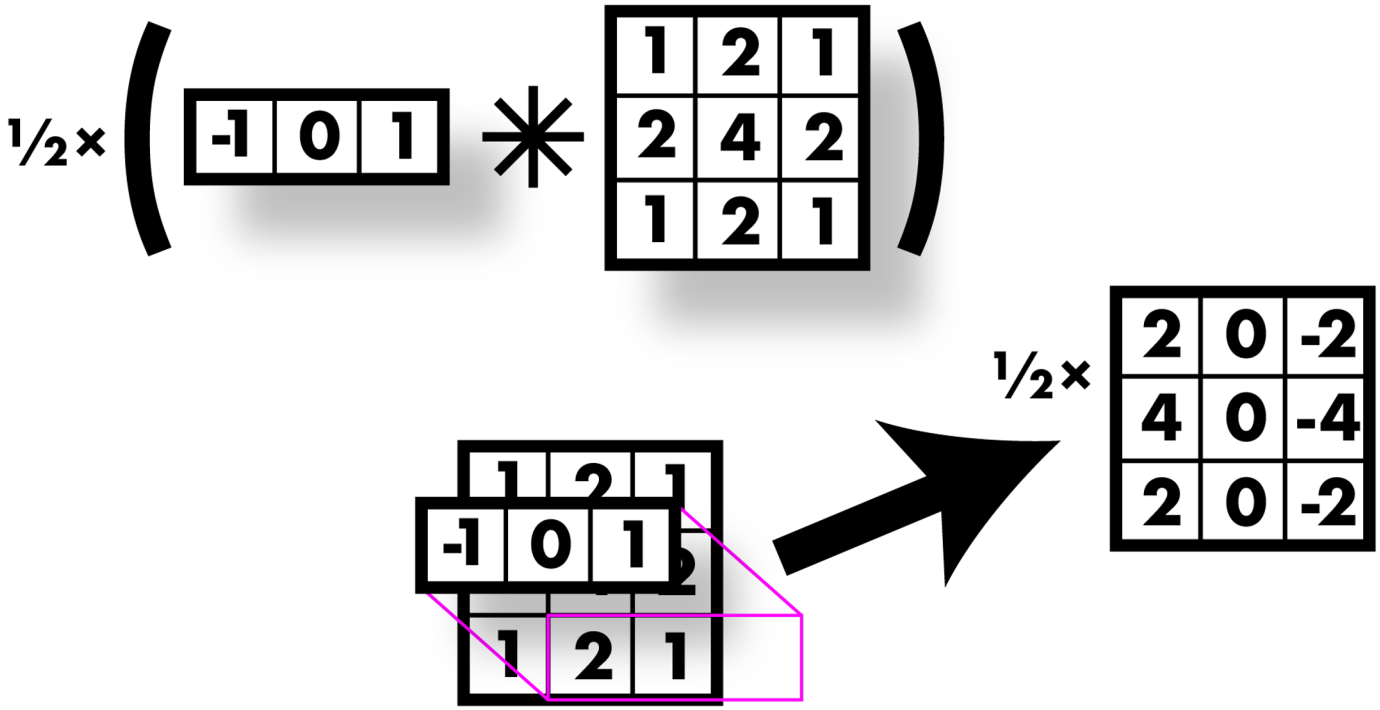
Smooth first, then derivative



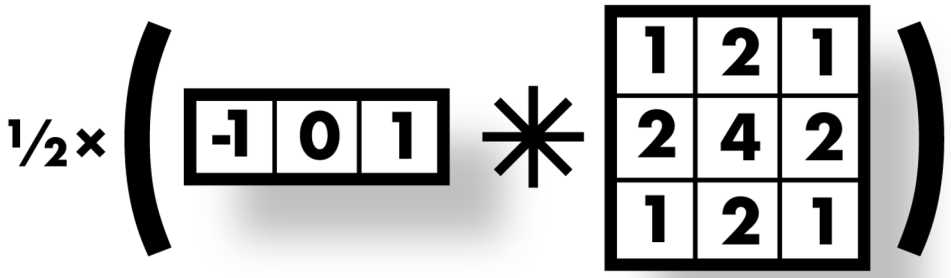
Smooth first, then derivative

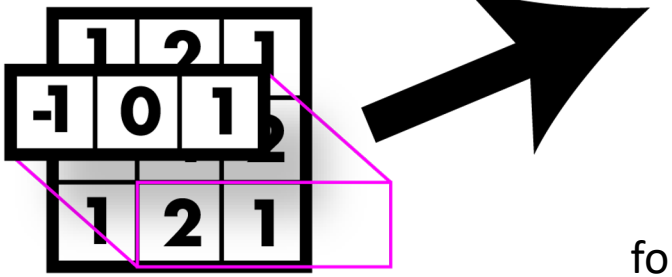


Smooth first, then derivative



Sobel filter: smooth & derivative





1	0	-1
2	0	-2
1	0	-1

Vertical Sobel filter
for computing the partial derivative

$$\frac{\partial f(x,y)}{\partial x}$$

Derivative of Gaussian filters - 2D

$$d_x \otimes (h \otimes I) = (d_x \otimes h) \otimes I$$

derivation filter

Gaussian filter

image

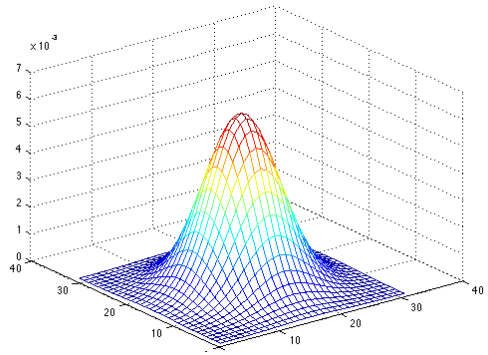
0	0	0
-1	0	1
0	0	0

$\otimes$

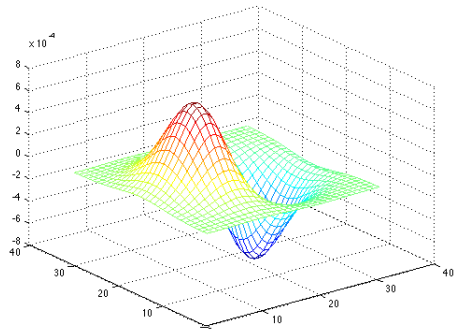
0.0030	0.0133	0.0219	0.0133	0.0030
0.0133	0.0596	0.0983	0.0596	0.0133
0.0219	0.0983	0.1621	0.0983	0.0219
0.0133	0.0596	0.0983	0.0596	0.0133
0.0030	0.0133	0.0219	0.0133	0.0030

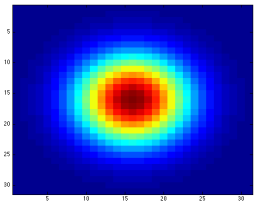
0	0	0
-1	0	1
0	0	0

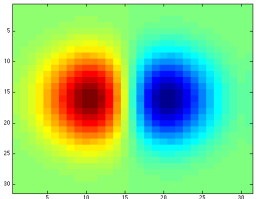
$\otimes$



$=$







Derivative of Gaussian filters - 2D

$$d_y \otimes (h \otimes I) = (d_y \otimes h) \otimes I$$

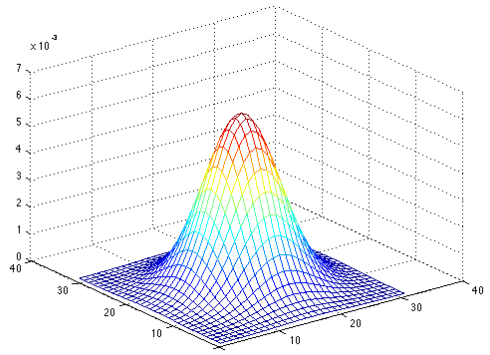
0	-1	0
0	0	0
0	1	0

$\otimes$

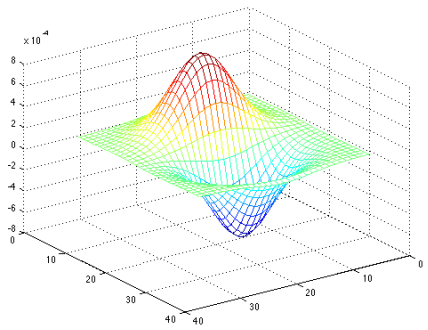
$$\begin{bmatrix} 0.0030 & 0.0133 & 0.0219 & 0.0133 & 0.0030 \\ 0.0133 & 0.0596 & 0.0983 & 0.0596 & 0.0133 \\ 0.0219 & 0.0983 & 0.1621 & 0.0983 & 0.0219 \\ 0.0133 & 0.0596 & 0.0983 & 0.0596 & 0.0133 \\ 0.0030 & 0.0133 & 0.0219 & 0.0133 & 0.0030 \end{bmatrix}$$

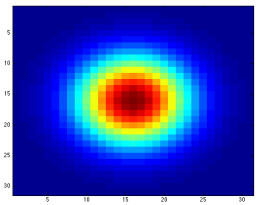
0	-1	0
0	0	0
0	1	0

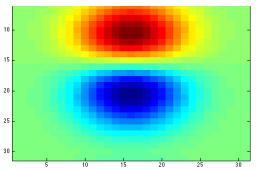
$\otimes$



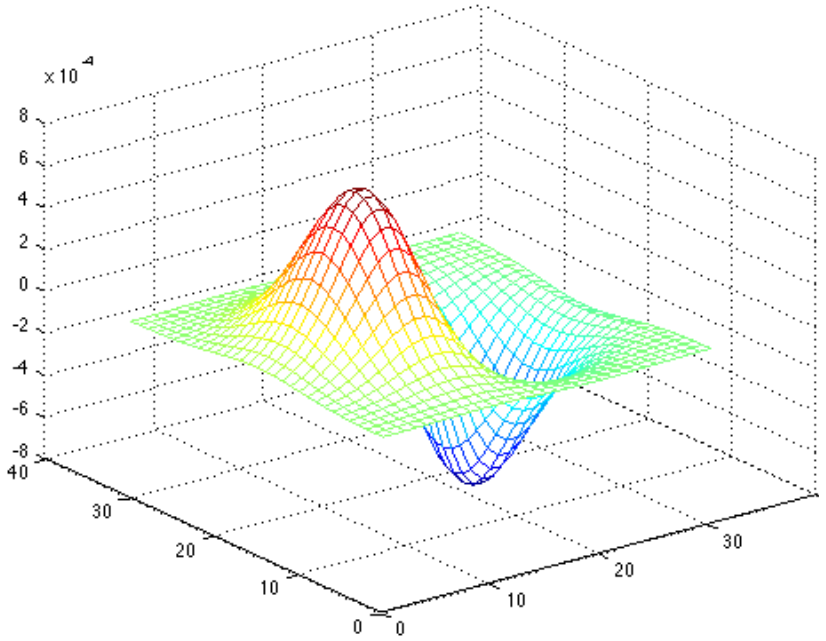
$=$



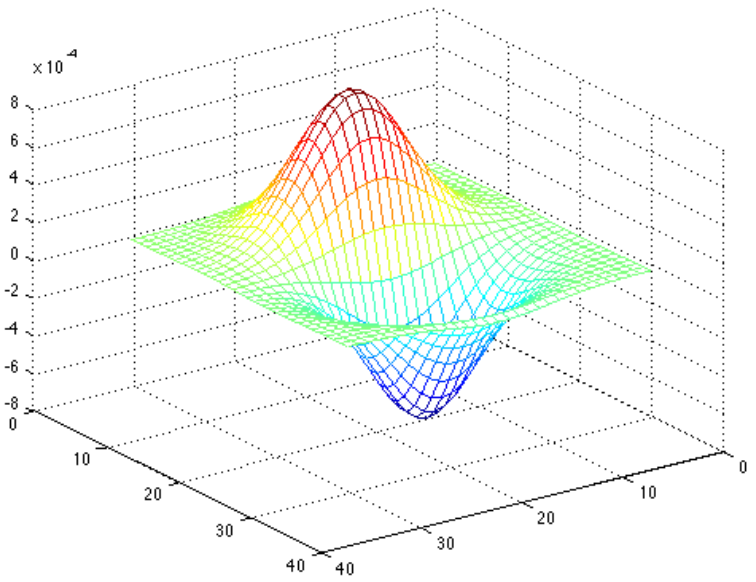




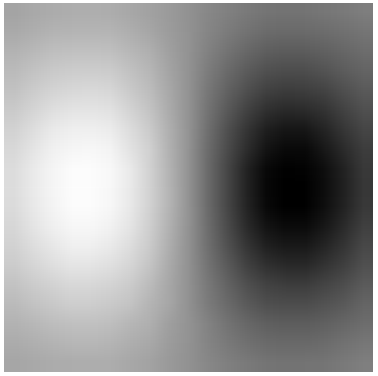
Derivative of Gaussian filters - 2D

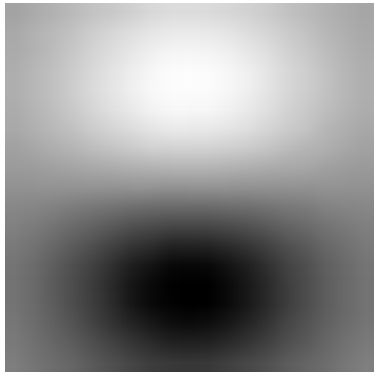


x - direction



y - direction



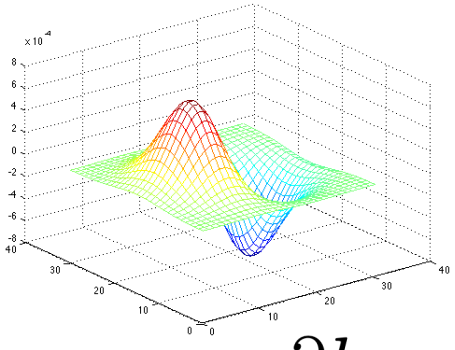


Derivative of Gaussian filters - 2D

0	0	0
-1	0	1
0	0	0

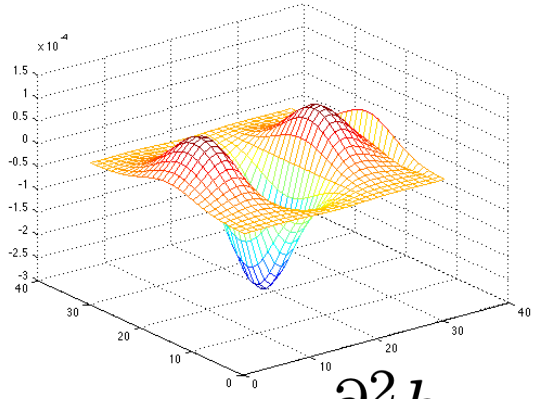
direction x

$\otimes$



$\frac{\partial h}{\partial x}$

$=$

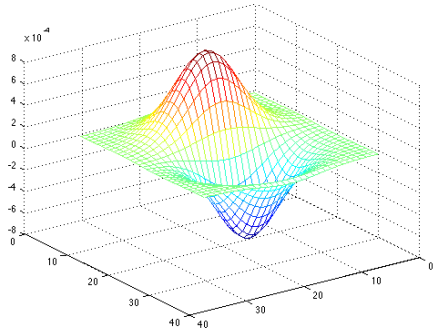


$\frac{\partial^2 h}{\partial x^2}$

0	-1	0
0	0	0
0	1	0

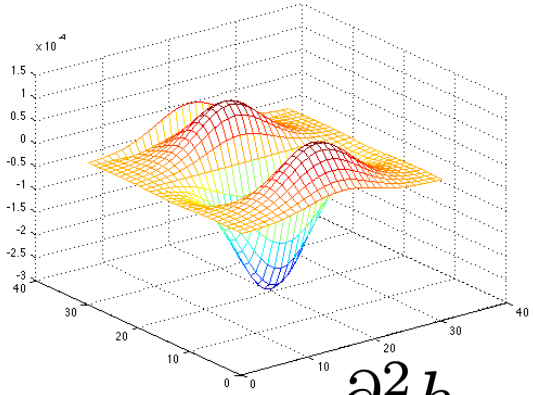
direction y

$\otimes$



$\frac{\partial h}{\partial y}$

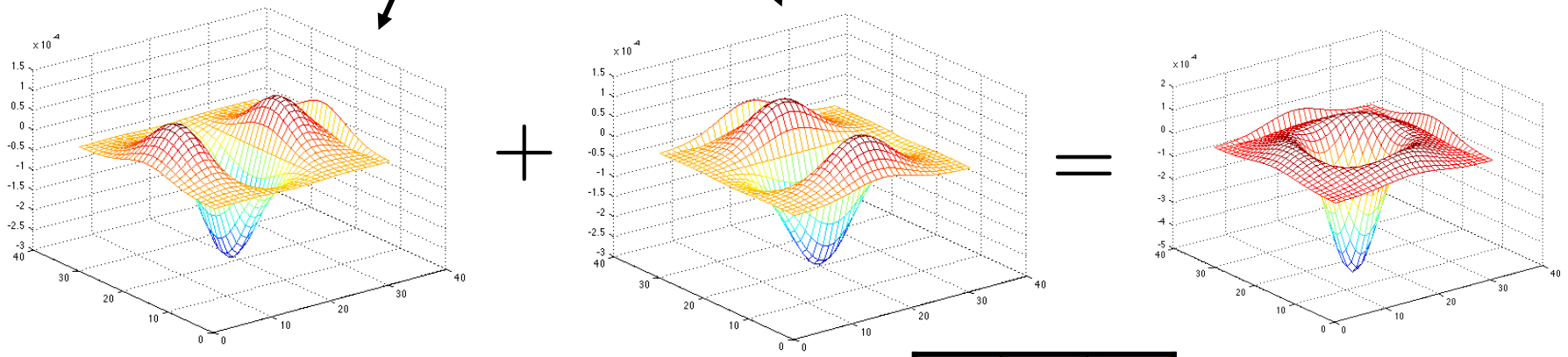
$=$



$\frac{\partial^2 h}{\partial y^2}$

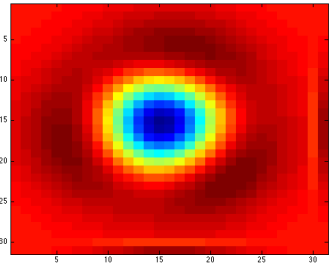
Laplacian of Gaussian

$$\nabla^2 h = \frac{\partial^2 h}{\partial x^2} + \frac{\partial^2 h}{\partial y^2}$$



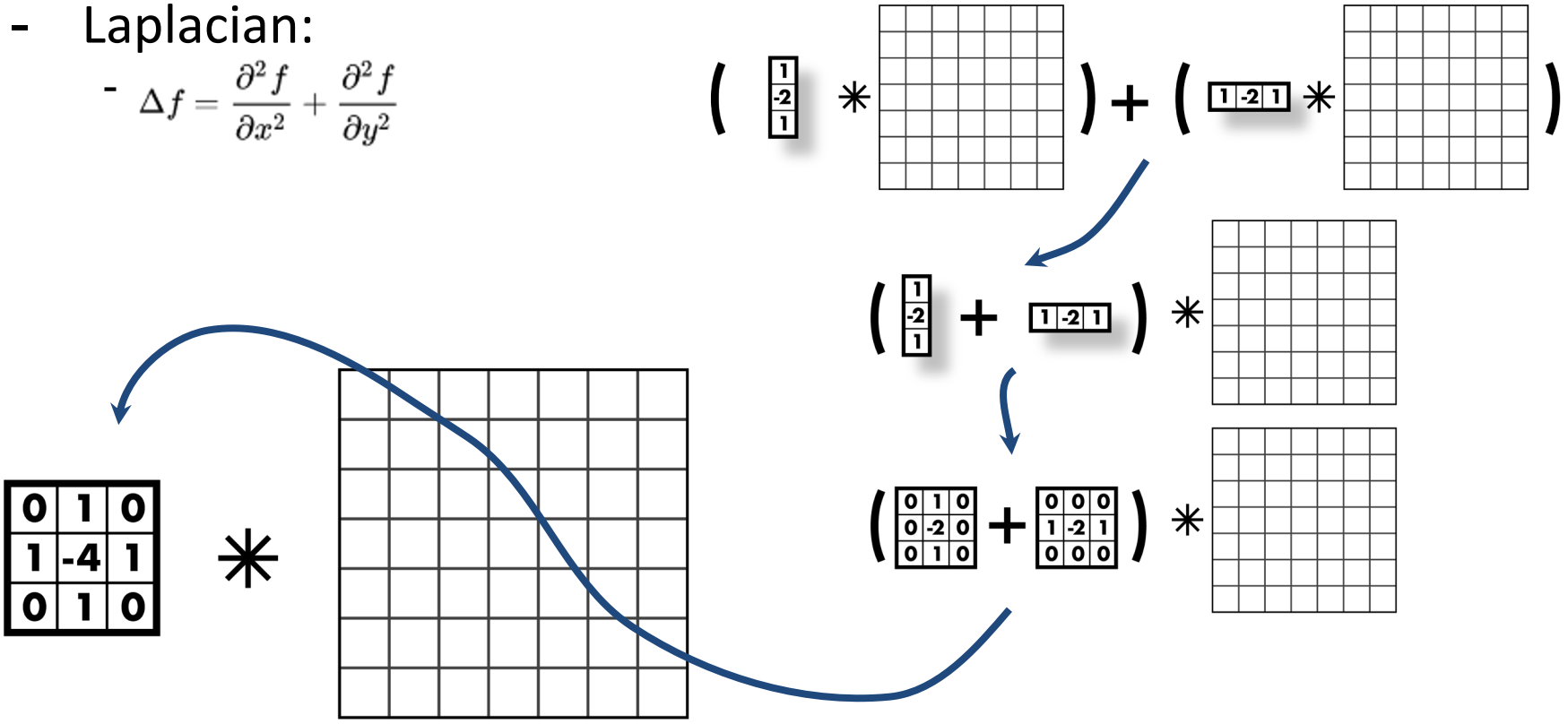
0	1	0
1	-4	1
0	1	0

Example of a Laplacian filter

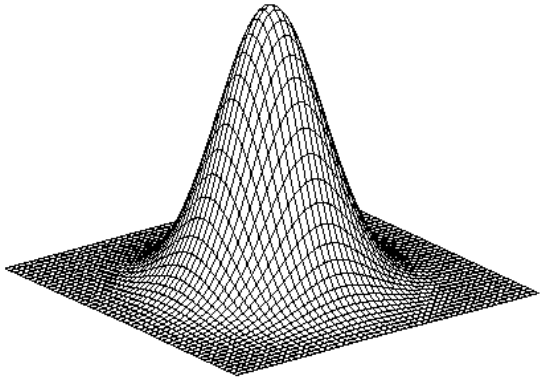


Laplacian of Gaussian

- Laplacian:
  - $\Delta f = \frac{\partial^2 f}{\partial x^2} + \frac{\partial^2 f}{\partial y^2}$

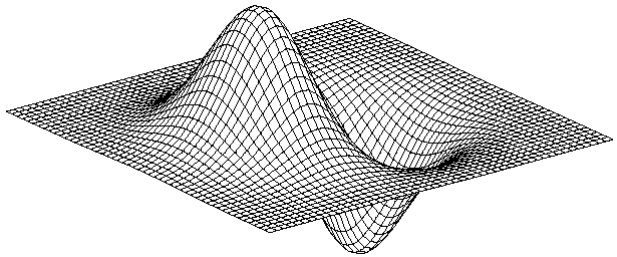


2D edge detection filters



Gaussian

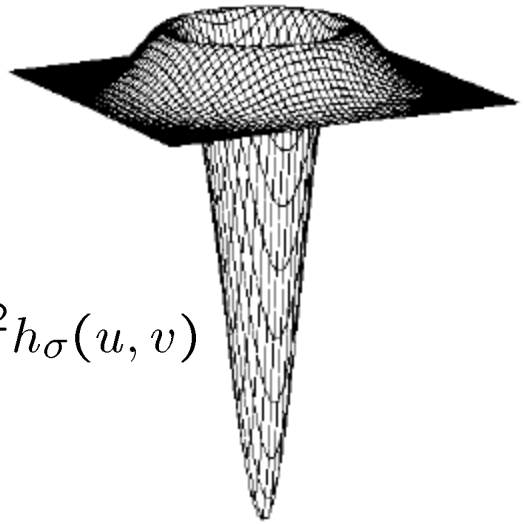
$$h_\sigma(u,v) = \frac{1}{2\pi\sigma^2} e^{-\frac{u^2+v^2}{2\sigma^2}}$$



derivative of Gaussian

$$\frac{\partial}{\partial x} h_\sigma(u,v)$$

Laplacian of Gaussian



$$\nabla^2 h_\sigma(u,v)$$

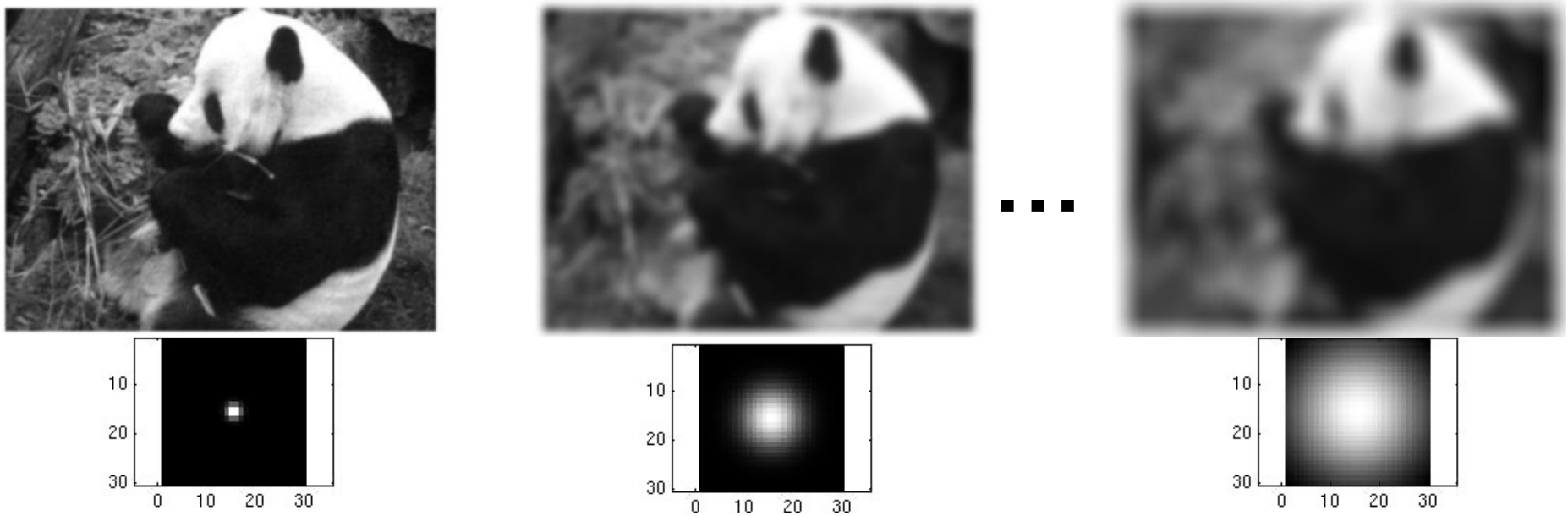
- ∇^2 is the Laplacian operator:

$$\nabla^2 f = \frac{\partial^2 f}{\partial x^2} + \frac{\partial^2 f}{\partial y^2}$$

Slide credit: Steve Seitz

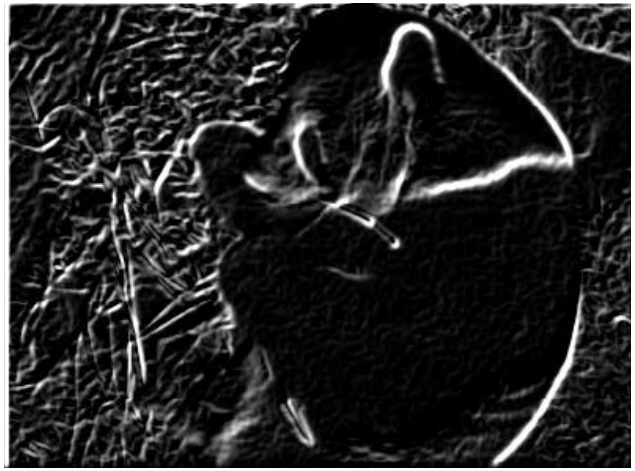
Smoothing with a Gaussian

Recall: parameter σ is the "scale" / "width" / "spread" of the Gaussian kernel, and controls the amount of smoothing.



Slide credit: Kristen Grauman

Effect of σ on derivatives



σ = 1 pixel

σ = 3 pixels

The apparent structures differ depending on Gaussian's scale parameter.

Larger values: larger scale edges detected
Smaller values: finer features detected

Slide credit: Kristen Grauman

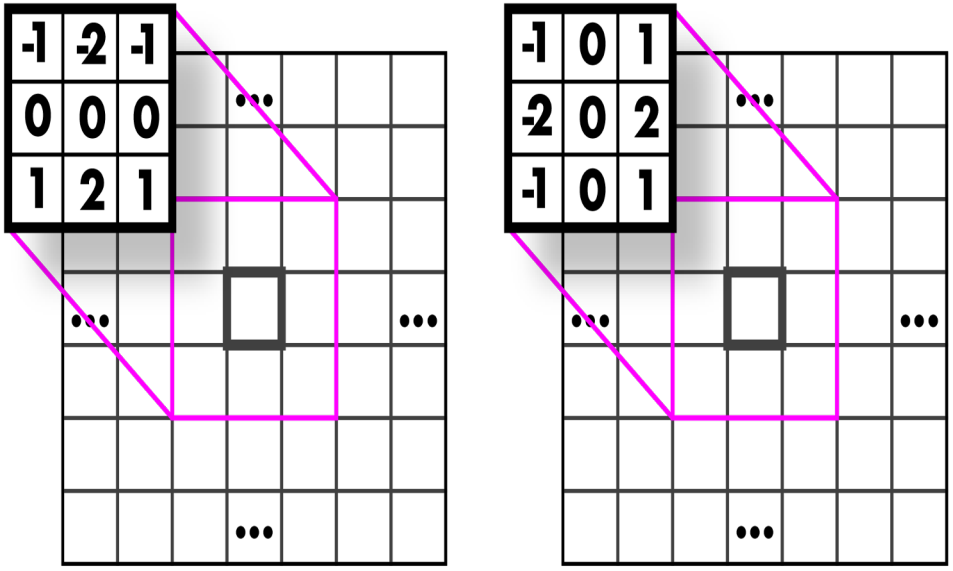
Mask properties

- Smoothing
  - Positive values
  - Sum to 1 → constant regions same as input
  - Amount of smoothing proportional to mask size
  - Remove “high-frequency” components; “low-pass” filter
- Derivatives
  - Opposite signs used to get high response in regions of high contrast
  - Sum to 0 → no response in constant regions
  - High absolute value at points of high contrast

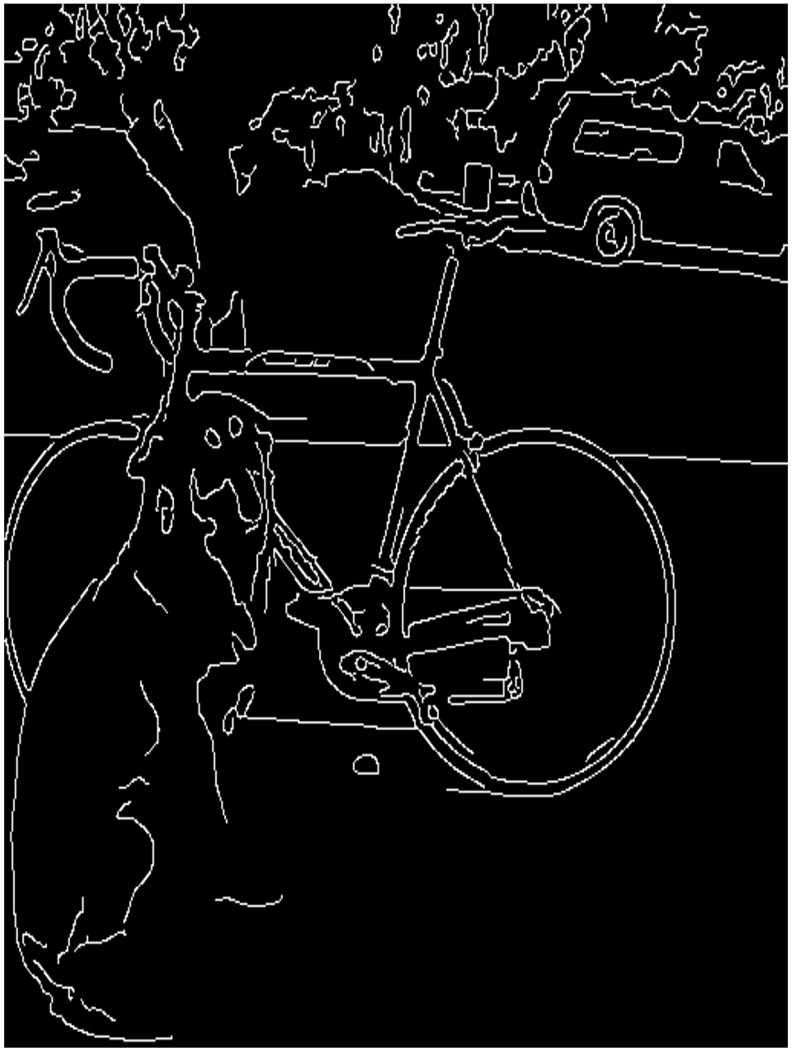
Slide credit: Kristen Grauman

Another approach: gradient magnitude

- Don't need 2nd derivatives
- Just use magnitude of gradient to compute a gradient map
- Could threshold his gradient map to obtain edges
- Are we done? No!



What we really want: line drawing



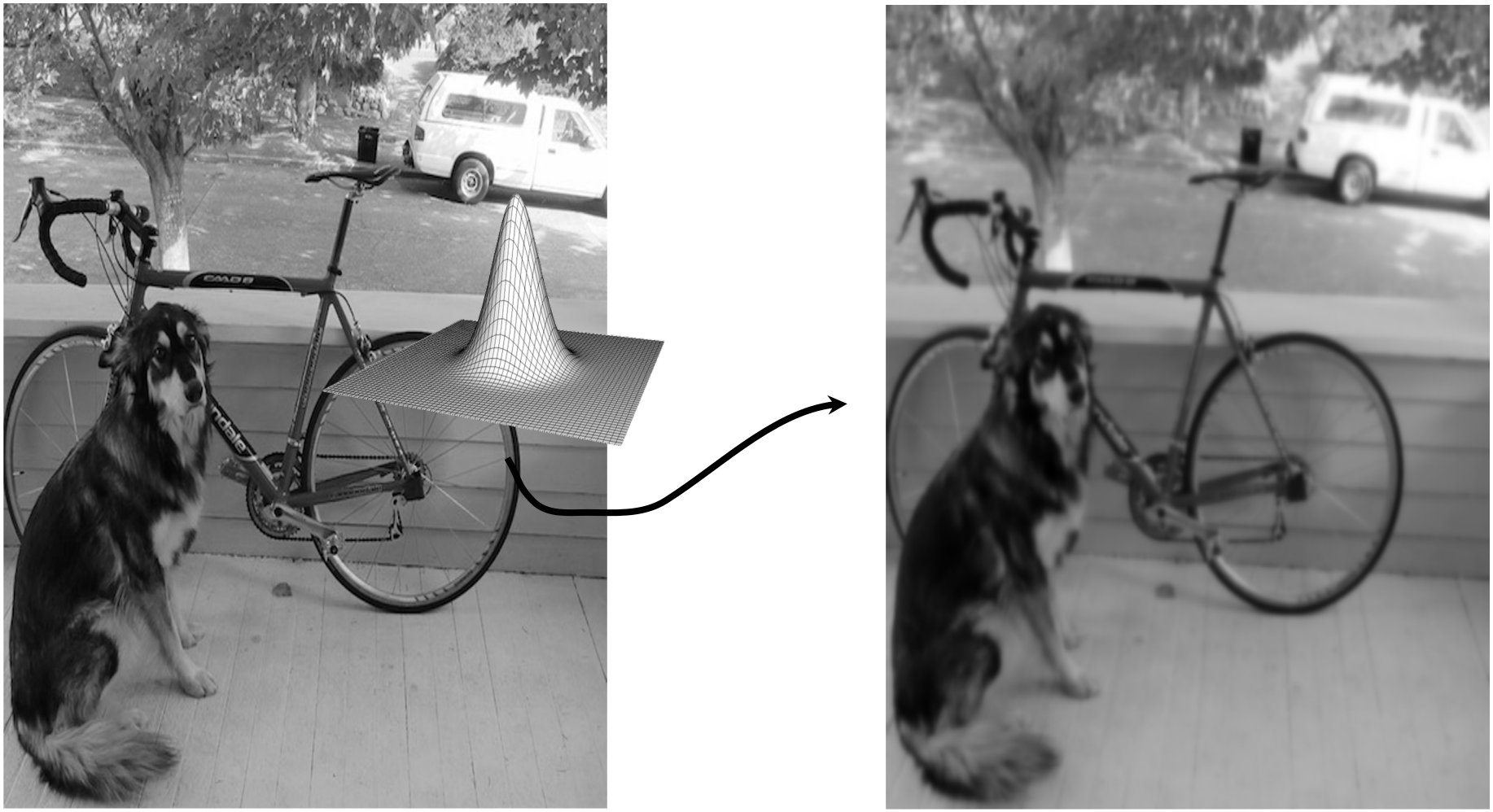
Canny Edge Detection

Algorithm:

- Smooth image (only want “real” edges, not noise)
- Calculate gradient direction and magnitude
- Non-maximum suppression perpendicular to edge
- Threshold into strong, weak, no edge
- Connect together components

Smooth image

- Apply a Gaussian filter

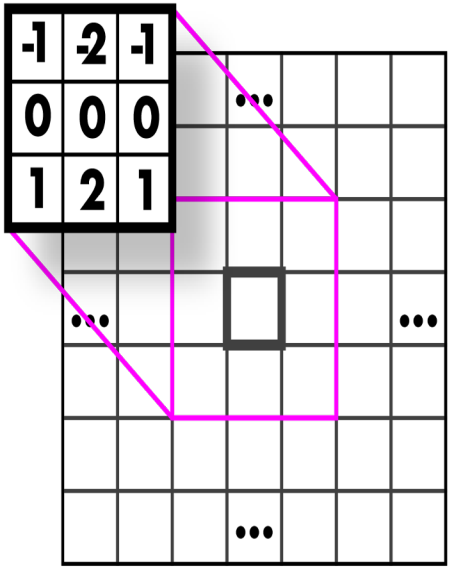


Gradient magnitude and direction

- Sobel filter

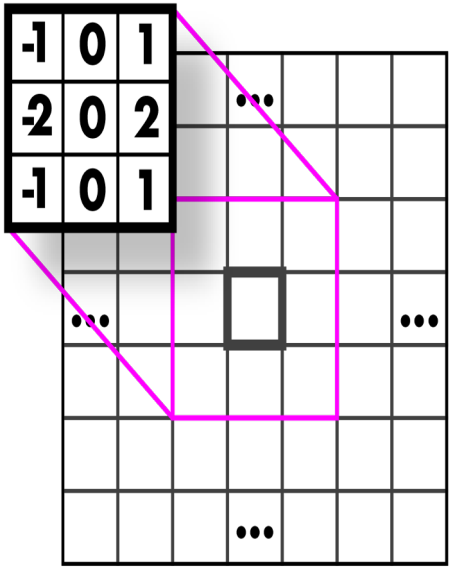
Horizontal Sobel filter for computing the partial derivative

$$\frac{\partial f(x,y)}{\partial y}$$



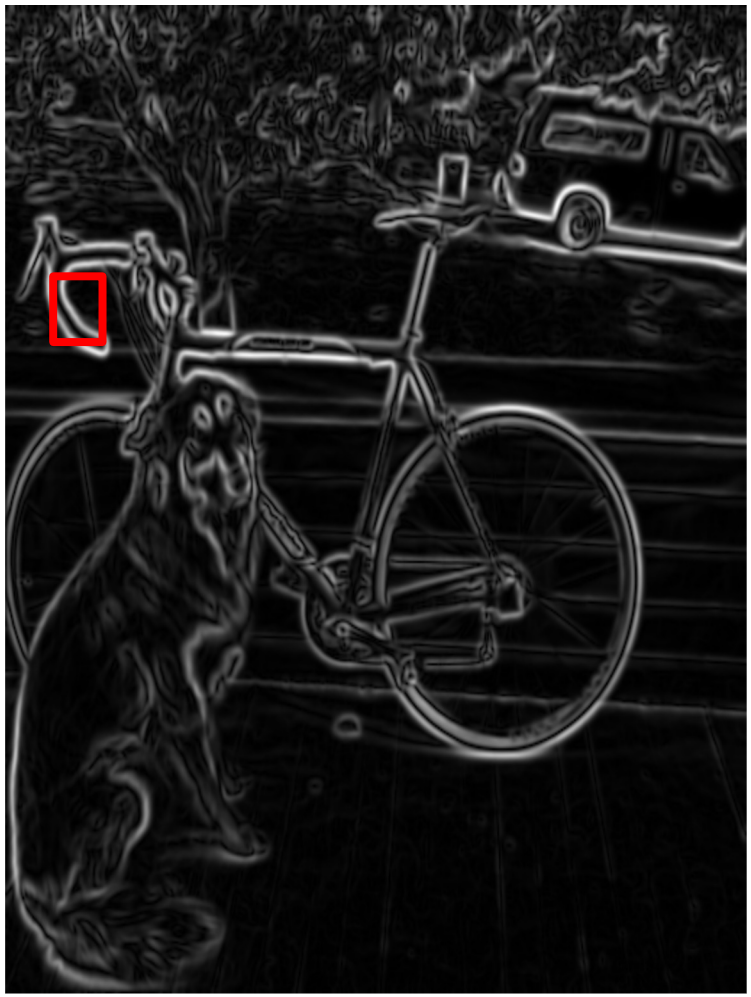
Vertical Sobel filter for computing the partial derivative

$$\frac{\partial f(x,y)}{\partial x}$$

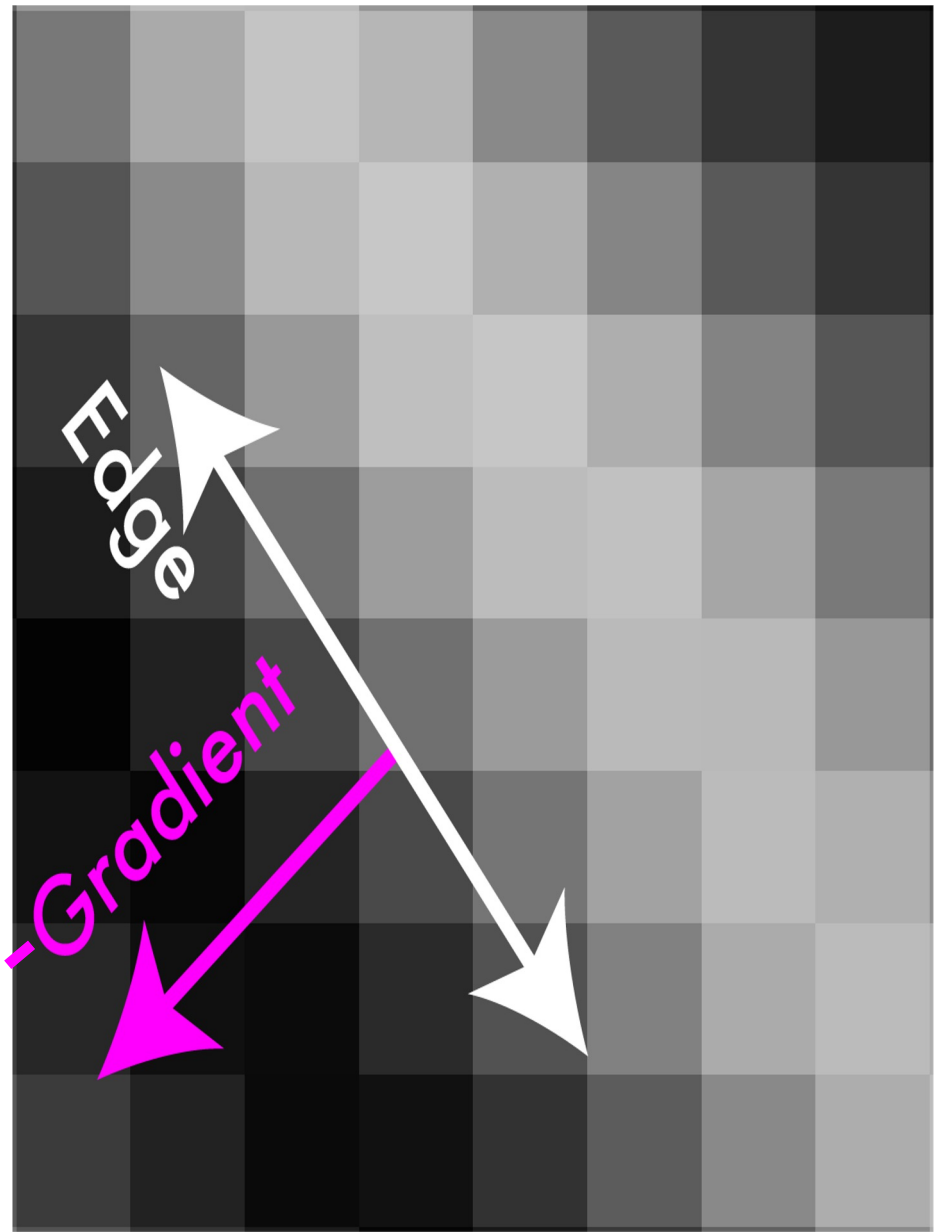


Non-maximum suppression

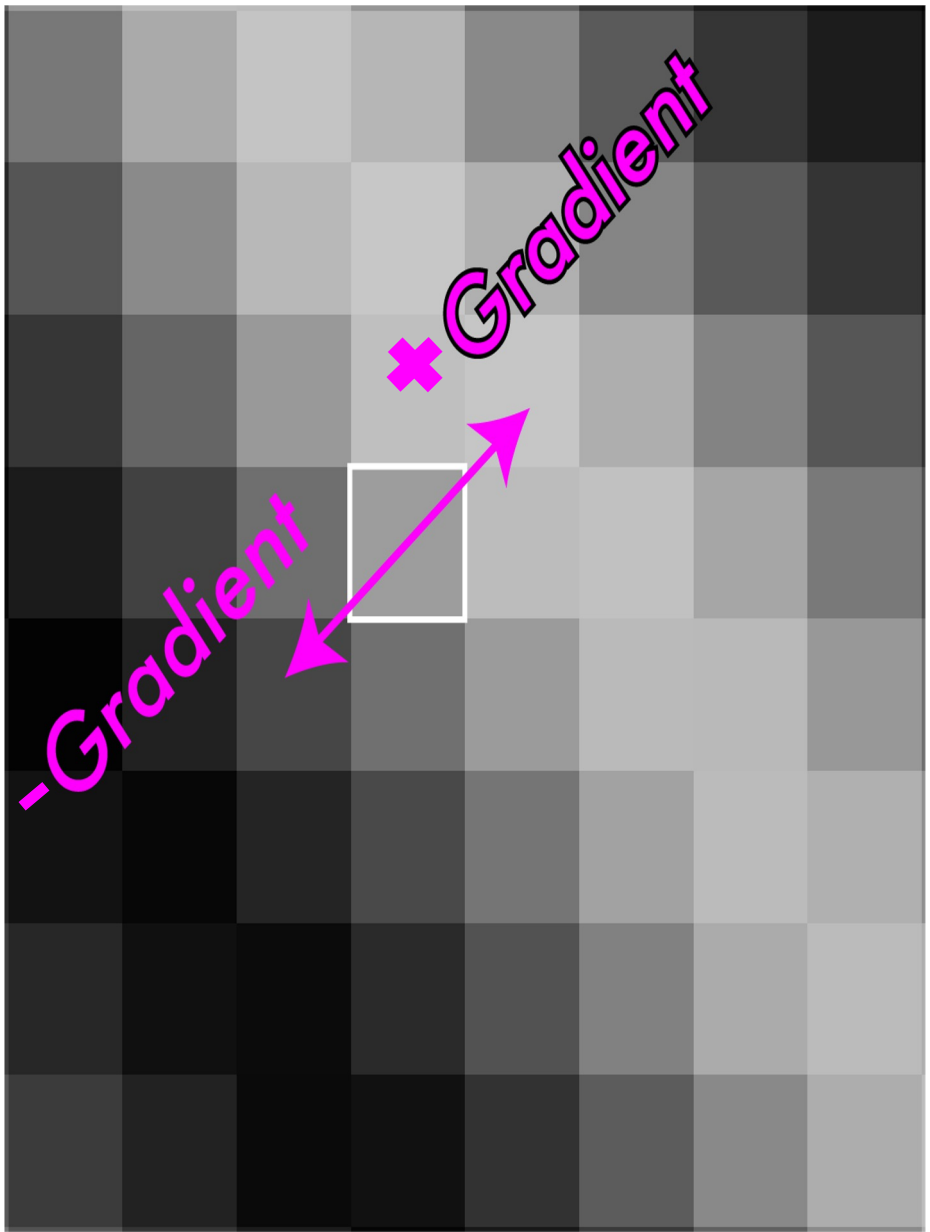
- Want single pixel edges, not thick blurry lines
- Need to check nearby pixels
- See if response is highest



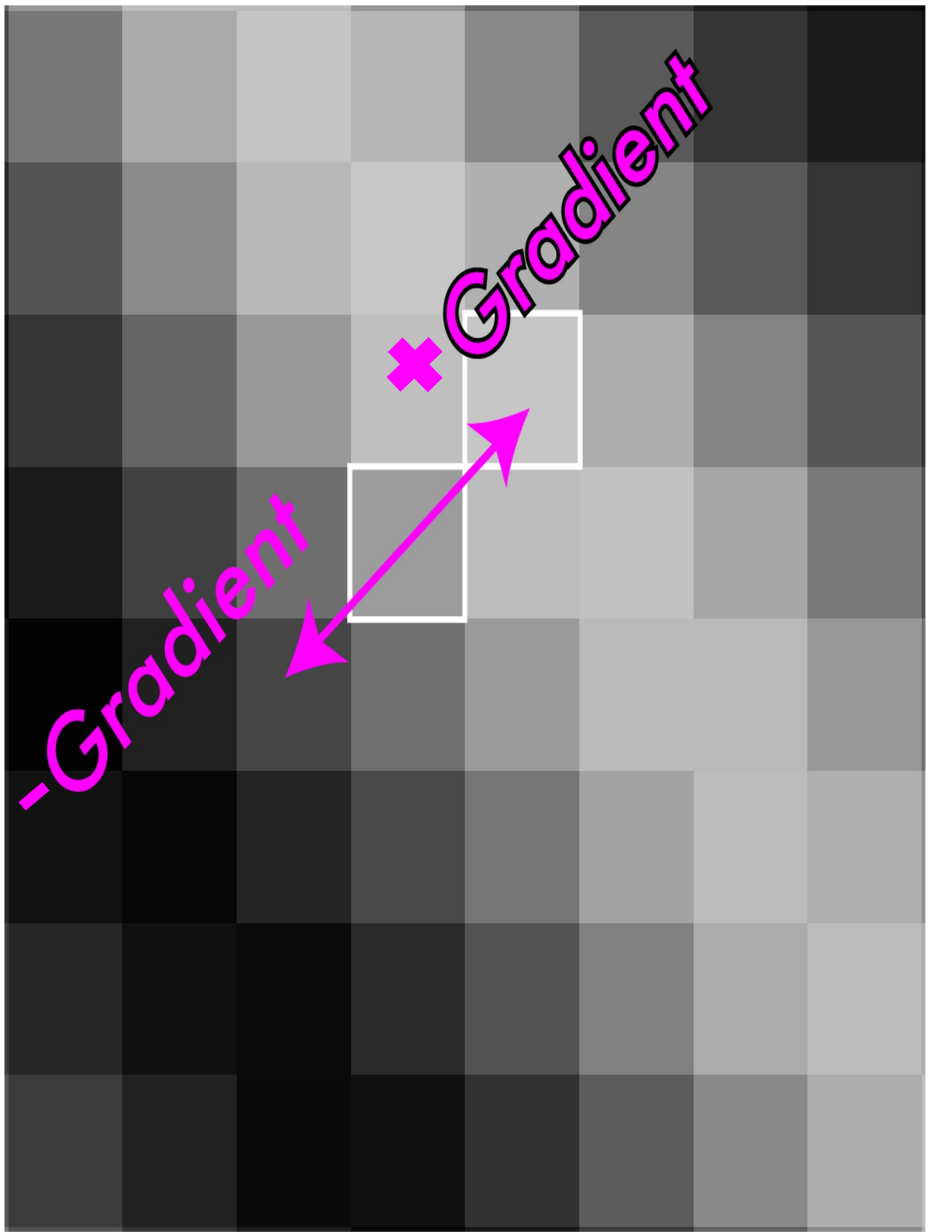
Non-maximum suppression



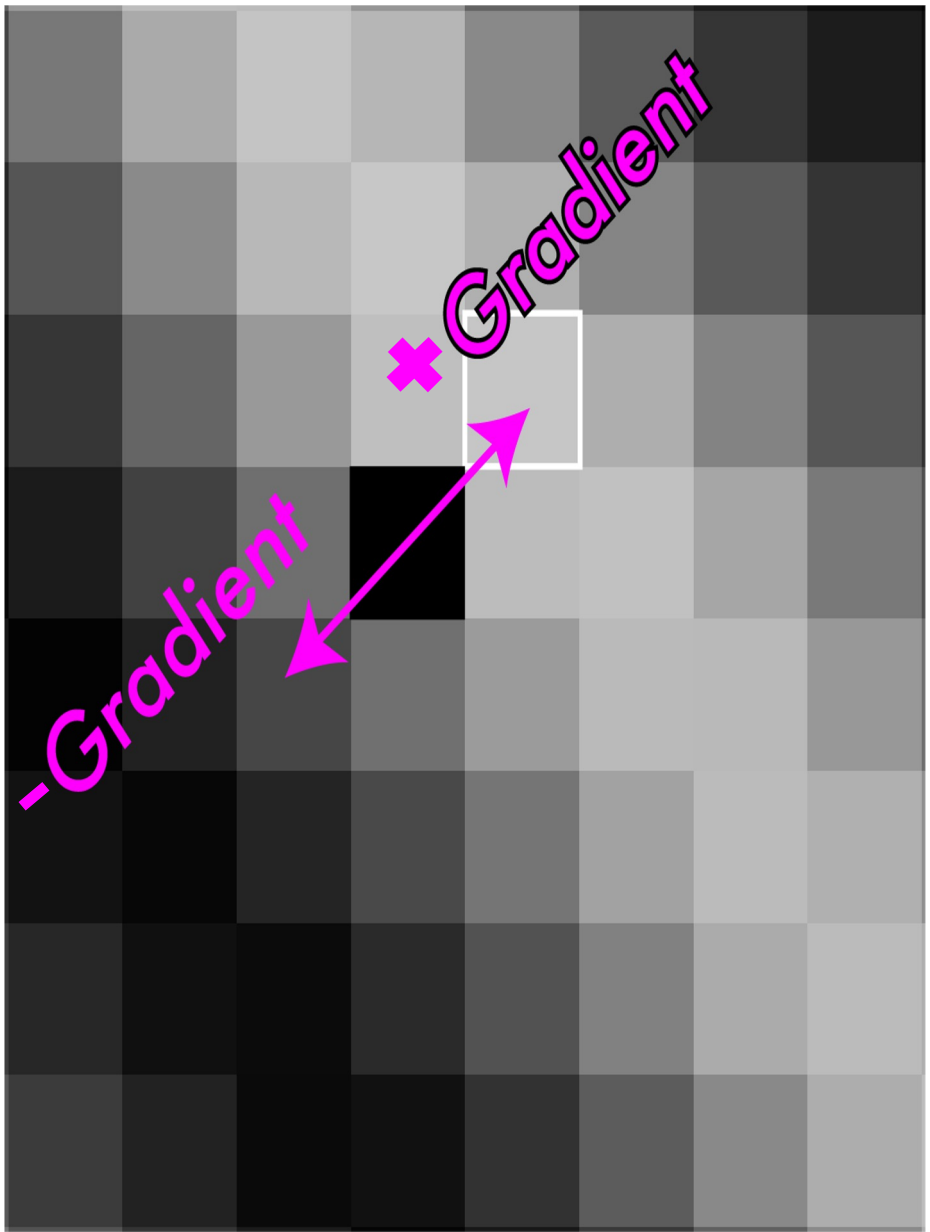
Non-maximum suppression



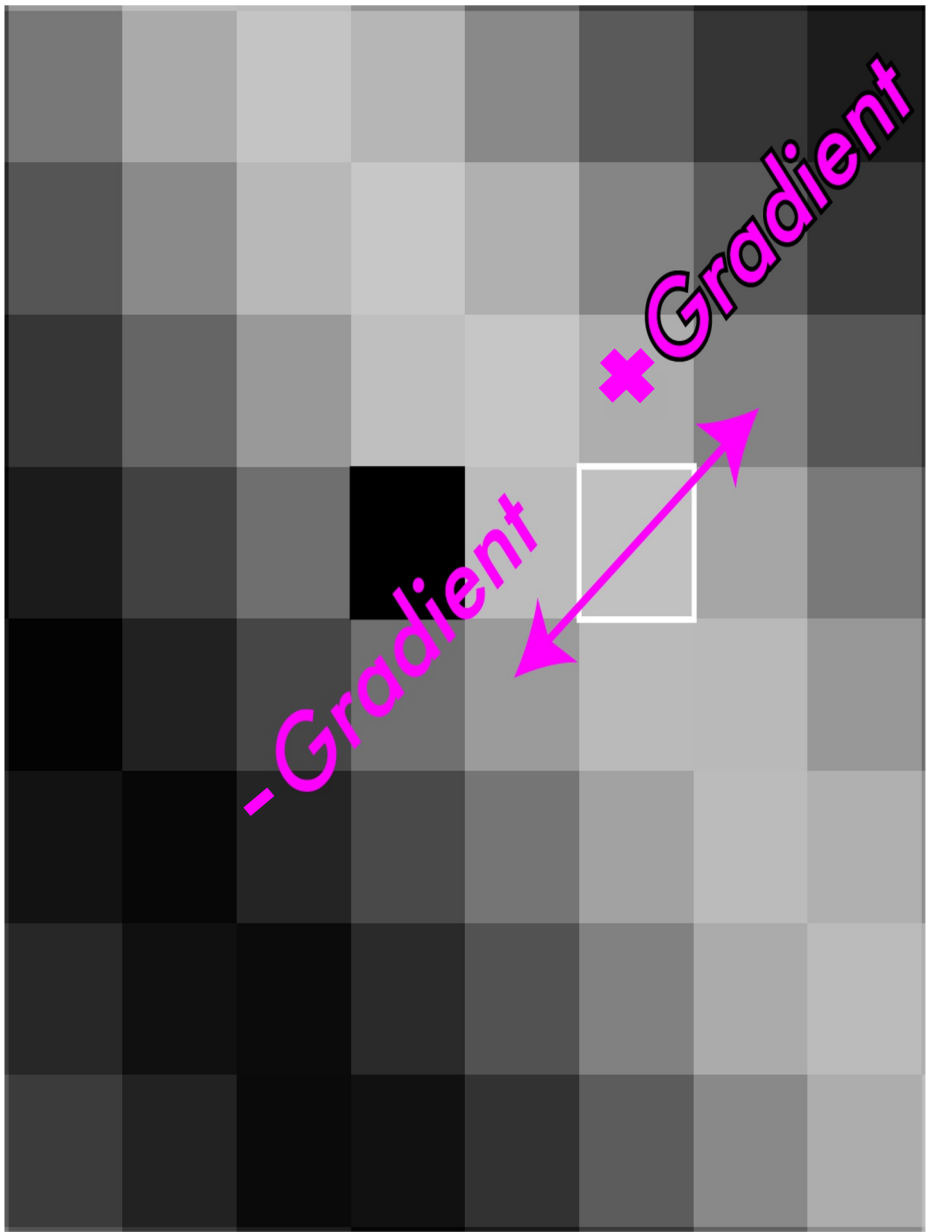
Non-maximum suppression



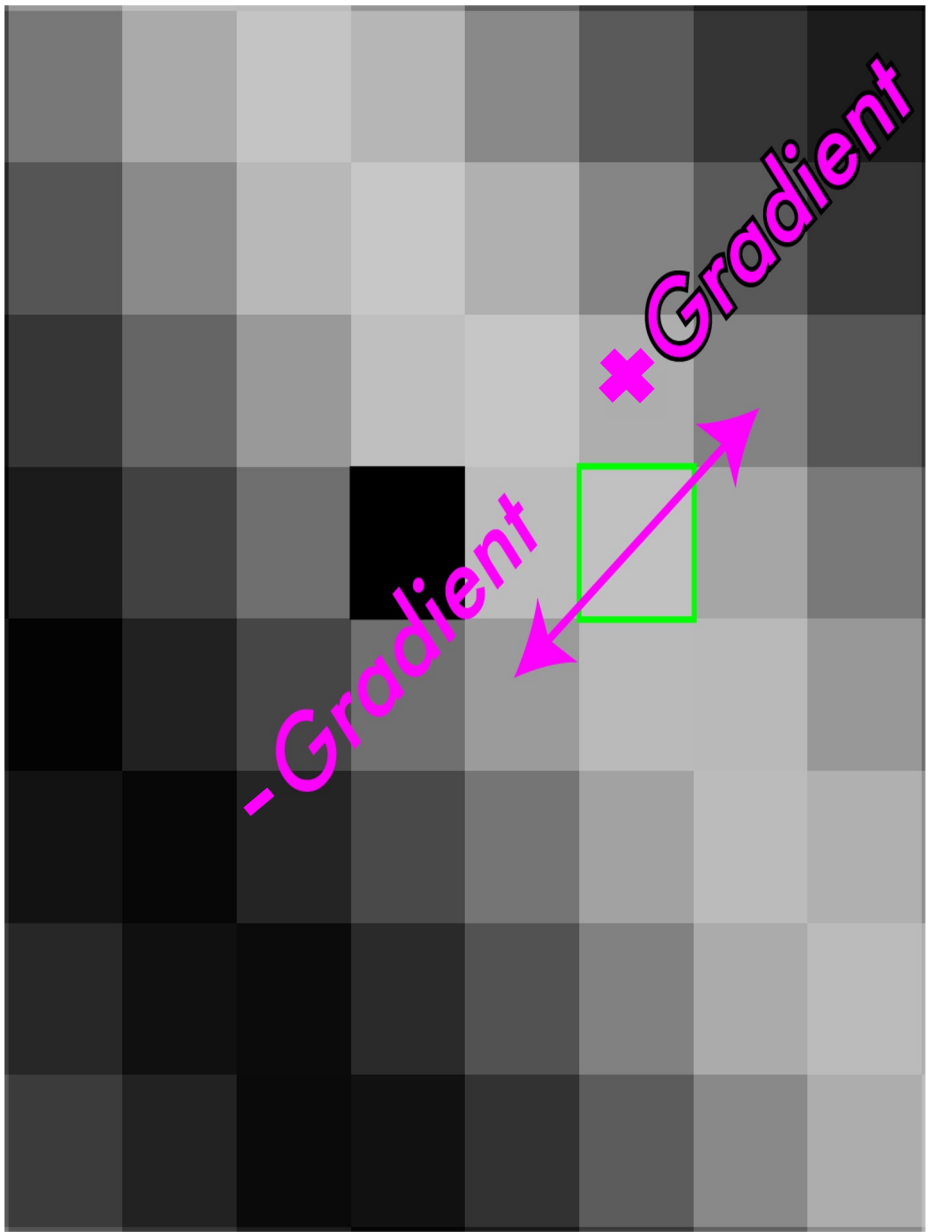
Non-maximum suppression



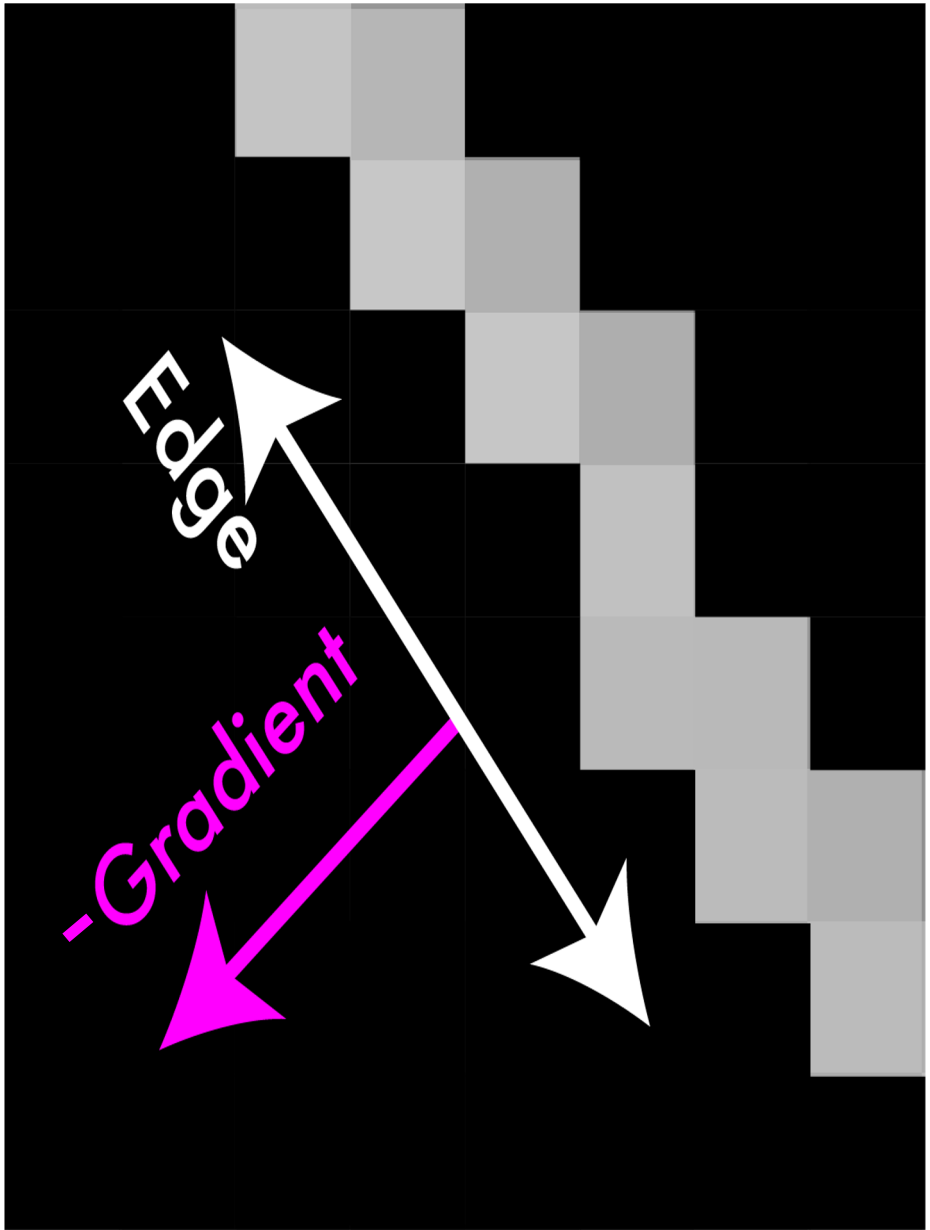
Non-maximum suppression



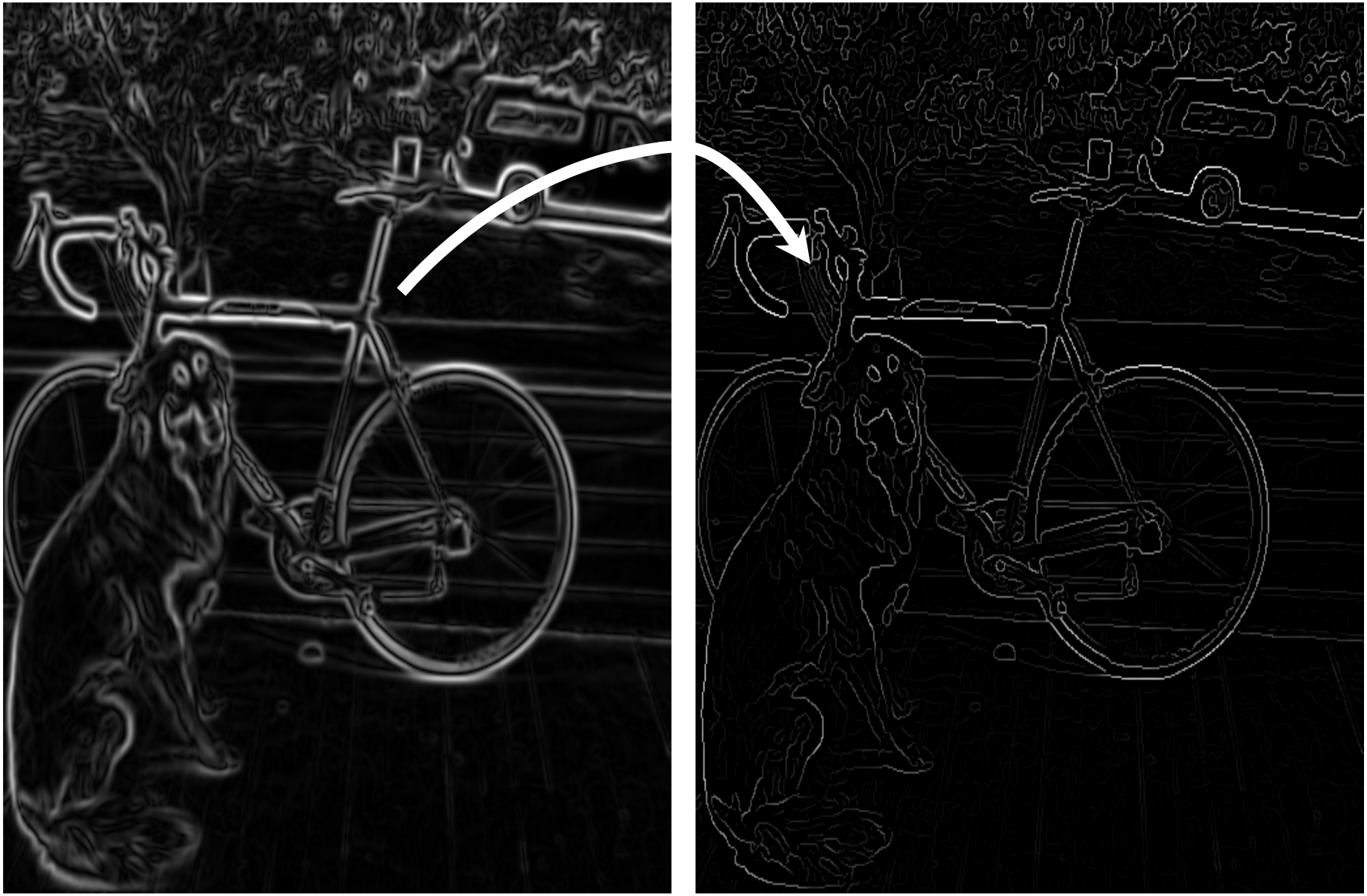
Non-maximum suppression



Non-maximum suppression



Non-maximum suppression



Hysteresis Thresholding

After non-maximum suppression, some noise may still remain.
We want to **keep only meaningful edges while discarding noise**.

Idea: Use **two thresholds** instead of one.
Let R be the gradient magnitude.

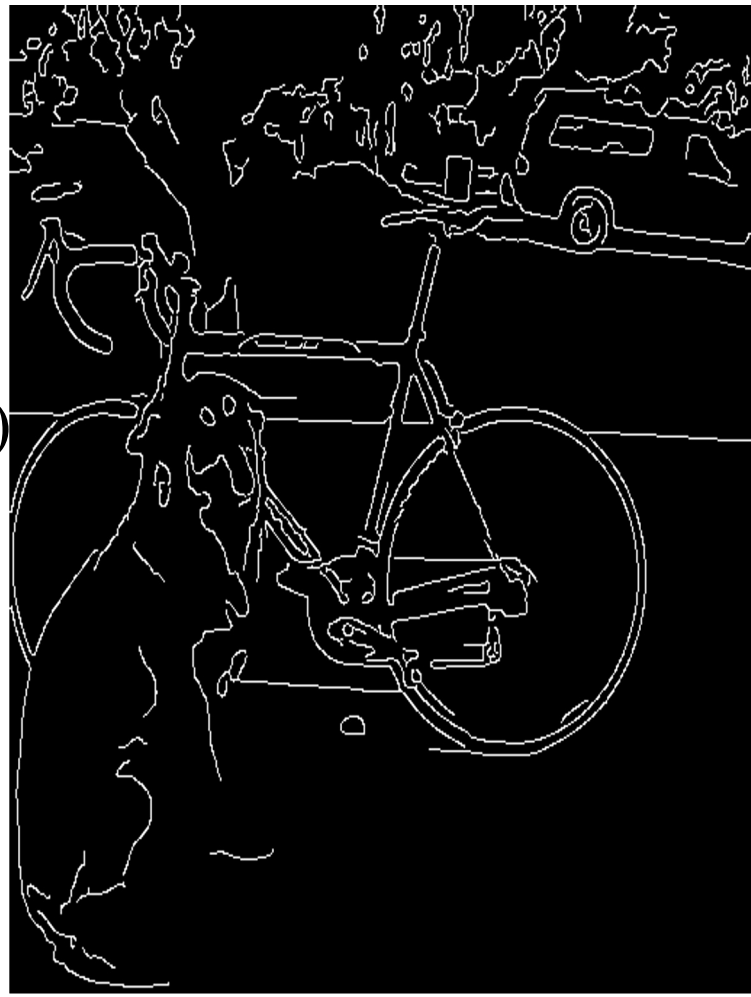
Three cases:

1. $R > T \rightarrow$ **Strong edge** (definitely an edge)
2. $t < R \leq T \rightarrow$ **Weak edge** (potential edge)
3. $R \leq t \rightarrow$ **Non-edge** (discard)

Why two thresholds?

Hysteresis Thresholding

After non-maximum suppression, some noise may still remain.
We want to **keep only meaningful edges while discarding noise**.



Idea: Use **two thresholds** instead of one.
Let R be the gradient magnitude.

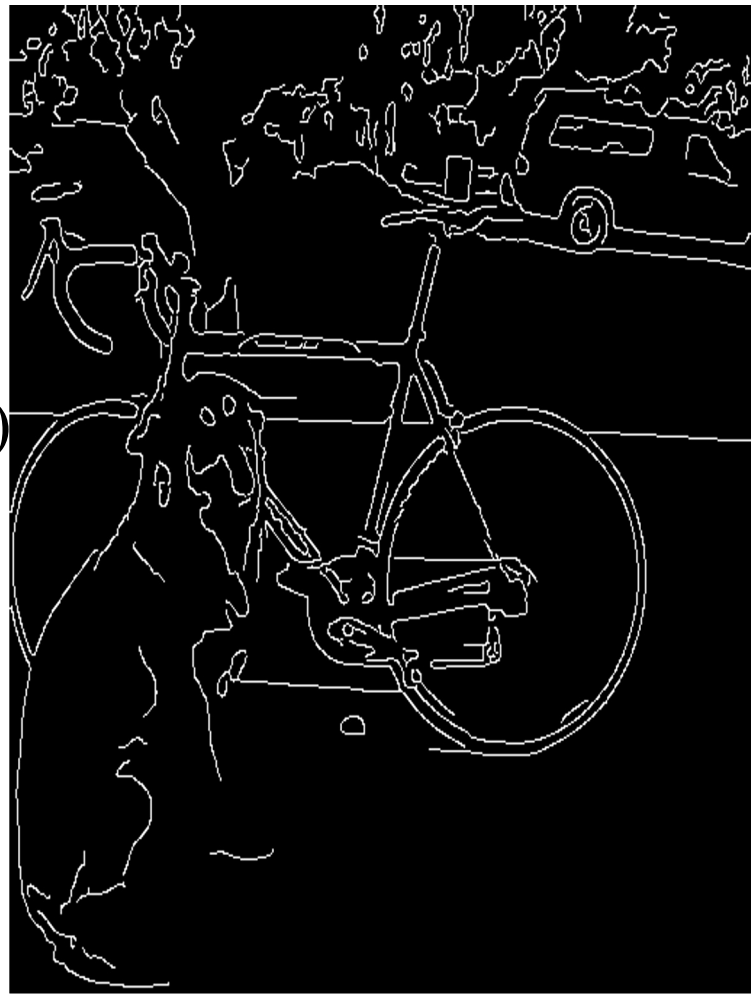
Three cases:

1. $R > T \rightarrow$ **Strong edge** (definitely an edge)
2. $t < R \leq T \rightarrow$ **Weak edge** (potential edge)
3. $R \leq t \rightarrow$ **Non-edge** (discard)

Why two thresholds?

Hysteresis Thresholding

After non-maximum suppression, some noise may still remain.
We want to **keep only meaningful edges while discarding noise**.



Idea: Use **two thresholds** instead of one.
Let R be the gradient magnitude.

Three cases:

1. $R > T \rightarrow$ **Strong edge** (definitely an edge)
2. $t < R \leq T \rightarrow$ **Weak edge** (potential edge)
3. $R \leq t \rightarrow$ **Non-edge** (discard)

Why two thresholds?

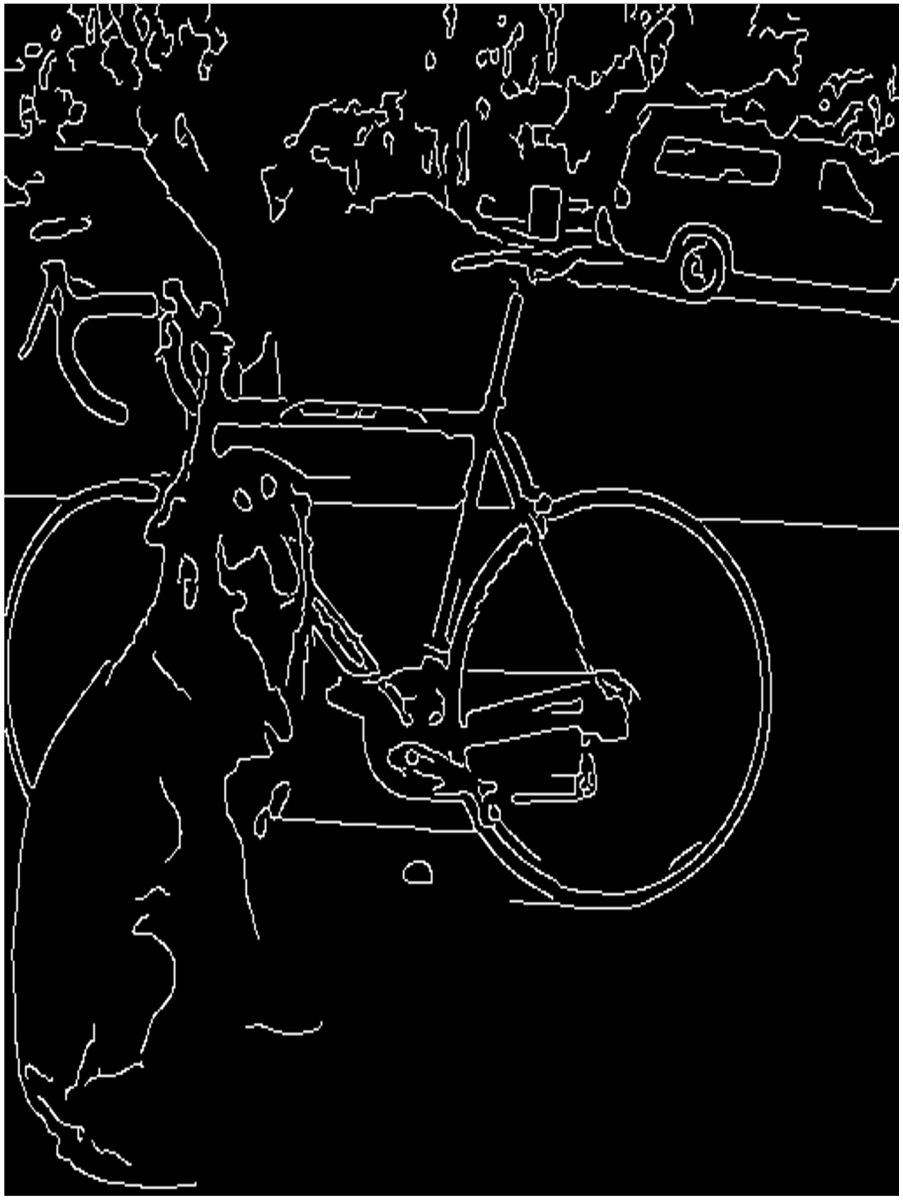
A **single threshold** may break real edges.
Weak edges are kept only if connected to strong edges (look in a neighborhood – usually 8 pixels).
This process is called **edge tracking by hysteresis**.

Canny Edge Detection

Algorithm:

- Smooth image (only want “real” edges, not noise)
- Calculate gradient direction and magnitude
- Non-maximum suppression perpendicular to edge
- Threshold into strong, weak, no edge
- Connect together components
- Tunable: Sigma, thresholds

Canny Edge Detection



Canny Edge Detection - demo

http://bigwww.epfl.ch/demo/ip/demos/edgeDetector/

Course structure

1. **Low-level vision: Features and filters**

   Linear filters, color, texture, edge detection, template matching

2. **Mid-level vision: Grouping and fitting**

   Fitting curves and lines, robust fitting, RANSAC, Hough transform, segmentation

3. **Mid-level vision: Multiple views**

   Local invariant feature and description, epipolar geometry and stereo, object instance recognition

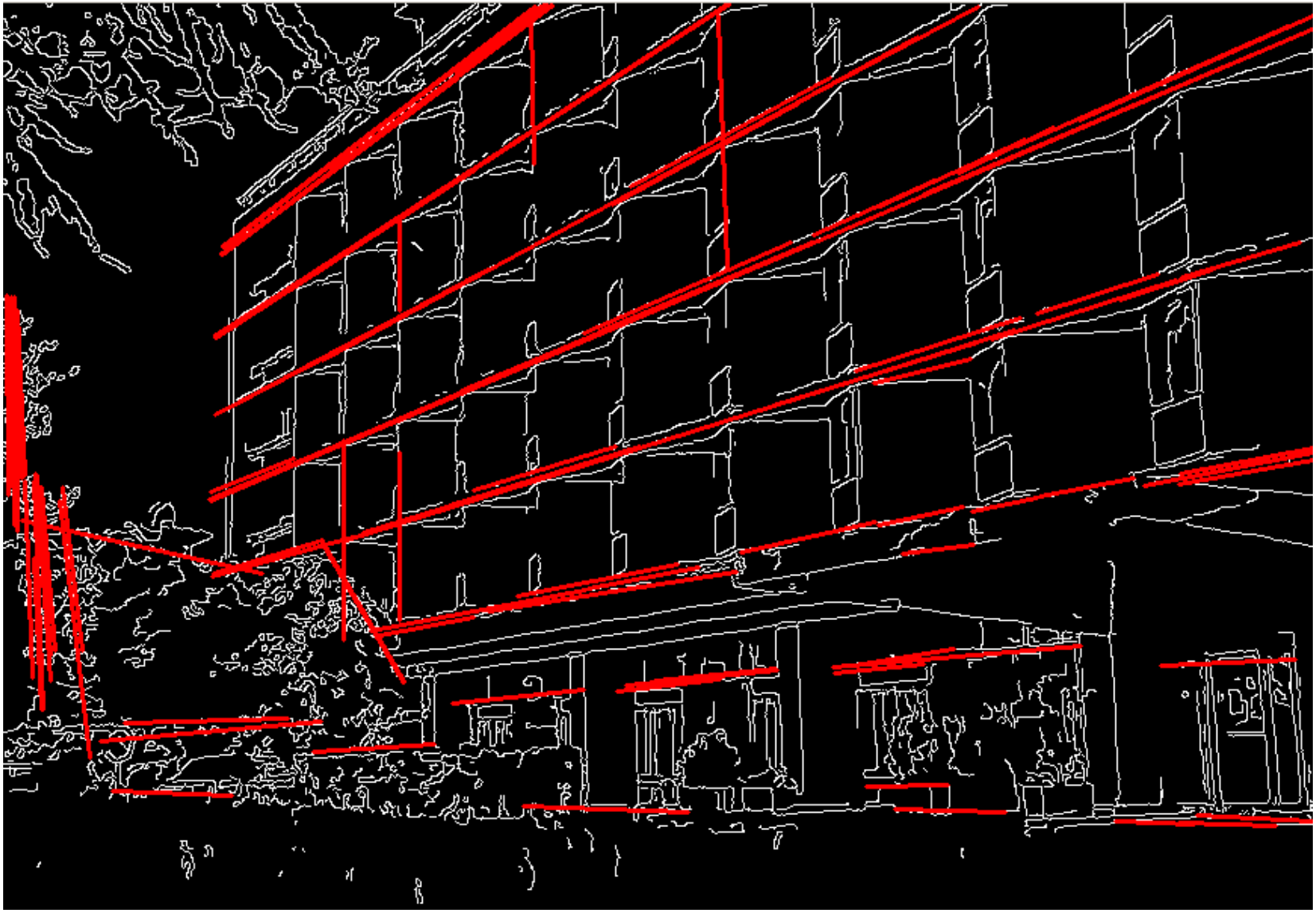
4. **High-level vision: Object recognition**

   Object classification, object detection, part based models, bovw models

5. **High-level vision: Video understanding**

   Object tracking, background subtraction, motion descriptors, optical flow

Fitting

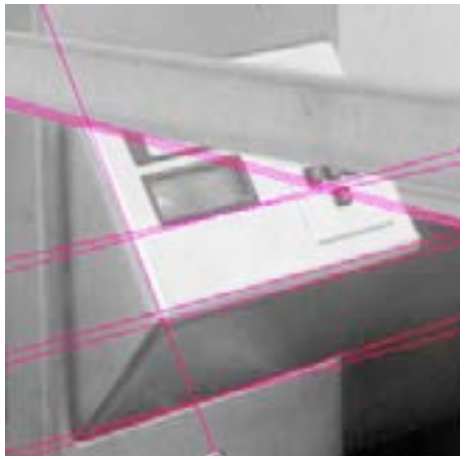


Fitting

- We have learned how to detect edges
- We would like to form a higher-level, more compact representation of the features in the image by grouping multiple features according to a simple model

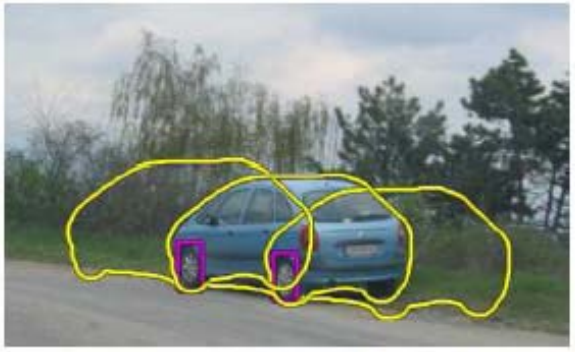
Fitting

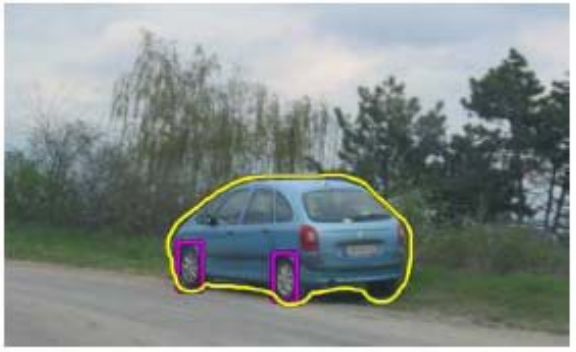
- Choose a *parametric model* to represent a set of features



simple model: lines

simple model: circles

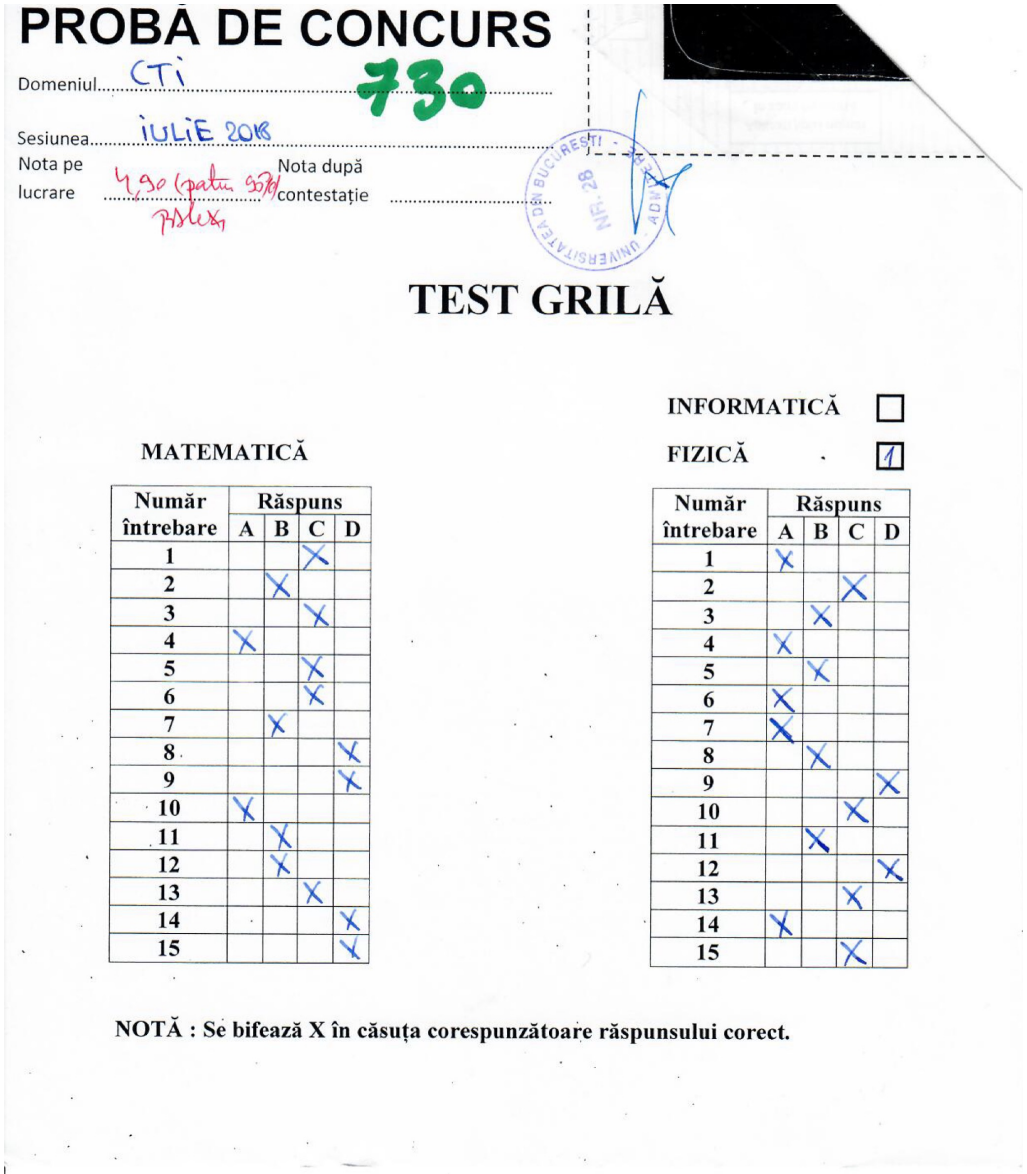




complex model: car

Slide credit : K. Grauman

Lab 2: Line detection - application



PROBĂ DE CONCURS

Domeniul CTI 730

Sesiunea IULIE 2018

Nota pe lucrare 4,90 (patru 90/100) Nota după contestație .....

TEST GRILĂ

INFORMATICĂ ☐

FIZICĂ ☒

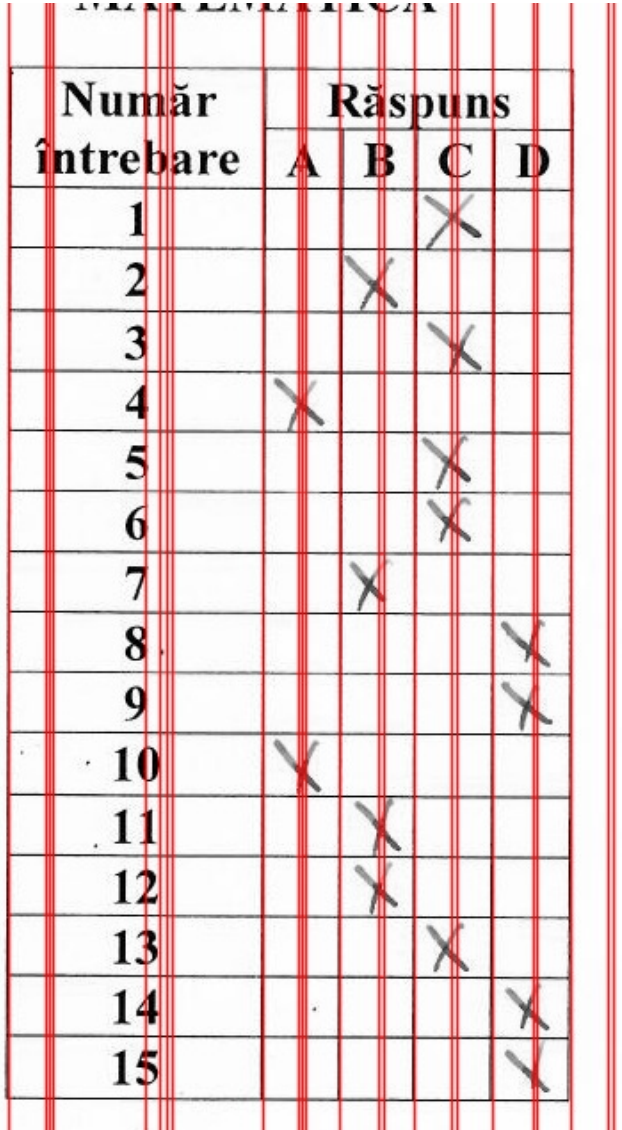
MATEMATICĂ

Număr întrebare	Răspuns A	B	C	D
1			X	
2		X		
3			X	
4	X			
5			X	
6			X	
7		X		
8				X
9				X
10	X			
11		X		
12		X		
13			X	
14				X
15				X

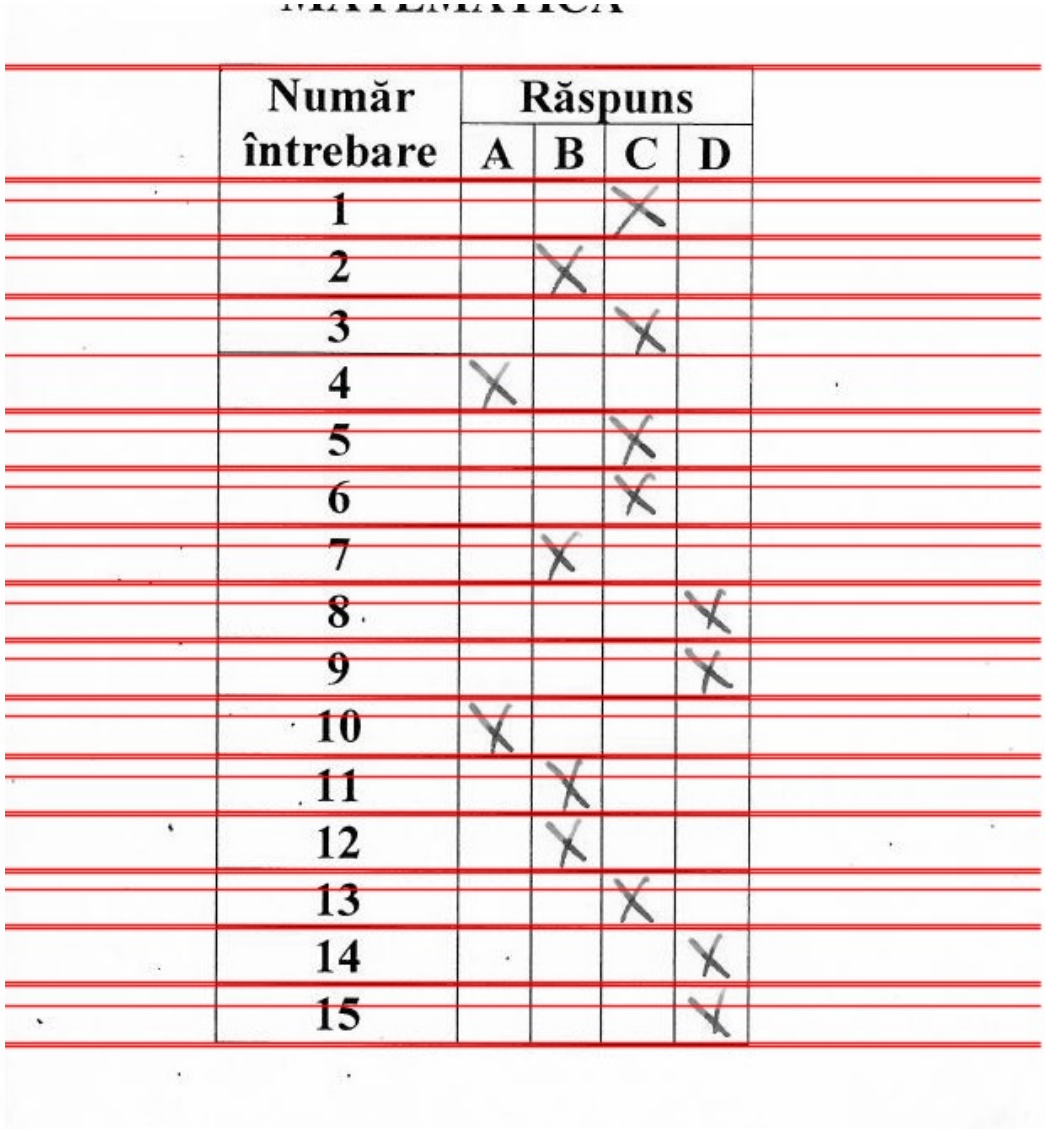
Număr întrebare	Răspuns A	B	C	D
1	X			
2			X	
3		X		
4	X			
5		X		
6	X			
7	X			
8		X		
9				X
10			X	
11		X		
12				X
13			X	
14	X			
15			X	

NOTĂ : Se bifează X în căsuța corespunzătoare răspunsului corect.

Lab 2: Line detection - application

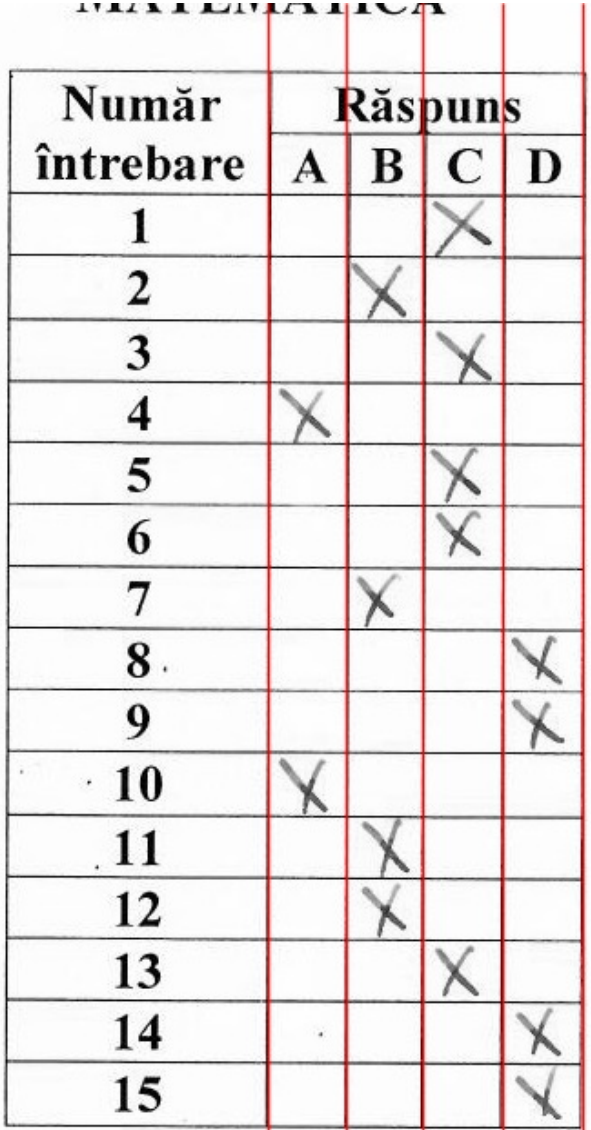


Număr întrebare	Răspuns A	B	C	D
1			X	
2		X		
3			X	
4	X			
5			X	
6			X	
7		X		
8				X
9				X
10	X			
11		X		
12		X		
13			X	
14				X
15				X

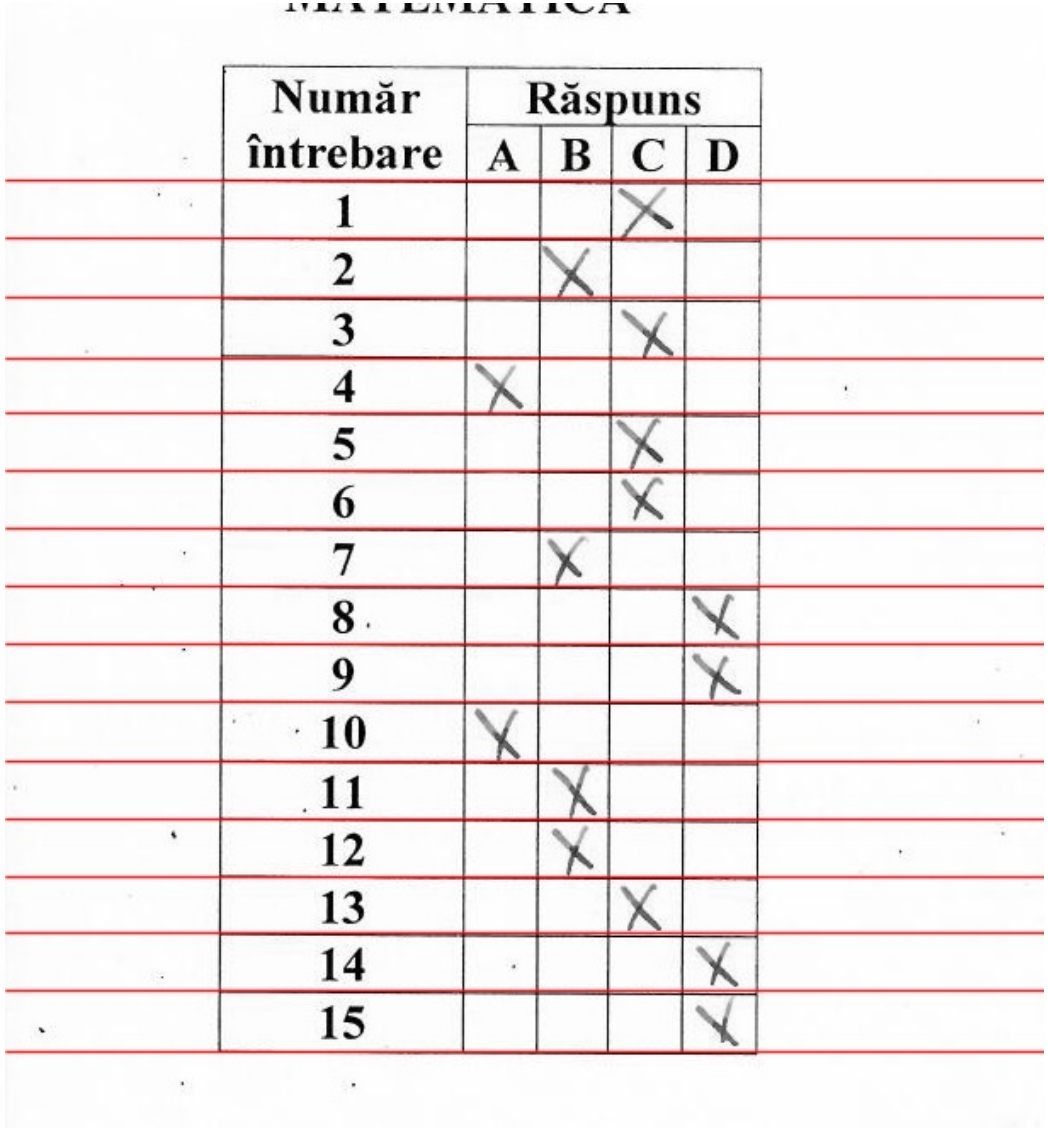


Număr întrebare	Răspuns A	B	C	D
1			X	
2		X		
3			X	
4	X			
5			X	
6			X	
7		X		
8				X
9				X
10	X			
11		X		
12		X		
13			X	
14				X
15				X

Lab 2: Line detection - application



Număr întrebare	Răspuns			
	A	B	C	D
1			X	
2		X		
3			X	
4	X			
5			X	
6			X	
7		X		
8				X
9				X
10	X			
11		X		
12		X		
13			X	
14				X
15				X



Număr întrebare	Răspuns			
	A	B	C	D
1			X	
2		X		
3			X	
4	X			
5			X	
6			X	
7		X		
8				X
9				X
10	X			
11		X		
12		X		
13			X	
14				X
15				X

Fitting: challenges

- If we know which points belong to the line, how do we find the "optimal" line parameters?
  - Least squares
- What if there are outliers?
  - Robust fitting, RANSAC
- What if there are many lines?
  - Voting methods: RANSAC, Hough transform

Slide credit : S. Lazebnik

Voting schemes

- Let each feature vote for all the models that are compatible with it
  - Hopefully the noise features will not vote consistently for any single model
  - Missing data doesn't matter as long as there are enough features remaining to agree on a good model

Slide credit : S. Lazebnik

Hough transform

- An early type of voting scheme
  - Discretize *parameter space* into bins
  - For each feature point in the image, put a vote in every bin in the parameter space that could have generated this point
  - Find bins that have the most votes

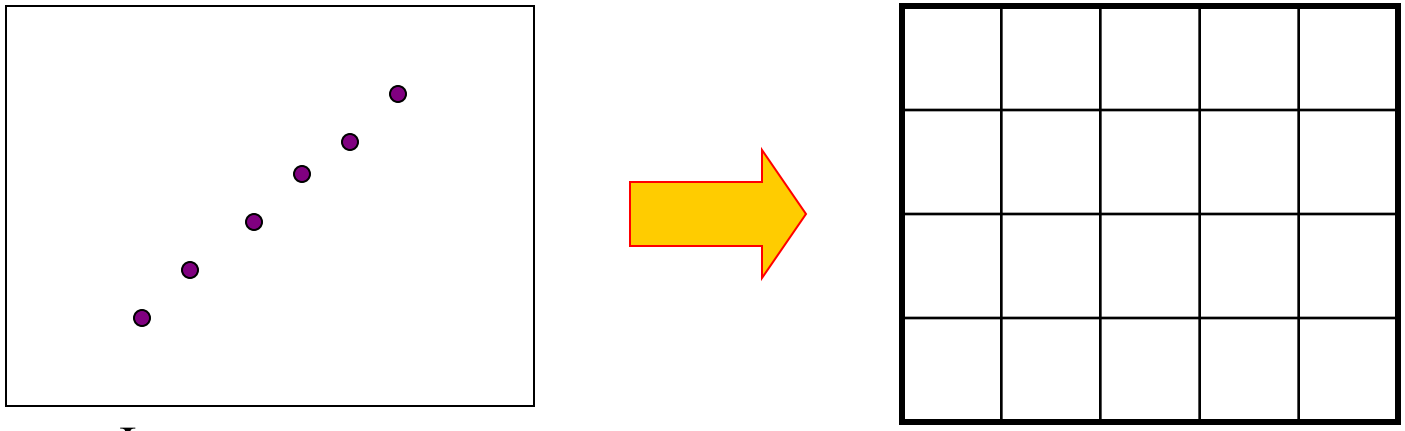


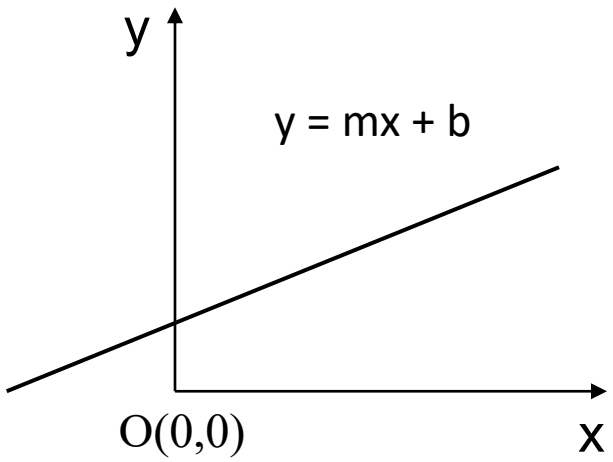
Image space

Hough parameter space

P.V.C. Hough, *Machine Analysis of Bubble Chamber Pictures*,
Proc. Int. Conf. High Energy Accelerators and Instrumentation, 1959

Slide credit : S. Lazebnik

Parameterization of a line



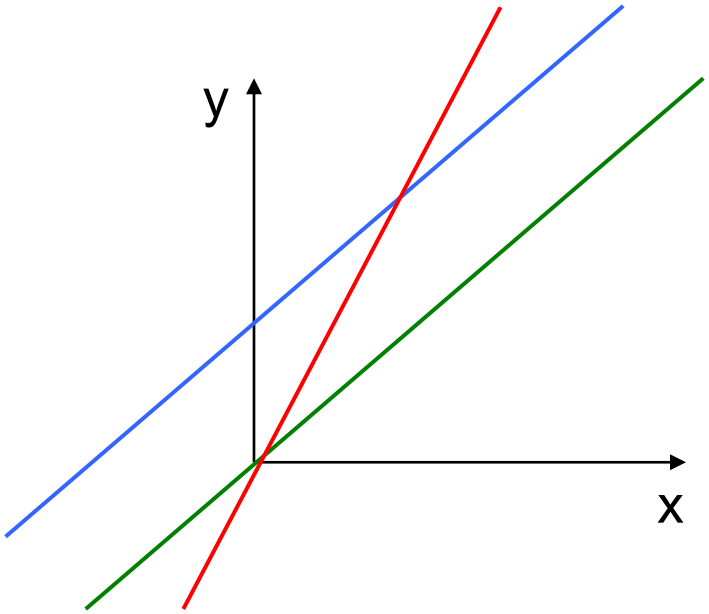
Parameterization of a line in image plane: $y = m * x + b$,

m – slope (defines the angle between the line and Ox axis),

b – intercept (bias to origin O(0,0))

(m,b) are line parameters

Parameterization of a line - examples



d_1: y = x, so m=1, b = 0

d_2: y = x + 2, so m=1, b = 2

d_3: y = 2x, so m=2, b = 0

Parameterization of a line in image plane: $y = m * x + b$,

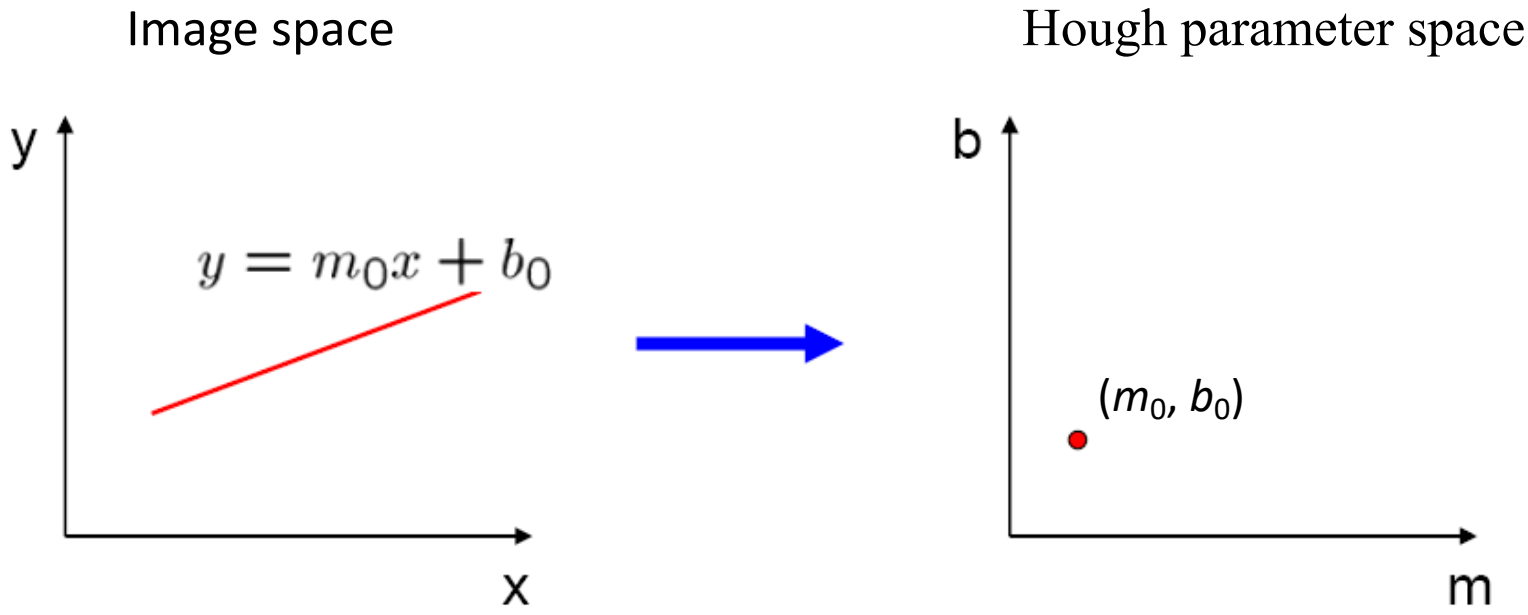
m – slope (defines the angle between the line and Ox axis),

b – intercept (bias to origin O(0,0))

(m,b) are line parameters

Parameter space representation

- A **line** in the image corresponds to a **point** in Hough space



Slide credit : S. Seitz

Hough parameter space

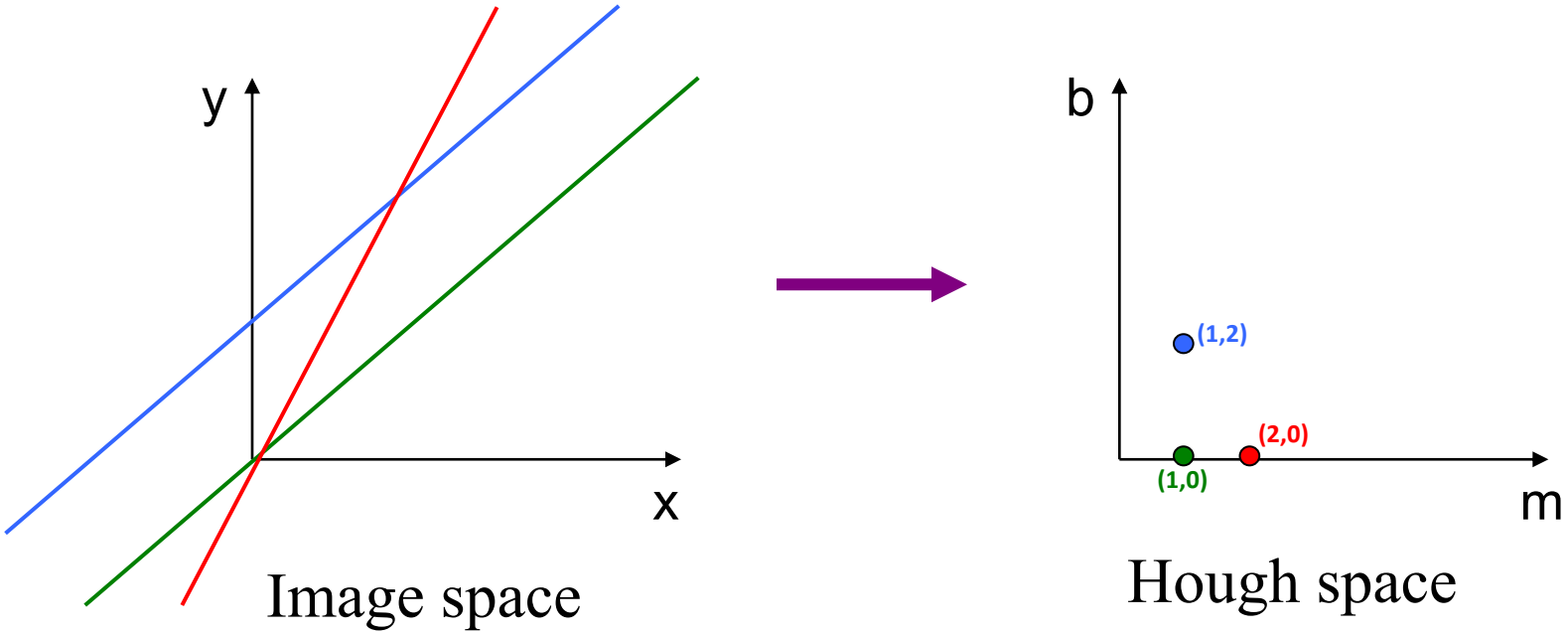


Image space

Hough space

d_1: y = x, so m=1, b = 0

d_1: m=1, b = 0 -> (1,0)

d_2: y = x + 2, so m=1, b = 2

d_2: m=1, b = 2 -> (1,2)

d_3: y = 2x, so m=2, b = 0

d_3: m=2, b = 0 -> (2,0)

A line in the image space corresponds to a point in the Hough space.

Correspondence between image and Hough spaces

A line in the image space corresponds to a point in the Hough space.

What does a **point** (x_0, y_0) in the image space map to in the Hough space?

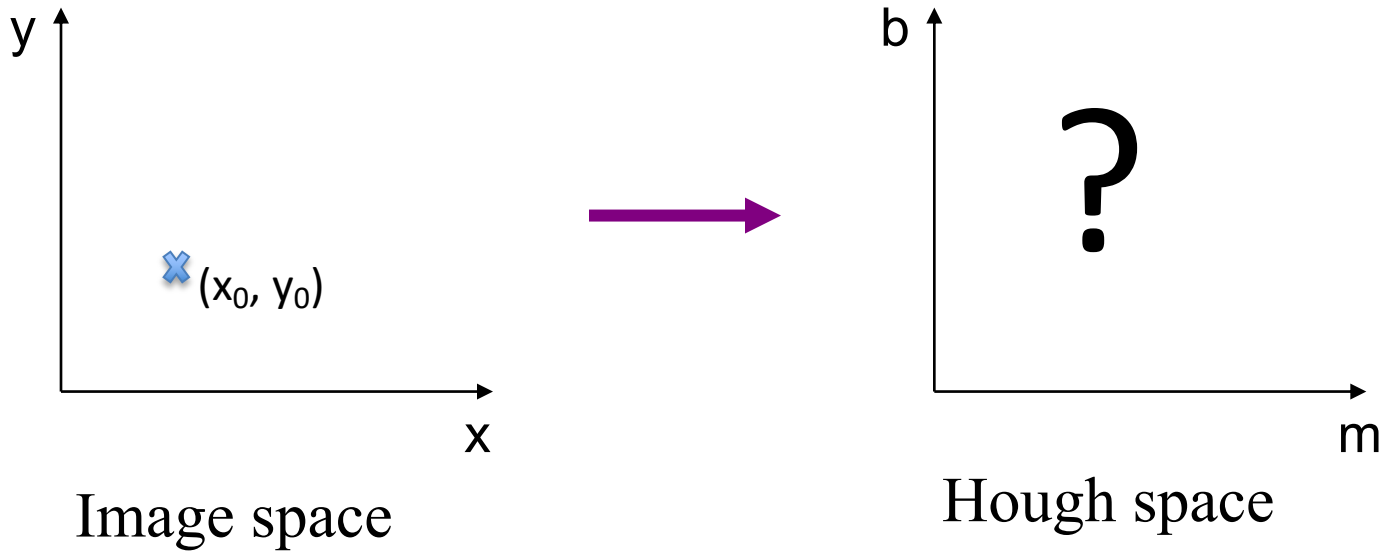


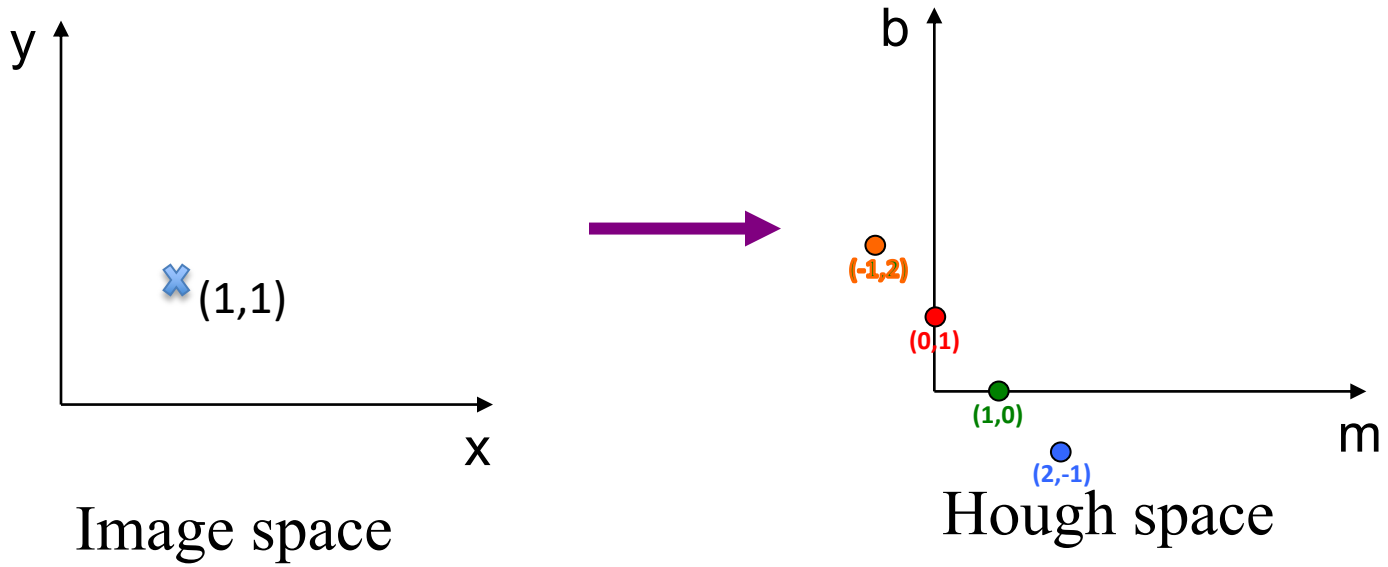
Image space

Hough space

Correspondence between image and Hough spaces

A line in the image space corresponds to a point in the Hough space.

What does a **point** (x_0, y_0) in the image space map to in the Hough space?



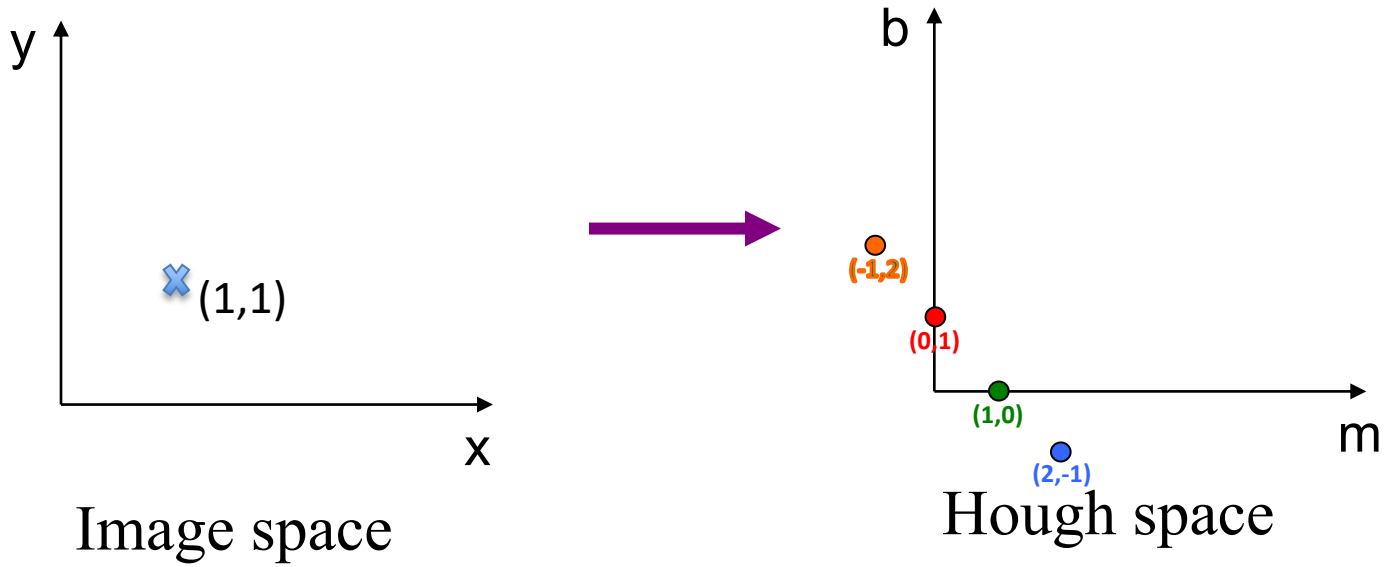
There is an infinity of lines of the form y = mx+b that contain point (1,1):

y = x (m=1, b=0); y = 2x-1 (m=2,b=-1), y = 1 (m=0, b = 1), y = -x + 2 (m=-1, b =2),

Correspondence between image and Hough spaces

A line in the image space corresponds to a point in the Hough space.

A **point** (x_0, y_0) in the image space maps to a line in the Hough space



There is an infinity of lines of the form y = mx+b that contain point (1,1):

y = x (m=1, b=0); y = 2x-1 (m=2,b=-1), y = 1 (m=0, b = 1), y = -x + 2 (m=-1, b =2),

All 4 points (1,0), (2,-1), (0,-1), (-1,2) are on the line b = -m+1

Correspondence between image and Hough spaces

A line in the image space corresponds to a point in the Hough space.

A **point** (x_0, y_0) in the image space maps to a line in the Hough space

Parameterization of a line in image plane: $y = m * x + b$,

All lines that contain point (x_0,y_0) can have any slope m and any intercept b, the only condition that should be satisfied is

$y_0 = m * x_0 + b$.

$y_0 = m * x_0 + b \Leftrightarrow b = -x_0 * m + y_0$ (parameterization of the line in the Hough space with slope $-x_0$ and intercept y_0)

$(x_0,y_0) = (1,1) \Rightarrow b = -m+1$

Parameter space representation

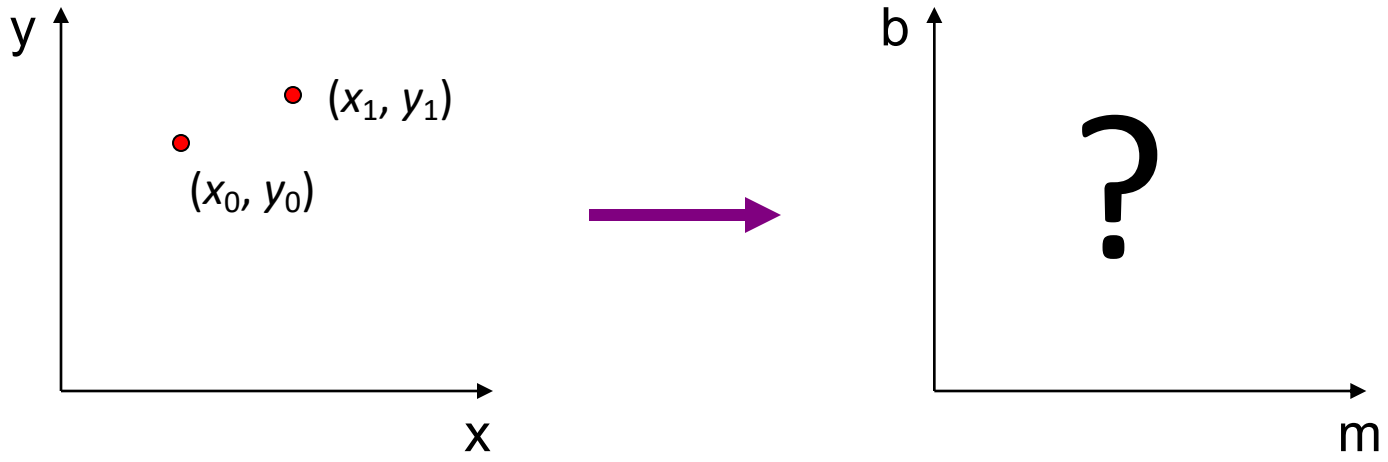


Image space

Hough space

Where is the line that contains both (x_0, y_0) and (x_1, y_1)?

Parameter space representation

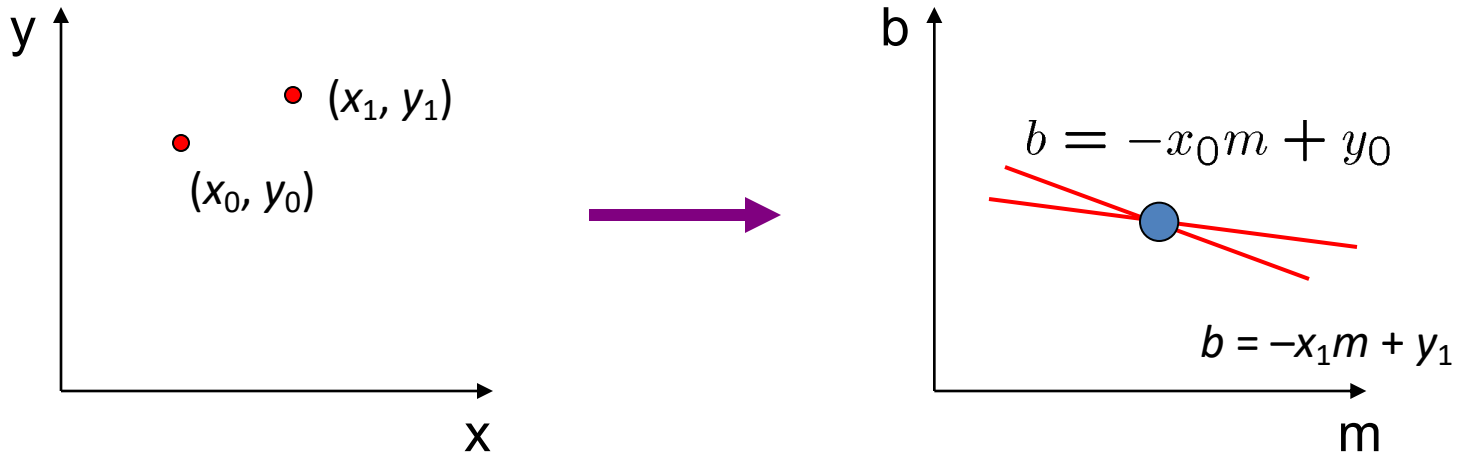


Image space

Hough space

Where is the line that contains both (x_0, y_0) and (x_1, y_1)?

Point (x_0,y_0) maps to the line in the Hough space: $b = -x_0 m + y_0$

Point (x_1,y_1) maps to the line in the Hough space: $b = -x_1 m + y_1$

The line that contains both (x_0, y_0) and (x_1, y_1) maps to a point in the Hough space. This point is the intersection of lines $b = -x_0 m + y_0$ și $b = -x_1 m + y_1$